In [1]:
```
import pandas
import os

# This query represents dataset "M2_V1" for domain "person" and was generated for A
dataset_31550912_person_sql = """
    SELECT
        person.person_id,
        p_gender_concept.concept_name as gender,
        person.birth_datetime as date_of_birth,
        p_race_concept.concept_name as race,
        p_ethnicity_concept.concept_name as ethnicity,
        p_sex_at_birth_concept.concept_name as sex_at_birth
    FROM
        `""" + os.environ["WORKSPACE_CDR"] + """.person` person
    LEFT JOIN
        `""" + os.environ["WORKSPACE_CDR"] + """.concept` p_gender_concept
            ON person.gender_concept_id = p_gender_concept.concept_id
    LEFT JOIN
        `""" + os.environ["WORKSPACE_CDR"] + """.concept` p_race_concept
            ON person.race_concept_id = p_race_concept.concept_id
    LEFT JOIN
        `""" + os.environ["WORKSPACE_CDR"] + """.concept` p_ethnicity_concept
            ON person.ethnicity_concept_id = p_ethnicity_concept.concept_id
    LEFT JOIN
        `""" + os.environ["WORKSPACE_CDR"] + """.concept` p_sex_at_birth_concept
            ON person.sex_at_birth_concept_id = p_sex_at_birth_concept.concept_id
    WHERE
        person.PERSON_ID IN (SELECT
            distinct person_id
        FROM
            `""" + os.environ["WORKSPACE_CDR"] + """.cb_search_person` cb_search_pe
        WHERE
            cb_search_person.person_id IN (SELECT
                person_id
            FROM
                `""" + os.environ["WORKSPACE_CDR"] + """.cb_search_person` p
            WHERE
                DATE_DIFF(CURRENT_DATE, dob, YEAR) - IF(EXTRACT(MONTH FROM dob)*100
                AND NOT EXISTS (      SELECT
                    'x'
                FROM
                    `""" + os.environ["WORKSPACE_CDR"] + """.death` d
                WHERE
                    d.person_id = p.person_id ) )
            AND cb_search_person.person_id IN (SELECT
                criteria.person_id
            FROM
                (SELECT
                    DISTINCT person_id, entry_date, concept_id
                FROM
                    `""" + os.environ["WORKSPACE_CDR"] + """.cb_search_all_events`
                WHERE
                    (concept_id IN (1586213)
                    AND is_standard = 0
```

```
                    AND  value_source_concept_id IN (1585634)
                    OR  concept_id IN (1586213)
                    AND is_standard = 0
                    AND  value_source_concept_id IN (1586217)
                    OR  concept_id IN (1586213)
                    AND is_standard = 0
                    AND  value_source_concept_id IN (1586216)
                    OR  concept_id IN (1586213)
                    AND is_standard = 0
                    AND  value_source_concept_id IN (1586215)
                    OR  concept_id IN (1586213)
                    AND is_standard = 0
                    AND  value_source_concept_id IN (1586214))) criteria )
                AND cb_search_person.person_id IN (SELECT
                    criteria.person_id
                FROM
                    (SELECT
                        DISTINCT person_id, entry_date, concept_id
                    FROM
                        `""" + os.environ["WORKSPACE_CDR"] + """.cb_search_all_events`
                    WHERE
                        (concept_id IN (1586207)
                        AND is_standard = 0
                        AND  value_source_concept_id IN (1586208)
                        OR  concept_id IN (1586207)
                        AND is_standard = 0
                        AND  value_source_concept_id IN (1586209)
                        OR  concept_id IN (1586207)
                        AND is_standard = 0
                        AND  value_source_concept_id IN (1586210)
                        OR  concept_id IN (1586207)
                        AND is_standard = 0
                        AND  value_source_concept_id IN (1586211)
                        OR  concept_id IN (1586207)
                        AND is_standard = 0
                        AND  value_source_concept_id IN (1586212))) criteria )
                AND cb_search_person.person_id IN (SELECT
                    criteria.person_id
                FROM
                    (SELECT
                        DISTINCT person_id, entry_date, concept_id
                    FROM
                        `""" + os.environ["WORKSPACE_CDR"] + """.cb_search_all_events`
                    WHERE
                        (concept_id IN (1586201)
                        AND is_standard = 0
                        AND  value_source_concept_id IN (1586203)
                        OR  concept_id IN (1586201)
                        AND is_standard = 0
                        AND  value_source_concept_id IN (1586202)
                        OR  concept_id IN (1586201)
                        AND is_standard = 0
                        AND  value_source_concept_id IN (1586204)
                        OR  concept_id IN (1586201)
                        AND is_standard = 0
                        AND  value_source_concept_id IN (1586205)
```

```
                    OR  concept_id IN (1586201)
                    AND is_standard = 0
                    AND  value_source_concept_id IN (1586206))) criteria ) )"""

dataset_31550912_person_df = pandas.read_gbq(
    dataset_31550912_person_sql,
    dialect="standard",
    use_bqstorage_api=("BIGQUERY_STORAGE_API_ENABLED" in os.environ),
    progress_bar_type="tqdm_notebook")

# dataset_31550912_person_df.head(5)
```

```
Downloading:   0%|          | 0/262087 [00:00<?, ?rows/s]
```

In [2]:

```
import pandas
import os

# This query represents dataset "M2_V1" for domain "survey" and was generated for A
dataset_31550912_survey_sql = """
    SELECT
        answer.person_id,
        answer.survey_datetime,
        answer.survey,
        answer.question_concept_id,
        answer.question,
        answer.answer_concept_id,
        answer.answer
    FROM
        `""" + os.environ["WORKSPACE_CDR"] + """.ds_survey` answer
    WHERE
        (
            question_concept_id IN (SELECT
                DISTINCT concept_id
            FROM
                `""" + os.environ["WORKSPACE_CDR"] + """.cb_criteria` c
            JOIN
                (SELECT
                    CAST(cr.id as string) AS id
                FROM
                    `""" + os.environ["WORKSPACE_CDR"] + """.cb_criteria` cr
                WHERE
                    concept_id IN (1585855, 1586134, 1585710)
                    AND domain_id = 'SURVEY') a
                    ON (c.path like CONCAT('%', a.id, '.%'))
            WHERE
                domain_id = 'SURVEY'
                AND type = 'PPI'
                AND subtype = 'QUESTION')
        )
        AND (
            answer.PERSON_ID IN (SELECT
                distinct person_id
            FROM
                `""" + os.environ["WORKSPACE_CDR"] + """.cb_search_person` cb_searc
            WHERE
                cb_search_person.person_id IN (SELECT
```

```
                    person_id
                FROM
                    `""" + os.environ["WORKSPACE_CDR"] + """.cb_search_person` p
                WHERE
                    DATE_DIFF(CURRENT_DATE, dob, YEAR) - IF(EXTRACT(MONTH FROM dob)
                    AND NOT EXISTS (      SELECT
                        'x'
                    FROM
                        `""" + os.environ["WORKSPACE_CDR"] + """.death` d
                    WHERE
                        d.person_id = p.person_id ) )
                AND cb_search_person.person_id IN (SELECT
                    criteria.person_id
                FROM
                    (SELECT
                        DISTINCT person_id, entry_date, concept_id
                    FROM
                        `""" + os.environ["WORKSPACE_CDR"] + """.cb_search_all_even
                    WHERE
                        (concept_id IN (1586213)
                        AND is_standard = 0
                        AND  value_source_concept_id IN (1585634)
                        OR  concept_id IN (1586213)
                        AND is_standard = 0
                        AND  value_source_concept_id IN (1586217)
                        OR  concept_id IN (1586213)
                        AND is_standard = 0
                        AND  value_source_concept_id IN (1586216)
                        OR  concept_id IN (1586213)
                        AND is_standard = 0
                        AND  value_source_concept_id IN (1586215)
                        OR  concept_id IN (1586213)
                        AND is_standard = 0
                        AND  value_source_concept_id IN (1586214))) criteria )
                AND cb_search_person.person_id IN (SELECT
                    criteria.person_id
                FROM
                    (SELECT
                        DISTINCT person_id, entry_date, concept_id
                    FROM
                        `""" + os.environ["WORKSPACE_CDR"] + """.cb_search_all_even
                    WHERE
                        (concept_id IN (1586207)
                        AND is_standard = 0
                        AND  value_source_concept_id IN (1586208)
                        OR  concept_id IN (1586207)
                        AND is_standard = 0
                        AND  value_source_concept_id IN (1586209)
                        OR  concept_id IN (1586207)
                        AND is_standard = 0
                        AND  value_source_concept_id IN (1586210)
                        OR  concept_id IN (1586207)
                        AND is_standard = 0
                        AND  value_source_concept_id IN (1586211)
                        OR  concept_id IN (1586207)
                        AND is_standard = 0
```

```
                        AND  value_source_concept_id IN (1586212))) criteria )
                AND cb_search_person.person_id IN (SELECT
                    criteria.person_id
                FROM
                    (SELECT
                        DISTINCT person_id, entry_date, concept_id
                    FROM
                        `""" + os.environ["WORKSPACE_CDR"] + """.cb_search_all_even
                    WHERE
                        (concept_id IN (1586201)
                        AND is_standard = 0
                        AND  value_source_concept_id IN (1586203)
                        OR  concept_id IN (1586201)
                        AND is_standard = 0
                        AND  value_source_concept_id IN (1586202)
                        OR  concept_id IN (1586201)
                        AND is_standard = 0
                        AND  value_source_concept_id IN (1586204)
                        OR  concept_id IN (1586201)
                        AND is_standard = 0
                        AND  value_source_concept_id IN (1586205)
                        OR  concept_id IN (1586201)
                        AND is_standard = 0
                        AND  value_source_concept_id IN (1586206))) criteria ) )
            )"""

dataset_31550912_survey_df = pandas.read_gbq(
    dataset_31550912_survey_sql,
    dialect="standard",
    use_bqstorage_api=("BIGQUERY_STORAGE_API_ENABLED" in os.environ),
    progress_bar_type="tqdm_notebook")

# dataset_31550912_survey_df.head(5)
```

```
Downloading:   0%|          | 0/13832956 [00:00<?, ?rows/s]
```

In [3]:

```
import pandas
import os

# This query represents dataset "M2_V1" for domain "measurement" and was generated
dataset_71486552_measurement_sql = """
    SELECT
        measurement.person_id,
        measurement.measurement_concept_id,
        m_standard_concept.concept_name as standard_concept_name,
        measurement.measurement_datetime,
        measurement.measurement_type_concept_id,
        m_type.concept_name as measurement_type_concept_name,
        measurement.value_as_number,
        measurement.value_as_concept_id,
        m_value.concept_name as value_as_concept_name,
        measurement.unit_concept_id,
        m_unit.concept_name as unit_concept_name,
        measurement.range_low,
        measurement.range_high,
        measurement.measurement_source_value,
```

```
    measurement.measurement_source_concept_id,
    m_source_concept.concept_name as source_concept_name,
    m_source_concept.concept_code as source_concept_code,
    measurement.unit_source_value,
FROM
    ( SELECT
        *
    FROM
        `""" + os.environ["WORKSPACE_CDR"] + """.measurement` measurement
    WHERE
        (
            measurement_concept_id IN (SELECT
                DISTINCT c.concept_id
            FROM
                `""" + os.environ["WORKSPACE_CDR"] + """.cb_criteria` c
            JOIN
                (SELECT
                    CAST(cr.id as string) AS id
                FROM
                    `""" + os.environ["WORKSPACE_CDR"] + """.cb_criteria` cr
                WHERE
                    concept_id IN (
                    3038553, 40782579, 40785861)
                    AND full_text LIKE '%_rank1]%'      ) a
                    ON (c.path LIKE CONCAT('%.', a.id, '.%')
                    OR c.path LIKE CONCAT('%.', a.id)
                    OR c.path LIKE CONCAT(a.id, '.%')
                    OR c.path = a.id)
            WHERE
                is_standard = 1
                AND is_selectable = 1)
        )
        AND (
            measurement.PERSON_ID IN (SELECT
                distinct person_id
            FROM
                `""" + os.environ["WORKSPACE_CDR"] + """.cb_search_person` cb_s
            WHERE
                cb_search_person.person_id IN (SELECT
                    person_id
                FROM
                    `""" + os.environ["WORKSPACE_CDR"] + """.cb_search_person`
                WHERE
                    DATE_DIFF(CURRENT_DATE, dob, YEAR) - IF(EXTRACT(MONTH FROM
                    AND NOT EXISTS (      SELECT
                        'x'
                    FROM
                        `""" + os.environ["WORKSPACE_CDR"] + """.death` d
                    WHERE
                        d.person_id = p.person_id ) )
                AND cb_search_person.person_id IN (SELECT
                    criteria.person_id
                FROM
                    (SELECT
                        DISTINCT person_id, entry_date, concept_id
                    FROM
```

```
                        `""" + os.environ["WORKSPACE_CDR"] + """.cb_search_all_
                    WHERE
                        (concept_id IN (1586213)
                        AND is_standard = 0
                        AND  value_source_concept_id IN (1585634)
                        OR  concept_id IN (1586213)
                        AND is_standard = 0
                        AND  value_source_concept_id IN (1586217)
                        OR  concept_id IN (1586213)
                        AND is_standard = 0
                        AND  value_source_concept_id IN (1586216)
                        OR  concept_id IN (1586213)
                        AND is_standard = 0
                        AND  value_source_concept_id IN (1586215)
                        OR  concept_id IN (1586213)
                        AND is_standard = 0
                        AND  value_source_concept_id IN (1586214))) criteria )
                AND cb_search_person.person_id IN (SELECT
                    criteria.person_id
                FROM
                    (SELECT
                        DISTINCT person_id, entry_date, concept_id
                    FROM
                        `""" + os.environ["WORKSPACE_CDR"] + """.cb_search_all_
                    WHERE
                        (concept_id IN (1586207)
                        AND is_standard = 0
                        AND  value_source_concept_id IN (1586208)
                        OR  concept_id IN (1586207)
                        AND is_standard = 0
                        AND  value_source_concept_id IN (1586209)
                        OR  concept_id IN (1586207)
                        AND is_standard = 0
                        AND  value_source_concept_id IN (1586210)
                        OR  concept_id IN (1586207)
                        AND is_standard = 0
                        AND  value_source_concept_id IN (1586211)
                        OR  concept_id IN (1586207)
                        AND is_standard = 0
                        AND  value_source_concept_id IN (1586212))) criteria )
                AND cb_search_person.person_id IN (SELECT
                    criteria.person_id
                FROM
                    (SELECT
                        DISTINCT person_id, entry_date, concept_id
                    FROM
                        `""" + os.environ["WORKSPACE_CDR"] + """.cb_search_all_
                    WHERE
                        (concept_id IN (1586201)
                        AND is_standard = 0
                        AND  value_source_concept_id IN (1586203)
                        OR  concept_id IN (1586201)
                        AND is_standard = 0
                        AND  value_source_concept_id IN (1586202)
                        OR  concept_id IN (1586201)
                        AND is_standard = 0
```

```
                            AND  value_source_concept_id IN (1586204)
                            OR  concept_id IN (1586201)
                            AND is_standard = 0
                            AND  value_source_concept_id IN (1586205)
                            OR  concept_id IN (1586201)
                            AND is_standard = 0
                            AND  value_source_concept_id IN (1586206))) criteria )
            )) measurement
    LEFT JOIN
        `""" + os.environ["WORKSPACE_CDR"] + """.concept` m_standard_concept
            ON measurement.measurement_concept_id = m_standard_concept.concept_id
    LEFT JOIN
        `""" + os.environ["WORKSPACE_CDR"] + """.concept` m_type
            ON measurement.measurement_type_concept_id = m_type.concept_id
    LEFT JOIN
        `""" + os.environ["WORKSPACE_CDR"] + """.concept` m_operator
            ON measurement.operator_concept_id = m_operator.concept_id
    LEFT JOIN
        `""" + os.environ["WORKSPACE_CDR"] + """.concept` m_value
            ON measurement.value_as_concept_id = m_value.concept_id
    LEFT JOIN
        `""" + os.environ["WORKSPACE_CDR"] + """.concept` m_unit
            ON measurement.unit_concept_id = m_unit.concept_id
    LEFT JOIn
        `""" + os.environ["WORKSPACE_CDR"] + """.visit_occurrence` v
            ON measurement.visit_occurrence_id = v.visit_occurrence_id
    LEFT JOIN
        `""" + os.environ["WORKSPACE_CDR"] + """.concept` m_visit
            ON v.visit_concept_id = m_visit.concept_id
    LEFT JOIN
        `""" + os.environ["WORKSPACE_CDR"] + """.concept` m_source_concept
            ON measurement.measurement_source_concept_id = m_source_concept.concept

dataset_71486552_measurement_df = pandas.read_gbq(
    dataset_71486552_measurement_sql,
    dialect="standard",
    use_bqstorage_api=("BIGQUERY_STORAGE_API_ENABLED" in os.environ),
    progress_bar_type="tqdm_notebook")

# dataset_71486552_measurement_df.head(5)
```

```
Downloading:   0%|          | 0/4842344 [00:00<?, ?rows/s]
```

In [4]:

```
import pandas
import os

# This query represents dataset "M2_V1" for domain "condition" and was generated fo
dataset_34451249_condition_sql = """
    SELECT
        c_occurrence.person_id,
        c_occurrence.condition_concept_id,
        c_standard_concept.concept_name as standard_concept_name,
        c_standard_concept.concept_code as standard_concept_code,
        c_standard_concept.vocabulary_id as standard_vocabulary,
        c_occurrence.condition_start_datetime,
        c_occurrence.condition_end_datetime,
```

```
    c_occurrence.condition_type_concept_id,
    c_type.concept_name as condition_type_concept_name,
    c_occurrence.stop_reason,
    c_occurrence.visit_occurrence_id,
    visit.concept_name as visit_occurrence_concept_name,
    c_occurrence.condition_source_value,
    c_occurrence.condition_source_concept_id,
    c_source_concept.concept_name as source_concept_name,
    c_source_concept.concept_code as source_concept_code,
    c_source_concept.vocabulary_id as source_vocabulary,
    c_occurrence.condition_status_source_value,
    c_occurrence.condition_status_concept_id,
    c_status.concept_name as condition_status_concept_name
FROM
    ( SELECT
        *
    FROM
        `""" + os.environ["WORKSPACE_CDR"] + """.condition_occurrence` c_occurr
    WHERE
        (
            condition_concept_id IN (SELECT
                DISTINCT c.concept_id
            FROM
                `""" + os.environ["WORKSPACE_CDR"] + """.cb_criteria` c
            JOIN
                (SELECT
                    CAST(cr.id as string) AS id
                FROM
                    `""" + os.environ["WORKSPACE_CDR"] + """.cb_criteria` cr
                WHERE
                    concept_id IN (194984, 316866, 440383, 441542)
                    AND full_text LIKE '%_rank1]%'      ) a
                    ON (c.path LIKE CONCAT('%.', a.id, '.%')
                    OR c.path LIKE CONCAT('%.', a.id)
                    OR c.path LIKE CONCAT(a.id, '.%')
                    OR c.path = a.id)
            WHERE
                is_standard = 1
                AND is_selectable = 1)
        )
        AND (
            c_occurrence.PERSON_ID IN (SELECT
                distinct person_id
            FROM
                `""" + os.environ["WORKSPACE_CDR"] + """.cb_search_person` cb_s
            WHERE
                cb_search_person.person_id IN (SELECT
                    person_id
                FROM
                    `""" + os.environ["WORKSPACE_CDR"] + """.cb_search_person`
                WHERE
                    DATE_DIFF(CURRENT_DATE, dob, YEAR) - IF(EXTRACT(MONTH FROM
                    AND NOT EXISTS (      SELECT
                        'x'
                    FROM
                        `""" + os.environ["WORKSPACE_CDR"] + """.death` d
```

```
                        WHERE
                            d.person_id = p.person_id ) )
                    AND cb_search_person.person_id IN (SELECT
                        criteria.person_id
                    FROM
                        (SELECT
                            DISTINCT person_id, entry_date, concept_id
                        FROM
                            `""" + os.environ["WORKSPACE_CDR"] + """.cb_search_all_
                        WHERE
                            (concept_id IN (1586213)
                            AND is_standard = 0
                            AND  value_source_concept_id IN (1585634)
                            OR  concept_id IN (1586213)
                            AND is_standard = 0
                            AND  value_source_concept_id IN (1586217)
                            OR  concept_id IN (1586213)
                            AND is_standard = 0
                            AND  value_source_concept_id IN (1586216)
                            OR  concept_id IN (1586213)
                            AND is_standard = 0
                            AND  value_source_concept_id IN (1586215)
                            OR  concept_id IN (1586213)
                            AND is_standard = 0
                            AND  value_source_concept_id IN (1586214))) criteria )
                    AND cb_search_person.person_id IN (SELECT
                        criteria.person_id
                    FROM
                        (SELECT
                            DISTINCT person_id, entry_date, concept_id
                        FROM
                            `""" + os.environ["WORKSPACE_CDR"] + """.cb_search_all_
                        WHERE
                            (concept_id IN (1586207)
                            AND is_standard = 0
                            AND  value_source_concept_id IN (1586208)
                            OR  concept_id IN (1586207)
                            AND is_standard = 0
                            AND  value_source_concept_id IN (1586209)
                            OR  concept_id IN (1586207)
                            AND is_standard = 0
                            AND  value_source_concept_id IN (1586210)
                            OR  concept_id IN (1586207)
                            AND is_standard = 0
                            AND  value_source_concept_id IN (1586211)
                            OR  concept_id IN (1586207)
                            AND is_standard = 0
                            AND  value_source_concept_id IN (1586212))) criteria )
                    AND cb_search_person.person_id IN (SELECT
                        criteria.person_id
                    FROM
                        (SELECT
                            DISTINCT person_id, entry_date, concept_id
                        FROM
                            `""" + os.environ["WORKSPACE_CDR"] + """.cb_search_all_
                        WHERE
```

```
                            (concept_id IN (1586201)
                            AND is_standard = 0
                            AND  value_source_concept_id IN (1586203)
                            OR  concept_id IN (1586201)
                            AND is_standard = 0
                            AND  value_source_concept_id IN (1586202)
                            OR  concept_id IN (1586201)
                            AND is_standard = 0
                            AND  value_source_concept_id IN (1586204)
                            OR  concept_id IN (1586201)
                            AND is_standard = 0
                            AND  value_source_concept_id IN (1586205)
                            OR  concept_id IN (1586201)
                            AND is_standard = 0
                            AND  value_source_concept_id IN (1586206))) criteria )
            )) c_occurrence
    LEFT JOIN
        `""" + os.environ["WORKSPACE_CDR"] + """.concept` c_standard_concept
            ON c_occurrence.condition_concept_id = c_standard_concept.concept_id
    LEFT JOIN
        `""" + os.environ["WORKSPACE_CDR"] + """.concept` c_type
            ON c_occurrence.condition_type_concept_id = c_type.concept_id
    LEFT JOIN
        `""" + os.environ["WORKSPACE_CDR"] + """.visit_occurrence` v
            ON c_occurrence.visit_occurrence_id = v.visit_occurrence_id
    LEFT JOIN
        `""" + os.environ["WORKSPACE_CDR"] + """.concept` visit
            ON v.visit_concept_id = visit.concept_id
    LEFT JOIN
        `""" + os.environ["WORKSPACE_CDR"] + """.concept` c_source_concept
            ON c_occurrence.condition_source_concept_id = c_source_concept.concept_
    LEFT JOIN
        `""" + os.environ["WORKSPACE_CDR"] + """.concept` c_status
            ON c_occurrence.condition_status_concept_id = c_status.concept_id"""

dataset_34451249_condition_df = pandas.read_gbq(
    dataset_34451249_condition_sql,
    dialect="standard",
    use_bqstorage_api=("BIGQUERY_STORAGE_API_ENABLED" in os.environ),
    progress_bar_type="tqdm_notebook")

# dataset_34451249_condition_df.head(5)
```

```
Downloading:   0%|          | 0/3680533 [00:00<?, ?rows/s]
```

In [5]:

```
import pandas
import os

# This query represents dataset "M2_V1" for domain "drug" and was generated for All
dataset_67921512_drug_sql = """
    SELECT
        d_exposure.person_id,
        d_exposure.drug_concept_id,
        d_standard_concept.concept_name as standard_concept_name,
        d_standard_concept.concept_code as standard_concept_code,
        d_standard_concept.vocabulary_id as standard_vocabulary,
```

```
        d_exposure.drug_exposure_start_datetime,
        d_exposure.drug_exposure_end_datetime,
        d_exposure.verbatim_end_date,
        d_exposure.drug_type_concept_id,
        d_type.concept_name as drug_type_concept_name,
        d_exposure.stop_reason,
        d_exposure.refills,
        d_exposure.quantity,
        d_exposure.days_supply,
        d_exposure.sig,
        d_exposure.route_concept_id,
        d_route.concept_name as route_concept_name,
        d_exposure.lot_number,
        d_exposure.visit_occurrence_id,
        d_visit.concept_name as visit_occurrence_concept_name,
        d_exposure.drug_source_value,
        d_exposure.drug_source_concept_id,
        d_source_concept.concept_name as source_concept_name,
        d_source_concept.concept_code as source_concept_code,
        d_source_concept.vocabulary_id as source_vocabulary,
        d_exposure.route_source_value,
        d_exposure.dose_unit_source_value
    FROM
        ( SELECT
            *
        FROM
            `""" + os.environ["WORKSPACE_CDR"] + """.drug_exposure` d_exposure
        WHERE
            (
                drug_concept_id IN (SELECT
                    DISTINCT ca.descendant_id
                FROM
                    `""" + os.environ["WORKSPACE_CDR"] + """.cb_criteria_ancestor`
                JOIN
                    (SELECT
                        DISTINCT c.concept_id
                    FROM
                        `""" + os.environ["WORKSPACE_CDR"] + """.cb_criteria` c
                    JOIN
                        (SELECT
                            CAST(cr.id as string) AS id
                        FROM
                            `""" + os.environ["WORKSPACE_CDR"] + """.cb_criteria` c
                        WHERE
                            concept_id IN (1714319, 19043959, 735850)
                            AND full_text LIKE '%_rank1]%'       ) a
                            ON (c.path LIKE CONCAT('%.', a.id, '.%')
                            OR c.path LIKE CONCAT('%.', a.id)
                            OR c.path LIKE CONCAT(a.id, '.%')
                            OR c.path = a.id)
                    WHERE
                        is_standard = 1
                        AND is_selectable = 1) b
                        ON (ca.ancestor_id = b.concept_id)))
                    AND (d_exposure.PERSON_ID IN (SELECT
                        distinct person_id
```

```
                FROM
                    `""" + os.environ["WORKSPACE_CDR"] + """.cb_search_person` cb_s
                WHERE
                    cb_search_person.person_id IN (SELECT
                        person_id
                    FROM
                        `""" + os.environ["WORKSPACE_CDR"] + """.cb_search_person`
                    WHERE
                        DATE_DIFF(CURRENT_DATE, dob, YEAR) - IF(EXTRACT(MONTH FROM
                        AND NOT EXISTS (      SELECT
                            'x'
                        FROM
                            `""" + os.environ["WORKSPACE_CDR"] + """.death` d
                        WHERE
                            d.person_id = p.person_id ) )
                        AND cb_search_person.person_id IN (SELECT
                        criteria.person_id
                        FROM
                        (SELECT
                            DISTINCT person_id, entry_date, concept_id
                        FROM
                            `""" + os.environ["WORKSPACE_CDR"] + """.cb_search_all_
                        WHERE
                            (concept_id IN (1586213)
                            AND is_standard = 0
                            AND  value_source_concept_id IN (1585634)
                            OR  concept_id IN (1586213)
                            AND is_standard = 0
                            AND  value_source_concept_id IN (1586217)
                            OR  concept_id IN (1586213)
                            AND is_standard = 0
                            AND  value_source_concept_id IN (1586216)
                            OR  concept_id IN (1586213)
                            AND is_standard = 0
                            AND  value_source_concept_id IN (1586215)
                            OR  concept_id IN (1586213)
                            AND is_standard = 0
                            AND  value_source_concept_id IN (1586214))) criteria )
                        AND cb_search_person.person_id IN (SELECT
                        criteria.person_id
                        FROM
                        (SELECT
                            DISTINCT person_id, entry_date, concept_id
                        FROM
                            `""" + os.environ["WORKSPACE_CDR"] + """.cb_search_all_
                        WHERE
                            (concept_id IN (1586207)
                            AND is_standard = 0
                            AND  value_source_concept_id IN (1586208)
                            OR  concept_id IN (1586207)
                            AND is_standard = 0
                            AND  value_source_concept_id IN (1586209)
                            OR  concept_id IN (1586207)
                            AND is_standard = 0
                            AND  value_source_concept_id IN (1586210)
                            OR  concept_id IN (1586207)
```

```
                            AND is_standard = 0
                            AND  value_source_concept_id IN (1586211)
                            OR  concept_id IN (1586207)
                            AND is_standard = 0
                            AND  value_source_concept_id IN (1586212))) criteria )
                    AND cb_search_person.person_id IN (SELECT
                        criteria.person_id
                    FROM
                        (SELECT
                            DISTINCT person_id, entry_date, concept_id
                        FROM
                            `""" + os.environ["WORKSPACE_CDR"] + """.cb_search_all_
                        WHERE
                            (concept_id IN (1586201)
                            AND is_standard = 0
                            AND  value_source_concept_id IN (1586203)
                            OR  concept_id IN (1586201)
                            AND is_standard = 0
                            AND  value_source_concept_id IN (1586202)
                            OR  concept_id IN (1586201)
                            AND is_standard = 0
                            AND  value_source_concept_id IN (1586204)
                            OR  concept_id IN (1586201)
                            AND is_standard = 0
                            AND  value_source_concept_id IN (1586205)
                            OR  concept_id IN (1586201)
                            AND is_standard = 0
                            AND  value_source_concept_id IN (1586206))) criteria )
            )) d_exposure
    LEFT JOIN
        `""" + os.environ["WORKSPACE_CDR"] + """.concept` d_standard_concept
            ON d_exposure.drug_concept_id = d_standard_concept.concept_id
    LEFT JOIN
        `""" + os.environ["WORKSPACE_CDR"] + """.concept` d_type
            ON d_exposure.drug_type_concept_id = d_type.concept_id
    LEFT JOIN
        `""" + os.environ["WORKSPACE_CDR"] + """.concept` d_route
            ON d_exposure.route_concept_id = d_route.concept_id
    LEFT JOIN
        `""" + os.environ["WORKSPACE_CDR"] + """.visit_occurrence` v
            ON d_exposure.visit_occurrence_id = v.visit_occurrence_id
    LEFT JOIN
        `""" + os.environ["WORKSPACE_CDR"] + """.concept` d_visit
            ON v.visit_concept_id = d_visit.concept_id
    LEFT JOIN
        `""" + os.environ["WORKSPACE_CDR"] + """.concept` d_source_concept
            ON d_exposure.drug_source_concept_id = d_source_concept.concept_id"""

dataset_67921512_drug_df = pandas.read_gbq(
    dataset_67921512_drug_sql,
    dialect="standard",
    use_bqstorage_api=("BIGQUERY_STORAGE_API_ENABLED" in os.environ),
    progress_bar_type="tqdm_notebook")

# dataset_67921512_drug_df.head(5)
```

```
Downloading:   0%|          | 0/35157 [00:00<?, ?rows/s]
```

Person Table/Demographics

In [6]:
```
person = dataset_31550912_person_df.copy()
print(person.shape)
person.columns
```

```
(262087, 6)
```

Out[6]:
```
Index(['person_id', 'gender', 'date_of_birth', 'race', 'ethnicity',
       'sex_at_birth'],
      dtype='object')
```

In [7]:
```
#Make Race2 Hispanic if ethnicity is Hispanic, else use Race
person['race2'] = person.apply(
lambda x: 'Hispanic' if x['ethnicity'] == 'Hispanic or Latino' else x['race'],
axis=1
)
person.race2.value_counts()
```

Out[7]:
```
race2
White                          137432
Hispanic                        52748
Black or African American       39519
More than one population        12592
Asian                           10731
Another single population        4099
None of these                    2552
PMI: Skip                        1349
I prefer not to answer           1063
None Indicated                      2
Name: count, dtype: int64
```

In [8]:
```
#Filter Race2 to be only singel demograhic
person_filtered = person[person["race2"].isin(["White", "Hispanic", "Black or Afric
person_filtered["race2"].value_counts()
```

Out[8]:
```
race2
White                          137432
Hispanic                        52748
Black or African American       39519
Asian                           10731
Name: count, dtype: int64
```

In [9]:
```
person_filtered.sex_at_birth.value_counts()
```

Out[9]:
```
sex_at_birth
Female                                                 159114
Male                                                    80185
Not male, not female, prefer not to answer, or skipped      1078
No matching concept                                           53
Name: count, dtype: int64
```

In [10]:
```
#Filter for patients with sex at birth male or female
person_filtered2 = person_filtered[person_filtered["sex_at_birth"].isin(["Female",
person_filtered2["sex_at_birth"].value_counts()
```

Out[10]:
```
sex_at_birth
Female    159114
Male       80185
Name: count, dtype: int64
```

In [11]:
```
#Map the values: 'Male' to 1 and 'female' to 0
#Makes sex_at_birth a boolean
person_filtered2["sex_at_birth"] = person_filtered2["sex_at_birth"].map({'Male': 1,

#Rename the column to 'sex_assigned_male'
person_filtered2 = person_filtered2.rename(columns={'sex_at_birth': 'sex_at_birth_m
```

In [12]:
```
person_filtered2.sex_at_birth_male.value_counts()
```

Out[12]:
```
sex_at_birth_male
0    159114
1     80185
Name: count, dtype: int64
```

In [13]:
```
#Age is not in df, so will have to calculate later based on time of alcohol survey
person_cleaned = person_filtered2[["person_id", "race2", "sex_at_birth_male", "date
person_cleaned.shape
```

Out[13]:
```
(239299, 4)
```

In [14]:
```
# person_cleaned.head()
```

Audit C

In [15]:
```
survey = dataset_31550912_survey_df.copy()
survey.columns
```

Out[15]:
```
Index(['person_id', 'survey_datetime', 'survey', 'question_concept_id',
       'question', 'answer_concept_id', 'answer'],
      dtype='object')
```

In [16]:
```
import pandas as pd

#Survey questions, their concept id, and their frequency
pd.set_option("display.max_rows", None)
print(survey[["question", "question_concept_id"]].value_counts())
pd.reset_option("display.max_rows")
```

question	question_concept_id	
Recreational Drug Use: Which Drugs Used	1585636	482070
Employment: Employment Status	1585952	278717
Race: What Race Ethnicity	1586140	274293
Health Insurance: Health Insurance Type	1585389	263858
Alcohol: Alcohol Participant	1586198	262087
Cigar Smoking: Cigar Smoke Participant	1586174	262087
Smoking: 100 Cigs Lifetime	1585857	262087
Smokeless Tobacco: Smokeless Tobacco Participant	1586190	262087
Alcohol: Drink Frequency Past Year	1586201	262087
Electronic Smoking: Electric Smoke Participant	1586166	262087
Alcohol: Average Daily Drink Count	1586207	262087
Hookah Smoking: Hookah Smoke Participant	1586182	262087
Alcohol: 6 or More Drinks Occurrence	1586213	262087
Living Situation: Stable House Concern	1585886	262085
Living Situation: People Under 18	1585890	262085
Home Own: Current Home Own	1585370	262085
Living Situation: How Many People	1585889	262085
Living Situation: How Many Living Years	1585879	262085
Insurance: Health Insurance	1585386	262085
Income: Annual Income	1585375	262085
Gender: Gender Identity	1585838	262085
Education Level: Highest Grade	1585940	262085
The Basics: Birthplace	1586135	262085
The Basics: Sexual Orientation	1585899	262085
Biological Sex At Birth: Sex At Birth	1585845	262023
Overall Health: Medical Form Confidence	1585766	261014
Overall Health: Health Material Assistance	1585772	261014
Overall Health: Emotional Problem 7 Days	1585760	

261014
Overall Health: Organ Transplant 1585803
261014
Overall Health: Outside Travel 6 Month 1585815
261014
Overall Health: Social Satisfaction 1585735
261014
Overall Health: General Social 1585754
261014
Overall Health: General Quality 1585717
261014
Overall Health: General Physical Health 1585723
261014
Overall Health: General Mental Health 1585729
261014
Overall Health: Everyday Activities 1585741
261014
Overall Health: General Health 1585711
261014
Overall Health: Difficult Understand Info 1585778
261014
Overall Health: Average Fatigue 7 Days 1585748
261014
Overall Health: Average Pain 7 Days 1585747
261014
Marital Status: Current Marital Status 1585892
235832
Health Insurance: Insurance Type Update 43528428
231094
Disability: Difficulty Concentrating 903575
176873
Disability: Deaf 903573
176873
Disability: Blind 903574
176873
Disability: Errands Alone 903578
176873
Disability: Walking Climbing 903576
176873
Disability: Dressing Bathing 903577
176873
Overall Health: Menstrual Stopped 1585784
174031
Cigar Smoking: Current Cigar Frequency 1586177
118017
Pregnancy: 1 Pregnancy Status 1585811
117323
Smoking: Smoke Frequency 1585860
93302
Smoking: Daily Smoke Starting Age 1585864
93279
Smoking: Serious Quit Attempt 1585867
93269
Smoking: Number Of Years 1585873
93164
Smoking: Current Daily Cigarette Number 1586159

```
93139
Smoking: Average Daily Cigarette Number                          1586162
93136
Hookah Smoking: Current Hookah Frequency                         1586185
80710
Past 3 Month Use Frequency: Marijuana 3 Month Use                1585650
80028
Attempt Quit Smoking: Completely Quit Age                        1585870
78507
Electronic Smoking: Electric Smoke Frequency                     1586169
74723
Overall Health: Hysterectomy History                             1585791
66076
Overall Health: Ovary Removal History                            1585796
66070
Yes None: Menstrual Stopped Reason                               1585789
57614
Outside Travel 6 Month: Outside Travel 6 Month How Long          1585820
36706
Smokeless Tobacco: Smokeless Tobacco Frequency                   1586193
34407
Overall Health: Hysterectomy History Age                         1585795
20059
Overall Health Ovary Removal History Age                         1585802
12185
Organ Transplant: Organ Transplant Description                   1585806
3569
Past 3 Month Use Frequency: Prescription Opioid 3 Month Use      1585698
2028
Past 3 Month Use Frequency: Prescription Stimulant 3 Month Use   903058
1559
Past 3 Month Use Frequency: Cocaine 3 Month Use                  1585656
1417
Past 3 Month Use Frequency: Sedative 3 Month Use                 1585680
1185
Past 3 Month Use Frequency: Other 3 Month Use                    1585704
472
Past 3 Month Use Frequency: Other Stimulant 3 Month Use          1585668
298
Past 3 Month Use Frequency: Hallucinogen 3 Month Use             1585686
236
Past 3 Month Use Frequency: Street Opioid 3 Month Use            1585692
201
Past 3 Month Use Frequency: Inhalant 3 Month Use                 1585674
193
Name: count, dtype: int64
```

In [17]:
```
#How often have you had a drink containing alcohol in the past year?:
audit_q1 = survey[survey['question_concept_id']== 1586201].copy()
audit_q1.shape
```

Out[17]: (262087, 7)

In [18]:
```
audit_q1.answer.value_counts()
```

Out[18]:
```
answer
Drink Frequency Past Year: Monthly Or Less       115507
Drink Frequency Past Year: 2 to 4 Per Month       72737
Drink Frequency Past Year: 2 to 3 Per Week        45714
Drink Frequency Past Year: 4 or More Per Week     28129
Name: count, dtype: int64
```

In [19]:
```
#On a typical day when you drink, how many drinks do you have?
audit_q2 = survey[survey['question_concept_id'] == 1586207].copy()
audit_q2.shape
```

Out[19]:
```
(262087, 7)
```

In [20]:
```
audit_q2.answer.value_counts()
```

Out[20]:
```
answer
Average Daily Drink Count: 1 or 2        183523
Average Daily Drink Count: 3 or 4         54641
Average Daily Drink Count: 5 or 6         15742
Average Daily Drink Count: 7 to 9          4271
Average Daily Drink Count: 10 or More      3910
Name: count, dtype: int64
```

In [21]:
```
#How often did you have six or more drinks on one occasion in the past year?
audit_q3 = survey[survey['question_concept_id'] == 1586213].copy()
audit_q3.shape
```

Out[21]:
```
(262087, 7)
```

In [22]:
```
audit_q3.answer.value_counts()
```

Out[22]:
```
answer
6 or More Drinks Occurrence: Never In Last Year    126016
6 or More Drinks Occurrence: Less Than Monthly      89686
6 or More Drinks Occurrence: Monthly                29162
6 or More Drinks Occurrence: Weekly                 12292
6 or More Drinks Occurrence: Daily                   4931
Name: count, dtype: int64
```

In [23]:
```
q1_map = {
    "Drink Frequency Past Year: Never In Last Year": 0,
    "Drink Frequency Past Year: Monthly Or Less": 1,
    "Drink Frequency Past Year: 2 to 4 Per Month": 2,
    "Drink Frequency Past Year: 2 to 3 Per Week": 3,
    "Drink Frequency Past Year: 4 or More Per Week": 4
}

q2_map = {
    "Average Daily Drink Count: 1 or 2": 0,
    "Average Daily Drink Count: 3 or 4": 1,
    "Average Daily Drink Count: 5 or 6": 2,
    "Average Daily Drink Count: 7 to 9": 3,
    "Average Daily Drink Count: 10 or More": 4
}
```

```
q3_map = {
    "6 or More Drinks Occurrence: Never In Last Year": 0,
    "6 or More Drinks Occurrence: Less Than Monthly": 1,
    "6 or More Drinks Occurrence: Monthly": 2,
    "6 or More Drinks Occurrence: Weekly": 3,
    "6 or More Drinks Occurrence: Daily": 4
}
```

In [24]:
```
audit_q1["audit_q1_score"] = audit_q1["answer"].map(q1_map)
audit_q2["audit_q2_score"] = audit_q2["answer"].map(q2_map)
audit_q3["audit_q3_score"] = audit_q3["answer"].map(q3_map)
```

In [25]:
```
audit_q1.audit_q1_score.value_counts()
```

Out[25]:
```
audit_q1_score
1    115507
2     72737
3     45714
4     28129
Name: count, dtype: int64
```

In [26]:
```
audit_q2.audit_q2_score.value_counts()
```

Out[26]:
```
audit_q2_score
0    183523
1     54641
2     15742
3      4271
4      3910
Name: count, dtype: int64
```

In [27]:
```
audit_q3.audit_q3_score.value_counts()
```

Out[27]:
```
audit_q3_score
0    126016
1     89686
2     29162
3     12292
4      4931
Name: count, dtype: int64
```

In [28]:
```
df1 = audit_q1.rename(columns={
    "survey_datetime": "survey_datetime_q1",
})
df1 = df1[['person_id', 'survey_datetime_q1', 'audit_q1_score']]


df2 = audit_q2.rename(columns={
    "survey_datetime": "survey_datetime_q2",
})
df2 = df2[['person_id', 'survey_datetime_q2', 'audit_q2_score']]


df3 = audit_q3.rename(columns={
    "survey_datetime": "survey_datetime_q3",
```

```
})
df3 = df3[['person_id', 'survey_datetime_q3', 'audit_q3_score']]
```

In [29]:
```
df_merged = df1.merge(df2, on="person_id", how="inner") \
               .merge(df3, on="person_id", how="inner")
```

In [30]:
```
#Use the most recent survey completion as the index date
df_merged["index_date"] = df_merged[[
    "survey_datetime_q1",
    "survey_datetime_q2",
    "survey_datetime_q3"
]].max(axis=1)

# df_merged
```

In [31]:
```
#Observation window is less than 2 years before survey completion:
from pandas.tseries.offsets import DateOffset

#Adding one day, so surveys taken on same day as alcohol survey are counted
df_merged["index_date"] = df_merged["index_date"] + pd.Timedelta(days=1)
df_merged["obs_window"] = df_merged["index_date"] - DateOffset(years=2)
```

In [32]:
```
df_merged["audit_c"] = df_merged[["audit_q1_score", "audit_q2_score", "audit_q3_sco
```

In [33]:
```
audit_merged_clean = df_merged[['person_id', 'audit_c', 'index_date', 'obs_window']
audit_merged_clean.shape
```

Out[33]:
```
(262087, 4)
```

In [34]:
```
# audit_merged_clean.head()
```

Measurements

In [35]:
```
measurements = dataset_71486552_measurement_df.copy()
measurements.columns
```

Out[35]:
```
Index(['person_id', 'measurement_concept_id', 'standard_concept_name',
       'measurement_datetime', 'measurement_type_concept_id',
       'measurement_type_concept_name', 'value_as_number',
       'value_as_concept_id', 'value_as_concept_name', 'unit_concept_id',
       'unit_concept_name', 'range_low', 'range_high',
       'measurement_source_value', 'measurement_source_concept_id',
       'source_concept_name', 'source_concept_code', 'unit_source_value'],
      dtype='object')
```

In [36]:
```
print(measurements.shape)
# measurements.head()
```

```
(4842344, 18)
```

In [37]:
```
measurements_filtered = measurements.merge(
    audit_merged_clean[["person_id", "index_date", "obs_window"]],
```

```
    on="person_id",
    how="inner"
)
```

In [38]:
```
#Only keep measurements that fall within the patient observation window
measurements_filtered = measurements_filtered[
    (measurements_filtered["measurement_datetime"] >= measurements_filtered["obs_wi
    (measurements_filtered["measurement_datetime"] <= measurements_filtered["index_
]
```

In [39]:
```
measurements_filtered.shape
```

Out[39]: (1267905, 20)

In [40]:
```
#Drop NA measurements
measurements_filtered = measurements_filtered.dropna(subset=["value_as_number"])
```

In [41]:
```
measurements_filtered.shape
```

Out[41]: (1254625, 20)

In [42]:
```
df_latest = measurements_filtered.sort_values(["person_id", "measurement_concept_id

#Keep the most recent measurement per person, per different measurement
df_latest = df_latest.drop_duplicates(
    subset=["person_id", "measurement_concept_id"],
    keep="last"
)
```

In [43]:
```
df_latest.shape
```

Out[43]: (304420, 20)

In [44]:
```
bmi_measure = df_latest[df_latest['measurement_concept_id'] == 3038553].copy()

UPPER_LIMIT = 80
LOWER_LIMIT = 10

# 3. Filter out the unreasonable scores
# This removes negatives, zeros (usually errors), and extreme outliers
bmi_measure = bmi_measure[
    (bmi_measure['value_as_number'] > LOWER_LIMIT) &
    (bmi_measure['value_as_number'] <= UPPER_LIMIT)
].copy()

bmi_measure = bmi_measure[['person_id', 'measurement_datetime', 'value_as_number']]

bmi_measure = bmi_measure.rename(columns={
    "value_as_number": "bmi_measure",
})
```

In [45]:
```
print(bmi_measure.shape)
```

```
bmi_measure.describe()
```

(167891, 3)

Out[45]:

	person_id	bmi_measure
count	167891.0	167891.000000
mean	3543202.016445	30.146309
std	2472769.273001	8.032285
min	1000093.0	10.680000
25%	1696281.0	24.300000
50%	2616702.0	28.600000
75%	4796354.5	34.300000
max	9999860.0	79.900000

In [46]:
```
ast_measure = df_latest[df_latest['measurement_concept_id'] == 3013721].copy()

UPPER_LIMIT = 1500
LOWER_LIMIT = 5

# 3. Filter out the unreasonable scores
# This removes negatives, zeros (usually errors), and extreme outliers
ast_measure = ast_measure[
    (ast_measure['value_as_number'] > LOWER_LIMIT) &
    (ast_measure['value_as_number'] <= UPPER_LIMIT)
].copy()


ast_measure = ast_measure[['person_id', 'measurement_datetime', 'value_as_number']]

ast_measure = ast_measure.rename(columns={
    "value_as_number": "ast_measure",
})
```

In [47]:
```
print(ast_measure.shape)
ast_measure.describe()
```

(63354, 3)

Out[47]:

	person_id	ast_measure
count	63354.0	63354.000000
mean	3674161.987341	26.535535
std	2533559.10381	35.495315
min	1000042.0	6.000000
25%	1725682.75	16.000000
50%	2716512.0	20.000000
75%	5220560.75	26.000000
max	9999860.0	1483.000000

In [48]:
```
alt_measure = df_latest[df_latest['measurement_concept_id'] == 3006923].copy()


UPPER_LIMIT = 1500
LOWER_LIMIT = 5

# 3. Filter out the unreasonable scores
# This removes negatives, zeros (usually errors), and extreme outliers
alt_measure = alt_measure[
    (alt_measure['value_as_number'] > LOWER_LIMIT) &
    (alt_measure['value_as_number'] <= UPPER_LIMIT)
].copy()


alt_measure = alt_measure[['person_id', 'measurement_datetime', 'value_as_number']]

alt_measure = alt_measure.rename(columns={
    "value_as_number": "alt_measure",
})
```

In [49]:
```
print(alt_measure.shape)
alt_measure.describe()
```

```
(57883, 3)
```

Out[49]:

	person_id	alt_measure
count	57883.0	57883.000000
mean	3675236.296305	28.782148
std	2533773.450641	42.252977
min	1000042.0	6.000000
25%	1727419.5	14.000000
50%	2717616.0	20.000000
75%	5226452.5	30.000000
max	9999860.0	1420.000000

In [50]:
```
measurements_cleaned = bmi_measure[['person_id', 'bmi_measure']].merge(
    ast_measure[['person_id', 'ast_measure']],
    on="person_id",
    how="inner"
)

measurements_cleaned = measurements_cleaned.merge(
    alt_measure[['person_id', 'alt_measure']],
    on="person_id",
    how="inner"
)
```

In [51]:
```
# measurements_cleaned
```

Socioeconomic Indicators

In [52]:
```
survey.shape
```

Out[52]: (13832956, 7)

In [53]:
```
#Socioeconomic questions
survey = survey[survey['question_concept_id'].isin([1585940, 1585952, 1585375, 1585
                                                    #Smoking questions
                                                   1586174, 1586177])]
survey.shape
```

Out[53]: (2683935, 7)

In [54]:
```
survey_filtered = survey.merge(
    audit_merged_clean[["person_id", "index_date", "obs_window"]],
    on="person_id",
    how="inner"
)
```

In [55]:
```
##Only keep surveys that fall within the patient observation window
survey = survey_filtered[
    (survey_filtered["survey_datetime"] >= survey_filtered["obs_window"]) &
    (survey_filtered["survey_datetime"] <= survey_filtered["index_date"])
]
```

In [56]:
```
survey.shape
```

Out[56]: (2677054, 9)

In [57]:
```
#Keep the most recent survey per person, per different different question
survey = survey.sort_values(["person_id", "question_concept_id", "survey_datetime"]

survey = survey.drop_duplicates(
    subset=["person_id", "question_concept_id"],
    keep="last"
)
```

In [58]:
```
survey.shape
```

Out[58]: (2440479, 9)

In [59]:
```
#What is the highest grade or year of school you completed?
se_q1 = survey[survey['question_concept_id']== 1585940].copy()

se_q1.shape
```

Out[59]: (261481, 9)

In [60]:
```
se_q1[['answer_concept_id', 'answer']].value_counts()
```

Out[60]:
```
answer_concept_id  answer
2000000006         College graduate or advanced degree            128106
1585946            Highest Grade: College One to Three             70516
1585945            Highest Grade: Twelve Or GED                    42561
2000000007         Less than a high school degree or equivalent    16447
903096             PMI: Skip                                        2897
903079             PMI: Prefer Not To Answer                         954
Name: count, dtype: int64
```

In [61]:
```
#Filter out people who didn't answer, about their education level
se_q1 = se_q1[se_q1['answer_concept_id'].isin([2000000006, 1585946, 1585945, 200000

#Make education level a binary variable of whether a person attended college or hig
se_q1_map = {
    "College graduate or advanced degree": 1,
    "Highest Grade: College One to Three": 1,
    "Highest Grade: Twelve Or GED": 0,
    "Less than a high school degree or equivalent": 0,
}

se_q1["some_college"] = se_q1["answer"].map(se_q1_map)

se_q1 = se_q1[['person_id', 'some_college']]
```

In [62]:
```
# What is your current employment status? Please select 1 or more of these categori
se_q2 = survey[survey['question_concept_id']== 1585952].copy()

se_q2.shape
```

Out[62]:
```
(261481, 9)
```

In [63]:
```
se_q2[['answer_concept_id', 'answer']].value_counts()
```

Out[63]:
```
answer_concept_id  answer
2000000004         Employed for wages or self-employed    173011
2000000005         Not currently employed for wages        83322
903079             PMI: Prefer Not To Answer                4198
903096             PMI: Skip                                 950
Name: count, dtype: int64
```

In [64]:
```
#Filter out people who didn't answer, about their employment status
se_q2 = se_q2[se_q2['answer_concept_id'].isin([2000000004, 2000000005])]
```

In [65]:
```
#Converting employment status to a boolean
se_q2_map = {
    "Employed for wages or self-employed": 1,
    "Not currently employed for wages": 0,
}

se_q2["employment_status"] = se_q2["answer"].map(se_q2_map)
```

In [66]:
```
se_q2 = se_q2[['person_id', "employment_status"]]
```

In [67]:
```
#What is your annual household income from all sources?
se_q3 = survey[survey['question_concept_id']== 1585375].copy()

se_q3.shape
```

Out[67]:
```
(261481, 9)
```

In [68]:
```
se_q3[['answer_concept_id', 'answer']].value_counts()
```

Out[68]:
```
answer_concept_id  answer
1585376            Annual Income: less 10k      32415
1585382            Annual Income: 100k 150k     31522
1585380            Annual Income: 50k 75k       30377
1585377            Annual Income: 10k 25k       26306
1585381            Annual Income: 75k 100k      24100
903079             PMI: Prefer Not To Answer    23809
1585379            Annual Income: 35k 50k       23005
1585384            Annual Income: more 200k     22521
1585378            Annual Income: 25k 35k       19468
1585383            Annual Income: 150k 200k     16279
903096             PMI: Skip                    11679
Name: count, dtype: int64
```

In [69]:
```
#Filter out people who didn't answer, about their yearly income
se_q3 = se_q3[~se_q3['answer_concept_id'].isin([903096, 903079])]

se_q3 = se_q3[['person_id', 'answer']]

se_q3 = se_q3.rename(columns={
    "answer": "household_income",
})
```

In [70]:
```
#In general, how would you rate your physical health?
se_q4 = survey[survey['question_concept_id']== 1585723].copy()

print(se_q4.shape)

se_q4[['answer_concept_id', 'answer']].value_counts()
```

```
(259524, 9)
```

Out[70]:
```
answer_concept_id  answer
1585726            General Physical Health: Good         94176
1585725            General Physical Health: Very Good    72222
1585727            General Physical Health: Fair         53266
1585724            General Physical Health: Excellent    24333
1585728            General Physical Health: Poor         13862
903096             PMI: Skip                              1665
Name: count, dtype: int64
```

In [71]:
```
se_q4= se_q4[~se_q4['answer_concept_id'].isin([903096])]

se_q4 = se_q4[['person_id', 'answer']]

se_q4 = se_q4.rename(columns={
    "answer": "physical_health",
})
```

In [72]:
```
#In general, how would you rate your mental health, including your mood and ability
se_q5 = survey[survey['question_concept_id']== 1585729].copy()

print(se_q5.shape)

se_q5[['answer_concept_id', 'answer']].value_counts()
```

```
(259524, 9)
```

Out[72]:
```
answer_concept_id  answer
1585732            General Mental Health: Good         82591
1585731            General Mental Health: Very Good    78693
1585733            General Mental Health: Fair         45492
903626             General Mental Health: Excellent    31426
1585734            General Mental Health: Poor         11327
1585730            General Mental Health: Exclllent     8705
903096             PMI: Skip                            1290
Name: count, dtype: int64
```

In [73]:
```
#Make the 2 excellents, the same answer
se_q5.loc[se_q5['answer_concept_id'] == 1585730, 'answer'] = 'General Mental Health
se_q5.loc[se_q5['answer_concept_id'] == 1585730, 'answer_concept_id'] = 903626


# Verify the result
se_q5[['answer_concept_id', 'answer']].value_counts()
```

Out[73]:
```
answer_concept_id  answer
1585732            General Mental Health: Good         82591
1585731            General Mental Health: Very Good    78693
1585733            General Mental Health: Fair         45492
903626             General Mental Health: Excellent    40131
1585734            General Mental Health: Poor         11327
903096             PMI: Skip                            1290
Name: count, dtype: int64
```

In [74]:
```
se_q5= se_q5[~se_q5['answer_concept_id'].isin([903096])]

se_q5 = se_q5[['person_id', 'answer']]

se_q5 = se_q5.rename(columns={
    "answer": "mental_health",
})
```

In [75]:
```
# What is your current marital status?
se_q6 = survey[survey['question_concept_id']== 1585892].copy()

print(se_q6.shape)

se_q6[['answer_concept_id', 'answer']].value_counts()
```

```
(235273, 9)
```

Out[75]:
```
answer_concept_id  answer
1585893            Current Marital Status: Married          107505
1585897            Current Marital Status: Never Married     81590
1585894            Current Marital Status: Divorced          29447
1585896            Current Marital Status: Separated          8534
903079             PMI: Prefer Not To Answer                  3617
1585895            Current Marital Status: Widowed            3604
903096             PMI: Skip                                   976
Name: count, dtype: int64
```

In [76]:
```
se_q6= se_q6[~se_q6['answer_concept_id'].isin([903096, 903079])]

se_q6 = se_q6[['person_id', 'answer']]
```

```
se_q6 = se_q6.rename(columns={
    "answer": "marrital_status",
})
```

In [77]:
```
#In your LIFETIME, which of the following substances have you ever used?
se_q7 = survey[survey['question_concept_id']== 1585636].copy()

print(se_q7.shape)

se_q7[['answer_concept_id', 'answer']].value_counts()
```

```
(262087, 9)
```

Out[77]:

answer_concept_id	answer	
1585637	Which Drugs Used: Marijuana Use	108102
1585648	Which Drugs Used: None Of These Drugs	80107
1585638	Which Drugs Used: Cocaine Use	15461
1585643	Which Drugs Used: Hallucinogens Use	11592
1585639	Which Drugs Used: Prescription Stimulants Use	9854
1585645	Which Drugs Used: Prescription Opioids Use	8339
1585642	Which Drugs Used: Sedatives Use	7856
903079	PMI: Prefer Not To Answer	5932
1585640	Which Drugs Used: Methamphetamine Use	4778
903096	PMI: Skip	4054
1585641	Which Drugs Used: Inhalants Use	3271
1585644	Which Drugs Used: Street Opioids Use	2081
1585646	Which Drugs Used: Other Specify	660

Name: count, dtype: int64

In [78]:
```
se_q7= se_q7[~se_q7['answer_concept_id'].isin([903096, 903079])]

se_q7 = se_q7[['person_id', 'answer']]

se_q7 = se_q7.rename(columns={
    "answer": "recreational_drug_use",
})
```

In [79]:
```
#In general, how would you rate your satisfcation with your social activities and r
se_q8 = survey[survey['question_concept_id']== 1585735].copy()

print(se_q8.shape)

se_q8[['answer_concept_id', 'answer']].value_counts()
```

```
(259524, 9)
```

Out[79]:

answer_concept_id	answer	
1585737	Social Satisfaction: Very Good	84155
1585738	Social Satisfaction: Good	77913
1585736	Social Satisfaction: Excellent	42733
1585739	Social Satisfaction: Fair	39708
1585740	Social Satisfaction: Poor	13470
903096	PMI: Skip	1545

Name: count, dtype: int64

In [80]:
```
se_q8 = se_q8[~se_q8['answer_concept_id'].isin([903096])]
```

```
se_q8 = se_q8[['person_id', 'answer']]

se_q8 = se_q8.rename(columns={
    "answer": "social_health",
})
```

In [81]:
```
survey_cleaned = se_q1.merge(
    se_q2,
    on='person_id',
    how='inner')

survey_cleaned = survey_cleaned.merge(
    se_q3,
    on='person_id',
    how='inner')

survey_cleaned = survey_cleaned.merge(
    se_q4,
    on='person_id',
    how='inner')

survey_cleaned = survey_cleaned.merge(
    se_q5,
    on='person_id',
    how='inner')

survey_cleaned = survey_cleaned.merge(
    se_q6,
    on='person_id',
    how='inner')

survey_cleaned = survey_cleaned.merge(
    se_q7,
    on='person_id',
    how='inner')

survey_cleaned = survey_cleaned.merge(
    se_q8,
    on='person_id',
    how='inner')
```

In [82]:
```
# survey_cleaned
```

Smoking

In [83]:
```
#Have you ever smoked a traditional cigar, cigarillo, or filtered cigar, even one o
# Do you now smoke a traditional cigar, cigarillo, or filtered cigar?
smoker = survey[survey['question_concept_id'].isin([1586174, 1586177])].copy()
print(smoker.shape)

smoker[['answer_concept_id', 'answer']].value_counts()
```

```
(380104, 9)
```

Out[83]:
```
answer_concept_id  answer
1586176            Cigar Smoke Participant: No           144042
1586175            Cigar Smoke Participant: Yes          114791
1586180            Current Cigar Frequency: Not At All    96774
1586179            Current Cigar Frequency: Some Days     16013
1586178            Current Cigar Frequency: Every Day      2928
903087             PMI: Dont Know                          2317
903096             PMI: Skip                               1915
903079             PMI: Prefer Not To Answer               1324
Name: count, dtype: int64
```

In [84]:
```
#Must give a valid answer
smoker = smoker[~smoker['answer_concept_id'].isin([903096, 903087, 903079])]
smoker[['answer_concept_id', 'answer']].value_counts()
```

Out[84]:
```
answer_concept_id  answer
1586176            Cigar Smoke Participant: No           144042
1586175            Cigar Smoke Participant: Yes          114791
1586180            Current Cigar Frequency: Not At All    96774
1586179            Current Cigar Frequency: Some Days     16013
1586178            Current Cigar Frequency: Every Day      2928
Name: count, dtype: int64
```

In [85]:
```
#Find patients who answered both smoking questions
valid_ids = (
    smoker.groupby('person_id')['question_concept_id']
    .nunique()
    .loc[lambda x: x == 2]
    .index
)

smoker_both = smoker[smoker['person_id'].isin(valid_ids)].copy()
smoker_both.shape
```

Out[85]:
```
(226208, 9)
```

In [86]:
```
smoker[['answer_concept_id', 'answer']].value_counts()
```

Out[86]:
```
answer_concept_id  answer
1586176            Cigar Smoke Participant: No           144042
1586175            Cigar Smoke Participant: Yes          114791
1586180            Current Cigar Frequency: Not At All    96774
1586179            Current Cigar Frequency: Some Days     16013
1586178            Current Cigar Frequency: Every Day      2928
Name: count, dtype: int64
```

In [87]:
```
# Initialize as NA
import numpy as np
smoker['current_smoker'] = np.nan

# YES conditions
yes_condition = (
((smoker['question'] == 'Cigar Smoking: Cigar Smoke Participant') &
(smoker['answer'] == 'Cigar Smoke Participant: Yes')) |
(smoker['question'] == 'Cigar Smoking: Current Cigar Frequency') &
(smoker['answer'].isin(['Current Cigar Frequency: Every Day ', \
```

```
'Current Cigar Frequency: Some Days'])))


# NO conditions
no_condition = (
((smoker['question'] == 'Cigar Smoking: Cigar Smoke Participant') &
(smoker['answer'] == 'Cigar Smoke Participant: No')) |
((smoker['question'] == 'Cigar Smoking: Current Cigar Frequency') &
(smoker['answer'] == 'Current Cigar Frequency: Not At All'))
)
```

In [88]:
```
#Make a binary variable
smoker.loc[yes_condition, 'current_smoker'] = 1
smoker.loc[no_condition, 'current_smoker'] = 0
```

In [89]:
```
smoker_cleaned = (
    smoker.sort_values('survey_datetime')
    .groupby('person_id')['current_smoker']
    .last()
    .reset_index()
).copy()

smoker_cleaned = smoker_cleaned[['person_id', 'current_smoker']]

smoker_cleaned.current_smoker.value_counts()
```

Out[89]:
```
current_smoker
0.0    193610
1.0     67733
Name: count, dtype: int64
```

In [90]:
```
smoker_cleaned = smoker_cleaned.dropna()
```

In [91]:
```
# smoker_cleaned
```

Conditions

In [92]:
```
conditions = dataset_34451249_condition_df.copy()
print(conditions.shape)
conditions.columns
```

```
(3680533, 20)
```

Out[92]:
```
Index(['person_id', 'condition_concept_id', 'standard_concept_name',
       'standard_concept_code', 'standard_vocabulary',
       'condition_start_datetime', 'condition_end_datetime',
       'condition_type_concept_id', 'condition_type_concept_name',
       'stop_reason', 'visit_occurrence_id', 'visit_occurrence_concept_name',
       'condition_source_value', 'condition_source_concept_id',
       'source_concept_name', 'source_concept_code', 'source_vocabulary',
       'condition_status_source_value', 'condition_status_concept_id',
       'condition_status_concept_name'],
      dtype='object')
```

In [93]:
```
# conditions.head()
```

In [94]:
```
conditions_filtered = conditions.merge(
    audit_merged_clean[["person_id", "index_date", "obs_window"]],
    on="person_id",
    how="inner"
)

##Only keep conditions that started within the patient observation window
conditions = conditions_filtered[
    (conditions_filtered["condition_start_datetime"] >= conditions_filtered["obs_wi
    (conditions_filtered["condition_start_datetime"] <= conditions_filtered["index_
]

#Keep the most recent condition per person, per different condition
conditions = conditions.sort_values(["person_id", "condition_concept_id", "conditio

conditions = conditions.drop_duplicates(
    subset=["person_id", "condition_concept_id"],
    keep="last"
)
```

In [95]:
```
conditions.shape
```

Out[95]: (132992, 22)

In [96]:
```
conditions[['condition_concept_id', 'standard_concept_name']].value_counts()[:20]
```

Out[96]:

condition_concept_id	standard_concept_name
320128	Essential hypertension
26703	
442077	Anxiety disorder
20341	
4282096	Major depression, single episode
12853	
440383	Depressive disorder
7772	
434613	Generalized anxiety disorder
7531	
436676	Posttraumatic stress disorder
4281	
4077577	Moderate recurrent major depression
4079	
4059290	Steatosis of liver
3832	
4113821	Anxiety state
3450	
4211231	Panic disorder without agoraphobia
2278	
4282316	Recurrent major depression
1930	
441542	Anxiety
1689	
435220	Severe recurrent major depression without psychotic features
1621	
4228802	Mild recurrent major depression
1560	
440083	Acute stress disorder
1541	
433440	Dysthymia
1526	
194984	Disease of liver
1506	
4141454	Recurrent major depression in partial remission
1358	
443414	Chronic post-traumatic stress disorder
1166	
197494	Viral hepatitis C
1108	

Name: count, dtype: int64

In [97]:
```
#Filter for hypertension
c1 = conditions[conditions['condition_concept_id'].isin([320128, 312648])]

#Remove people who have multiple hypertension conditions
c1 = c1.drop_duplicates(
    subset=["person_id"],
    keep="last"
)

c1['has_hypertension'] = 1

c1 = c1[['person_id', 'has_hypertension']]
```

```
# c1
```

In [98]:
```
#Filter for liver disease
c2 = conditions[conditions['condition_concept_id'].isin([4059290, 194984])]

#Remove people who have multiple liver diseases
c2 = c2.drop_duplicates(
    subset=["person_id"],
    keep="last"
)

c2['has_liver_disease'] = 1

c2 = c2[['person_id', 'has_liver_disease']]

# c2
```

In [99]:
```
#Filter for depression
c3 = conditions[conditions['condition_concept_id'].isin([4282096, 440383, 4077577,

#Remove people who have multiple depression conditions
c3 = c3.drop_duplicates(
    subset=["person_id"],
    keep="last"
)

c3['has_depression'] = 1

c3 = c3[['person_id', 'has_depression']]

# c3
```

In [100…
```
#Filter for anxiety
c4 = conditions[conditions['condition_concept_id'].isin([442077, 434613, 436676, 41

#Remove people who have multiple anxiety conditions
c4 = c4.drop_duplicates(
    subset=["person_id"],
    keep="last"
)

c4['has_anxiety'] = 1

c4 = c4[['person_id', 'has_anxiety']]

# c4
```

In [101…
```
conditions_cleaned = c1.merge(c2, on="person_id", how="outer")

conditions_cleaned = conditions_cleaned.merge(c3, on="person_id", how="outer")

conditions_cleaned = conditions_cleaned.merge(c4, on="person_id", how="outer")

# Replace missing values with 0
```

```
conditions_cleaned[["has_hypertension", "has_liver_disease", "has_depression", "has
    [["has_hypertension", "has_liver_disease", "has_depression", "has_anxiety"]].fi
```

In [102…
```
# conditions_cleaned
```

Medication

In [103…
```
#Data on patient's taking alcoholism medication
#Includes: naltrexone, acamprosate, and disulfiram
medication = dataset_67921512_drug_df.copy()
print(medication.shape)
medication.columns
```

```
(35157, 27)
```

Out[103…
```
Index(['person_id', 'drug_concept_id', 'standard_concept_name',
       'standard_concept_code', 'standard_vocabulary',
       'drug_exposure_start_datetime', 'drug_exposure_end_datetime',
       'verbatim_end_date', 'drug_type_concept_id', 'drug_type_concept_name',
       'stop_reason', 'refills', 'quantity', 'days_supply', 'sig',
       'route_concept_id', 'route_concept_name', 'lot_number',
       'visit_occurrence_id', 'visit_occurrence_concept_name',
       'drug_source_value', 'drug_source_concept_id', 'source_concept_name',
       'source_concept_code', 'source_vocabulary', 'route_source_value',
       'dose_unit_source_value'],
      dtype='object')
```

In [104…
```
# medication.head()
```

In [105…
```
medication_filtered = medication.merge(
    audit_merged_clean[["person_id", "index_date", "obs_window"]],
    on="person_id",
    how="inner"
)

##Only keep medications that started within the patient observation window
medication = medication_filtered[
    (medication_filtered["drug_exposure_start_datetime"] >= medication_filtered["ob
    (medication_filtered["drug_exposure_start_datetime"] <= medication_filtered["in
]
```

In [106…
```
#All the included drugs are perscribed to treat alcoholism, so if a patient is taki
#they would qualify. So, if they are in at least one row.
medication_cleaned = pd.DataFrame(
    medication['person_id'].unique(),
    columns=['person_id']
)

medication_cleaned['alcoholism_medication'] = 1
```

In [107…
```
# medication_cleaned
```

Combining Features

In [108…

```
print(audit_merged_clean.shape)

final_cleaned = person_cleaned.merge(
    audit_merged_clean,
    on="person_id",
    how="inner")
print(final_cleaned.shape)

final_cleaned = final_cleaned.merge(
    measurements_cleaned,
    on="person_id",
    how="inner")
print(final_cleaned.shape)

final_cleaned = final_cleaned.merge(
    survey_cleaned,
    on="person_id",
    how="inner")
print(final_cleaned.shape)

final_cleaned = final_cleaned.merge(
    smoker_cleaned,
    on="person_id",
    how="inner")
print(final_cleaned.shape)


#Want to include all people with and without conditions
final_cleaned = final_cleaned.merge(
    conditions_cleaned,
    on="person_id",
    how="left")
print(final_cleaned.shape)

#Those without a condition will be NA and we can change to a value of 0, meaning th
final_cleaned[["has_hypertension", "has_liver_disease", "has_depression", "has_anxi
    [["has_hypertension", "has_liver_disease", "has_depression", "has_anxiety"]].fi
print(final_cleaned.shape)


#Want to include all people on and off alcoholism treatment medication
final_cleaned = final_cleaned.merge(
    medication_cleaned,
    on="person_id",
    how="left")
print(final_cleaned.shape)

#Those not on the medication will be NA and we can change to a value of 0, meaning
final_cleaned['alcoholism_medication'] = final_cleaned\
    ['alcoholism_medication'].fillna(0)
print(final_cleaned.shape)
```

```
#Calculate age from when the patient took the alcohol survey (don't use their curre
final_cleaned["age"] = (
    (final_cleaned["index_date"] - final_cleaned["date_of_birth"]).dt.days / 365.25
).astype(int)
print(final_cleaned.shape)
```

```
(262087, 4)
(239299, 7)
(45234, 10)
(31219, 18)
(31174, 19)
(31174, 23)
(31174, 23)
(31174, 24)
(31174, 24)
(31174, 25)
```

In [109…
```
print(final_cleaned.shape)
# final_cleaned.head()
```

```
(31174, 25)
```

In [110…
```
final_cleaned.columns
```

Out[110…
```
Index(['person_id', 'race2', 'sex_at_birth_male', 'date_of_birth', 'audit_c',
       'index_date', 'obs_window', 'bmi_measure', 'ast_measure', 'alt_measure',
       'some_college', 'employment_status', 'household_income',
       'physical_health', 'mental_health', 'marrital_status',
       'recreational_drug_use', 'social_health', 'current_smoker',
       'has_hypertension', 'has_liver_disease', 'has_depression',
       'has_anxiety', 'alcoholism_medication', 'age'],
      dtype='object')
```

In [111…
```
final_cleaned.isna().sum()
```

Out[111…

```
person_id                0
race2                    0
sex_at_birth_male        0
date_of_birth            0
audit_c                  0
index_date               0
obs_window               0
bmi_measure              0
ast_measure              0
alt_measure              0
some_college             0
employment_status        0
household_income         0
physical_health          0
mental_health            0
marrital_status          0
recreational_drug_use    0
social_health            0
current_smoker           0
has_hypertension         0
has_liver_disease        0
has_depression           0
has_anxiety              0
alcoholism_medication    0
age                      0
dtype: int64
```

Save data to cloud for easy retrieval

In [2]:

```
# import os

# # 1. Get the bucket path
# bucket = os.getenv('M2_WORKSPACE_BUCKET')

# # 2. Save the dataframe (this overwrites any local file of the same name)
# final_cleaned.to_csv('final_cleaned.csv', index=False)

# # 3. Explicitly copy the file to the bucket path
# # Adding a trailing slash to the destination helps gsutil know it's a directory
# !gsutil cp ./final_cleaned.csv {bucket}/data/
```

EDA

Retrieve from cloud (don't need to run code above each time to generate df)

In [3]:

```
import os
import pandas as pd

# 1. Get the bucket path
```

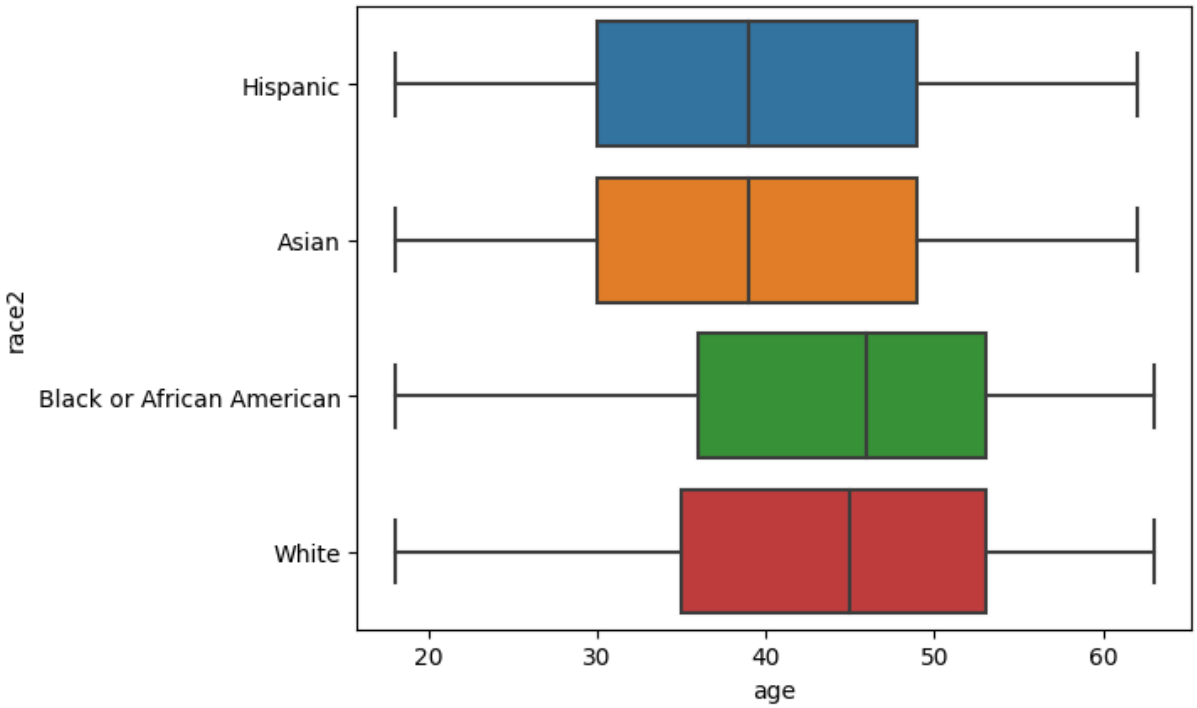
```
bucket = os.getenv('M2_WORKSPACE_BUCKET')

# 2. Copy the file from the Cloud Bucket back to your local environment
!gsutil cp {bucket}/data/final_cleaned.csv .

# 3. Load it into your dataframe
final_cleaned = pd.read_csv('final_cleaned.csv')

print(f"Data loaded successfully! Shape: {final_cleaned.shape}")
```

```
Copying file://None/data/final_cleaned.csv...
/ [1 files][  9.8 MiB/  9.8 MiB]
Operation completed over 1 objects/9.8 MiB.
Data loaded successfully! Shape: (31174, 25)
```

EDA continued

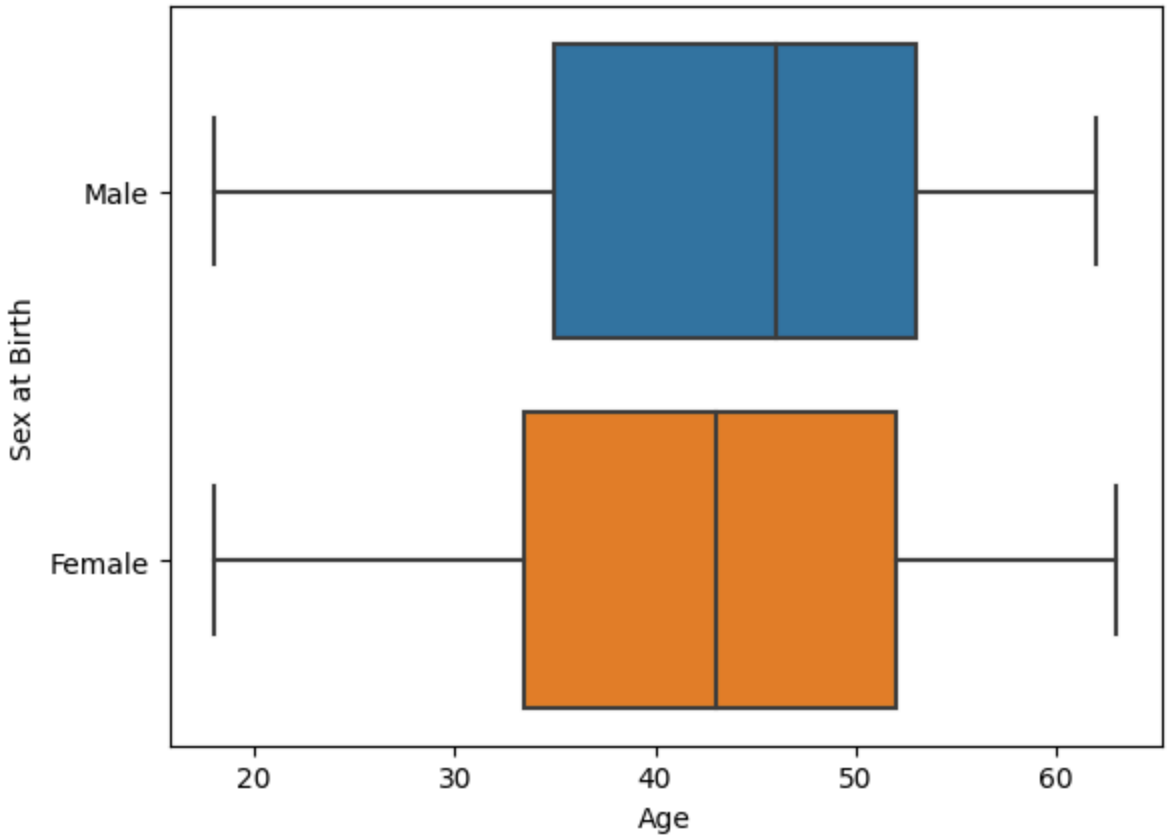
In [4]:
```
import seaborn as sns
sns.boxplot(final_cleaned, x = 'age', y = 'race2')
```

Out[4]: <Axes: xlabel='age', ylabel='race2'>



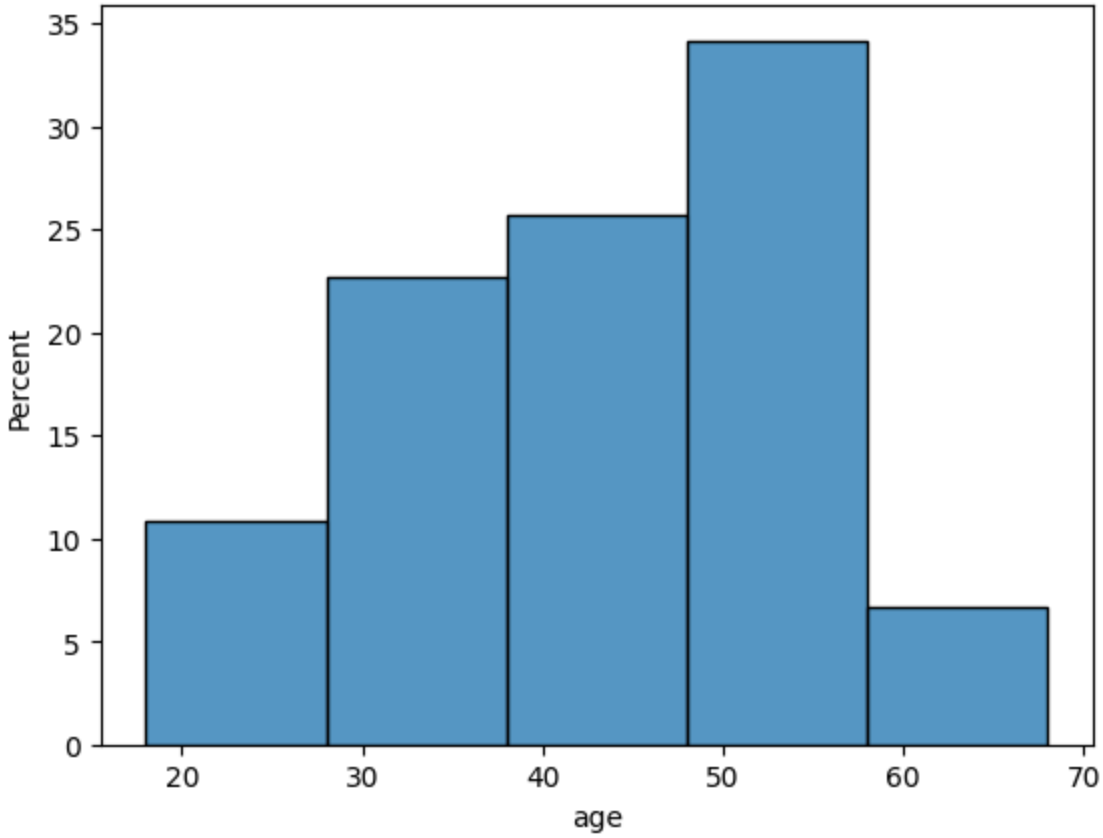
In [5]:
```
import matplotlib.pyplot as plt

sex_labeled = pd.DataFrame()
sex_labeled['Sex at Birth'] = final_cleaned['sex_at_birth_male'].map({1: 'Male', 0:
sex_labeled['age'] = final_cleaned['age']

sns.boxplot(data=sex_labeled, x='age', y='Sex at Birth')

plt.ylabel('Sex at Birth')   # <- change y label
plt.xlabel('Age')            # optional, for clarity
```

```
plt.show()
```

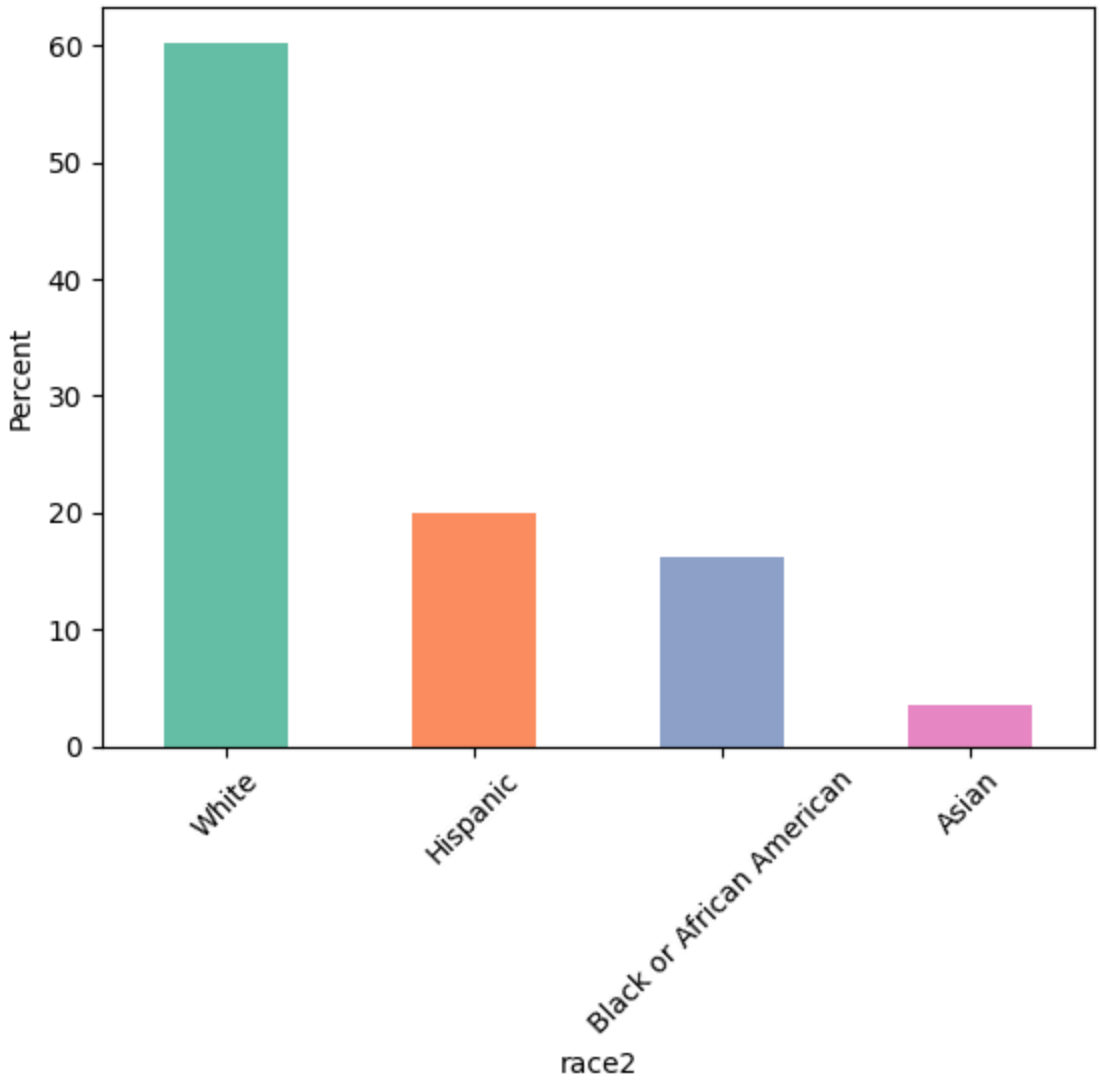


In [6]:
```
sns.histplot(data=final_cleaned, x='age', binwidth=10, stat='percent')
```

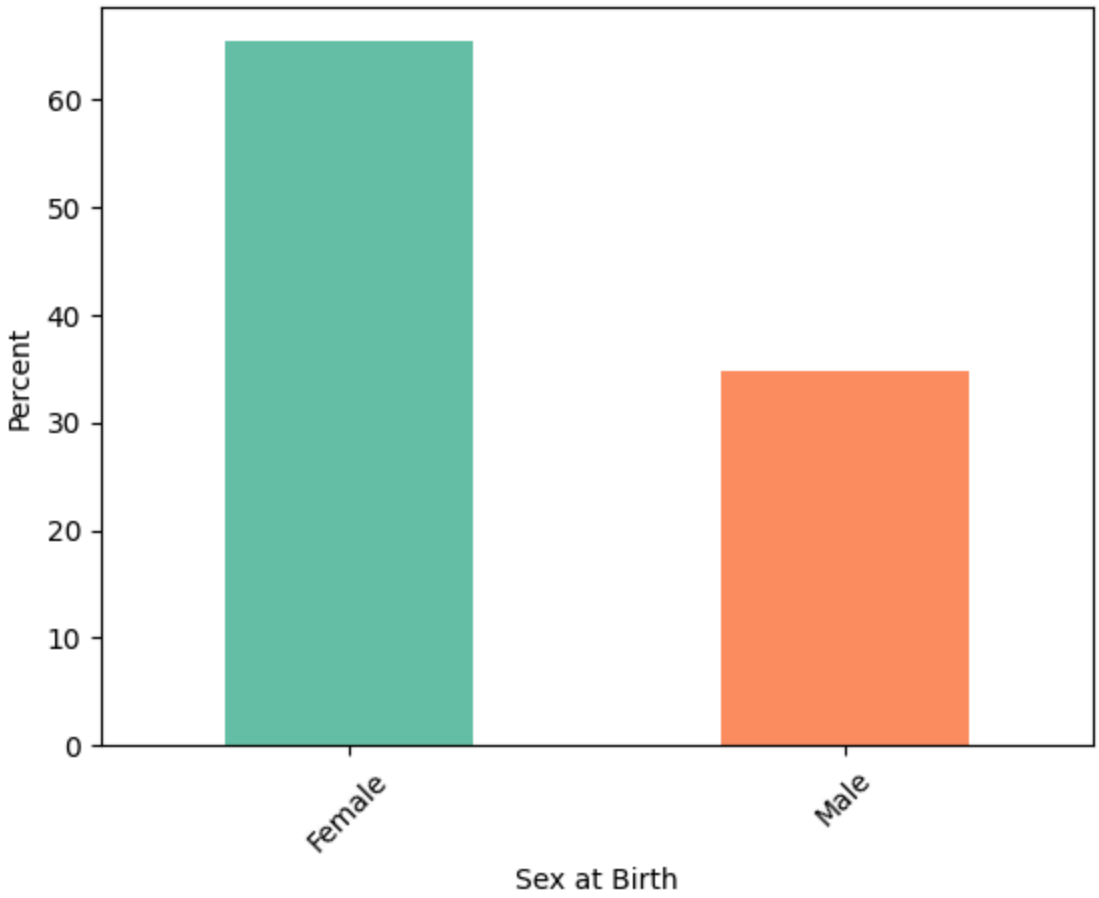
Out[6]:
```
<Axes: xlabel='age', ylabel='Percent'>
```



In [7]:
```
import matplotlib.pyplot as plt

percent = final_cleaned['race2'].value_counts(normalize=True) * 100

percent.plot(kind='bar', color=sns.color_palette('Set2', len(percent)))

plt.ylabel('Percent')
plt.xlabel('race2')
plt.xticks(rotation=45)
plt.show()
```



In [8]:
```
import matplotlib.pyplot as plt
import seaborn as sns

# Map values
sex_labeled = final_cleaned['sex_at_birth_male'].map({1: 'Male', 0: 'Female'}).copy

percent = sex_labeled.value_counts(normalize=True) * 100

percent.plot(kind='bar', color=sns.color_palette('Set2', len(percent)))

plt.ylabel('Percent')
plt.xlabel('Sex at Birth')
plt.xticks(rotation=45)
plt.show()
```

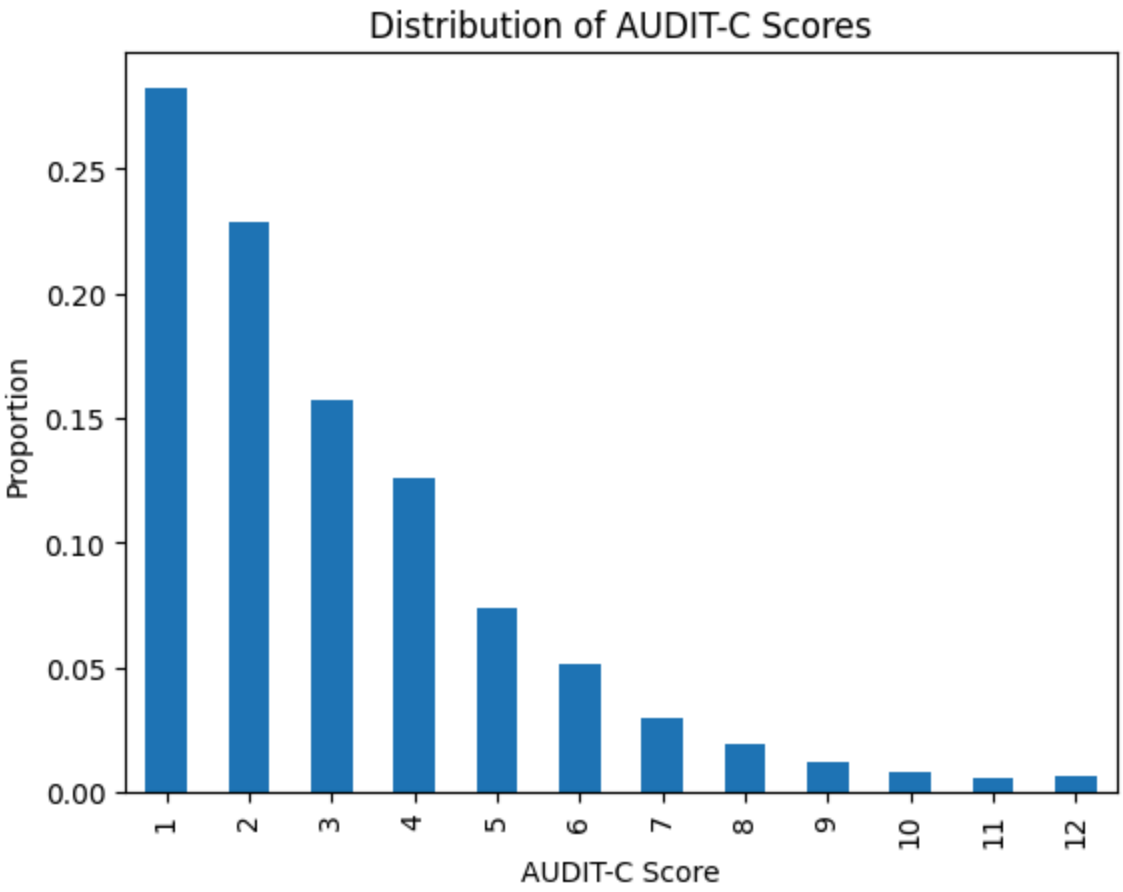


In [9]:
```
final_cleaned['audit_c'].value_counts(normalize=True).sort_index()
```

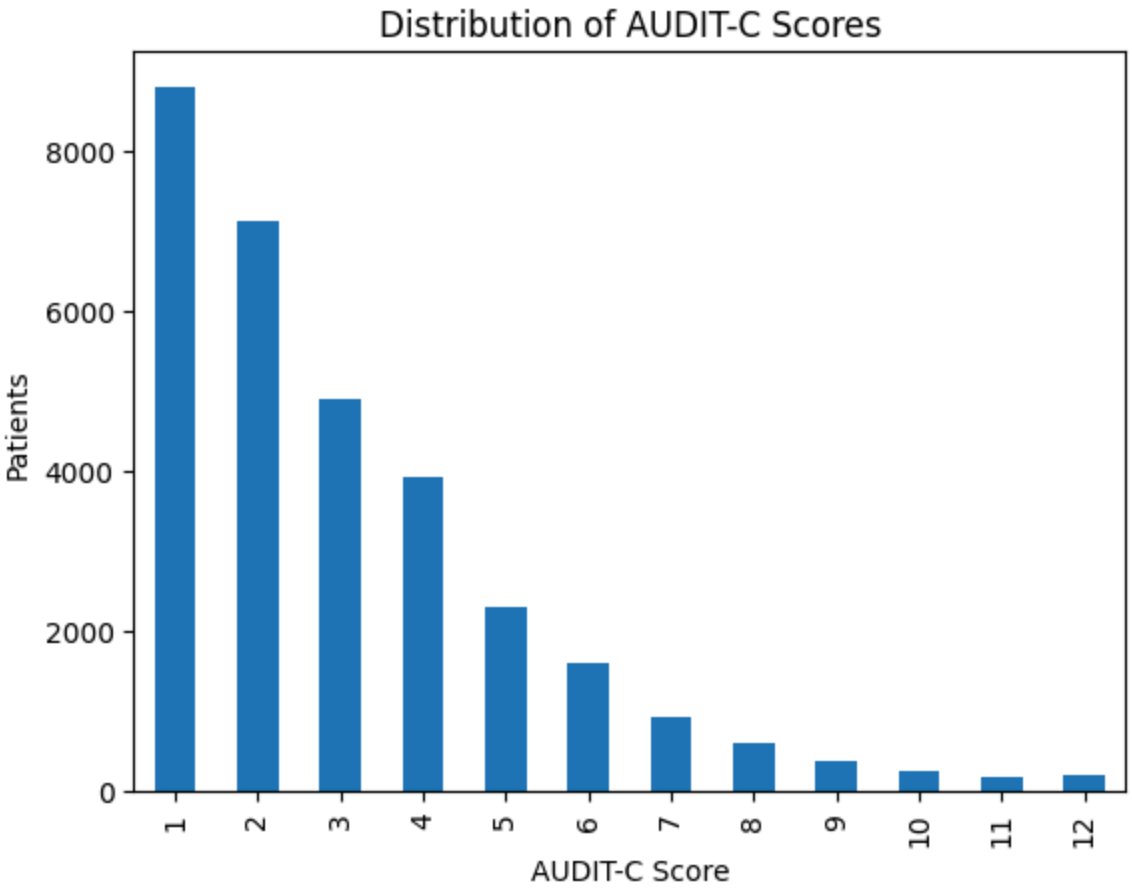
Out[9]:
```
audit_c
1     0.282254
2     0.228652
3     0.156958
4     0.126259
5     0.074100
6     0.051036
7     0.029736
8     0.019247
9     0.012029
10    0.007795
11    0.005229
12    0.006704
Name: proportion, dtype: float64
```

In [10]:
```
import matplotlib.pyplot as plt

counts = final_cleaned['audit_c'].value_counts(normalize=True).sort_index()

counts.plot(kind='bar')

plt.xlabel('AUDIT-C Score')
plt.ylabel('Proportion')
plt.title('Distribution of AUDIT-C Scores')
```

```
plt.show()
```



In [11]:
```
import matplotlib.pyplot as plt

counts = final_cleaned['audit_c'].value_counts().sort_index()

counts.plot(kind='bar')

plt.xlabel('AUDIT-C Score')
plt.ylabel('Patients')
plt.title('Distribution of AUDIT-C Scores')

plt.show()
```



Final Feature Cleaning

In [12]:
```
q1_map = {
    "General Physical Health: Poor": 0,
    "General Physical Health: Fair": 1,
    "General Physical Health: Good": 2,
    "General Physical Health: Very Good": 3,
    "General Physical Health: Excellent": 4
}

q2_map = {
    "General Mental Health: Poor": 0,
    "General Mental Health: Fair": 1,
    "General Mental Health: Good": 2,
    "General Mental Health: Very Good": 3,
    "General Mental Health: Excellent": 4
}

q3_map = {
    "Social Satisfaction: Poor": 0,
    "Social Satisfaction: Fair": 1,
    "Social Satisfaction: Good": 2,
    "Social Satisfaction: Very Good": 3,
    "Social Satisfaction: Excellent": 4
}
```

```
final_cleaned["physical_health_score"] = final_cleaned["physical_health"].map(q1_ma
final_cleaned["mental_health_score"] = final_cleaned["mental_health"].map(q2_map)
final_cleaned["social_health_score"] = final_cleaned["social_health"].map(q3_map)
```

In [13]:
```
final_cleaned.describe()
```

Out[13]:

	person_id	sex_at_birth_male	audit_c	bmi_measure	ast_measure	alt_me
count	3.117400e+04	31174.000000	31174.000000	31174.000000	31174.000000	31174.00
mean	3.550987e+06	0.346795	3.078495	31.012217	25.912818	28.21
std	2.479409e+06	0.475958	2.209818	8.378184	34.062982	39.89
min	1.000095e+06	0.000000	1.000000	10.680000	6.000000	6.00
25%	1.699254e+06	0.000000	1.000000	24.900000	16.000000	14.00
50%	2.626405e+06	0.000000	2.000000	29.405000	20.000000	20.00
75%	4.814892e+06	1.000000	4.000000	35.627500	26.000000	30.00
max	9.999678e+06	1.000000	12.000000	79.900000	1429.000000	1420.00

In [14]:
```
final_cleaned_numeric = final_cleaned[['audit_c', 'bmi_measure', 'ast_measure', 'al
                                       'current_smoker', 'has_hypertension', 'has_l
                                       'has_anxiety', 'alcoholism_medication', 'age
                                      'physical_health_score', 'mental_health_score
                                       'some_college'
                                      ]]
```

In [15]:
```
final_cleaned_numeric.corr()
```

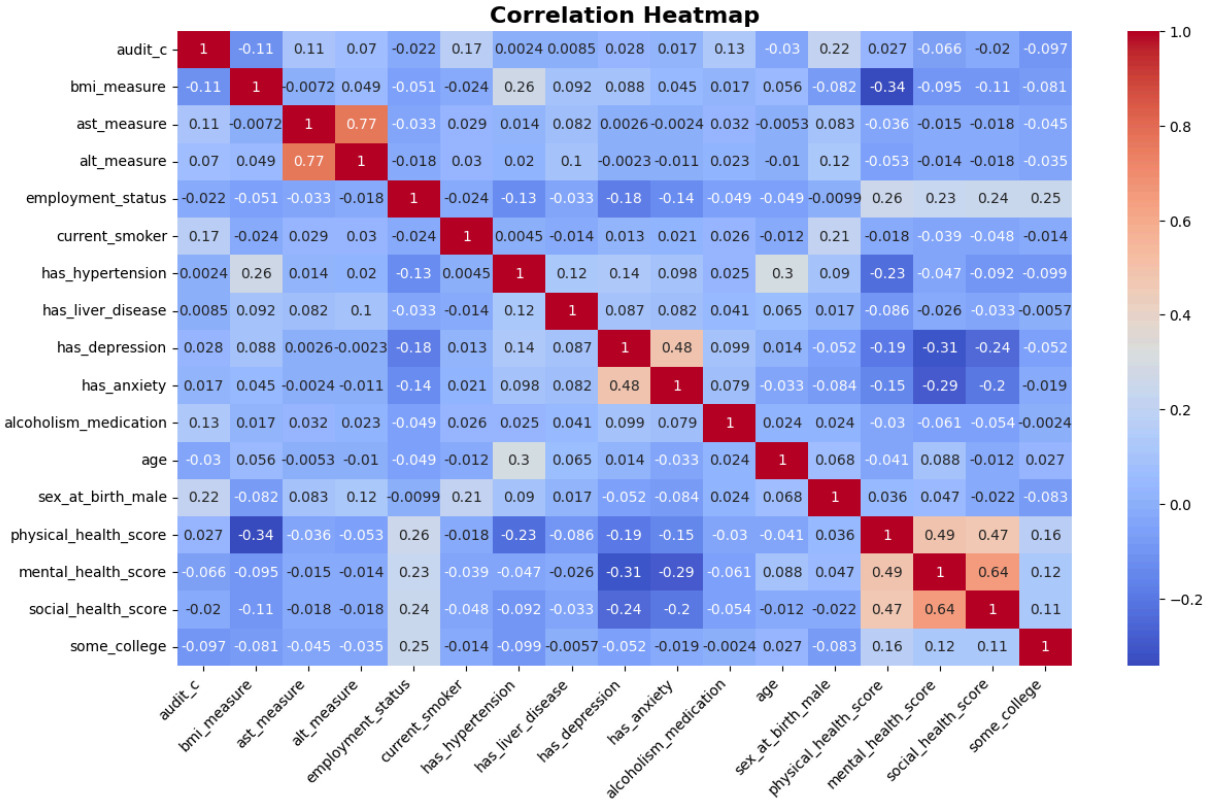
Out[15]:

	audit_c	bmi_measure	ast_measure	alt_measure	employment_sta
audit_c	1.000000	-0.107950	0.107944	0.070472	-0.0218
bmi_measure	-0.107950	1.000000	-0.007213	0.049214	-0.0510
ast_measure	0.107944	-0.007213	1.000000	0.768012	-0.0329
alt_measure	0.070472	0.049214	0.768012	1.000000	-0.0183
employment_status	-0.021894	-0.051096	-0.032903	-0.018382	1.0000
current_smoker	0.170876	-0.023586	0.029471	0.029618	-0.0238
has_hypertension	0.002399	0.261631	0.014157	0.020312	-0.1296
has_liver_disease	0.008473	0.092377	0.082049	0.103417	-0.0332
has_depression	0.027938	0.088454	0.002560	-0.002306	-0.1776
has_anxiety	0.017151	0.044938	-0.002444	-0.011328	-0.1358
alcoholism_medication	0.133763	0.016885	0.031907	0.022570	-0.0493
age	-0.029929	0.055992	-0.005315	-0.010232	-0.0487
sex_at_birth_male	0.223240	-0.081845	0.083213	0.119341	-0.0098
physical_health_score	0.027101	-0.341540	-0.036320	-0.053229	0.2600
mental_health_score	-0.065528	-0.094658	-0.014848	-0.014121	0.2340
social_health_score	-0.019898	-0.105434	-0.018024	-0.018146	0.2398
some_college	-0.097152	-0.080659	-0.045205	-0.035498	0.2471

In [16]:

```
corr_matrix = final_cleaned_numeric.corr()
fig, ax = plt.subplots(figsize=(14, 8))

sns.heatmap(corr_matrix, annot=True, cmap='coolwarm', ax=ax)
plt.title('Correlation Heatmap', fontsize = 16, fontweight ='bold')
plt.xticks(rotation=45, ha='right')
```

Out[16]:
```
(array([ 0.5,  1.5,  2.5,  3.5,  4.5,  5.5,  6.5,  7.5,  8.5,  9.5, 10.5,
        11.5, 12.5, 13.5, 14.5, 15.5, 16.5]),
 [Text(0.5, 0, 'audit_c'),
  Text(1.5, 0, 'bmi_measure'),
  Text(2.5, 0, 'ast_measure'),
  Text(3.5, 0, 'alt_measure'),
  Text(4.5, 0, 'employment_status'),
  Text(5.5, 0, 'current_smoker'),
  Text(6.5, 0, 'has_hypertension'),
  Text(7.5, 0, 'has_liver_disease'),
  Text(8.5, 0, 'has_depression'),
  Text(9.5, 0, 'has_anxiety'),
  Text(10.5, 0, 'alcoholism_medication'),
  Text(11.5, 0, 'age'),
  Text(12.5, 0, 'sex_at_birth_male'),
  Text(13.5, 0, 'physical_health_score'),
  Text(14.5, 0, 'mental_health_score'),
  Text(15.5, 0, 'social_health_score'),
  Text(16.5, 0, 'some_college')])
```



Modeling

In [17]:
```
# List your categorical columns
categorical_cols = ['race2', 'household_income',
#                     'physical_health', 'mental_health', 'social_health',  'educat
                    'marrital_status', 'recreational_drug_use']

# Perform One-Hot Encoding
# drop_first=True is CRITICAL for Linear Regression to avoid perfect multicollinear
```

```
final_cleaned_dummies = pd.get_dummies(
    final_cleaned,
    columns=categorical_cols,
    drop_first=True
)

# Create the specific interaction columns
final_cleaned_dummies['age_sex_at_birth_male'] = final_cleaned_dummies['age'] * fin

#Drop non-predictor columns
final_cleaned_dummies = final_cleaned_dummies.drop('person_id', axis = 1)
final_cleaned_dummies = final_cleaned_dummies.drop('date_of_birth', axis = 1)
final_cleaned_dummies = final_cleaned_dummies.drop('index_date', axis = 1)
final_cleaned_dummies = final_cleaned_dummies.drop('obs_window', axis = 1)

# Preview all columns final columns
print(final_cleaned_dummies.columns)
```

```
Index(['sex_at_birth_male', 'audit_c', 'bmi_measure', 'ast_measure',
       'alt_measure', 'some_college', 'employment_status', 'physical_health',
       'mental_health', 'social_health', 'current_smoker', 'has_hypertension',
       'has_liver_disease', 'has_depression', 'has_anxiety',
       'alcoholism_medication', 'age', 'physical_health_score',
       'mental_health_score', 'social_health_score',
       'race2_Black or African American', 'race2_Hispanic', 'race2_White',
       'household_income_Annual Income: 10k 25k',
       'household_income_Annual Income: 150k 200k',
       'household_income_Annual Income: 25k 35k',
       'household_income_Annual Income: 35k 50k',
       'household_income_Annual Income: 50k 75k',
       'household_income_Annual Income: 75k 100k',
       'household_income_Annual Income: less 10k',
       'household_income_Annual Income: more 200k',
       'marrital_status_Current Marital Status: Married',
       'marrital_status_Current Marital Status: Never Married',
       'marrital_status_Current Marital Status: Separated',
       'marrital_status_Current Marital Status: Widowed',
       'recreational_drug_use_Which Drugs Used: Hallucinogens Use',
       'recreational_drug_use_Which Drugs Used: Inhalants Use',
       'recreational_drug_use_Which Drugs Used: Marijuana Use',
       'recreational_drug_use_Which Drugs Used: Methamphetamine Use',
       'recreational_drug_use_Which Drugs Used: None Of These Drugs',
       'recreational_drug_use_Which Drugs Used: Other Specify',
       'recreational_drug_use_Which Drugs Used: Prescription Opioids Use',
       'recreational_drug_use_Which Drugs Used: Prescription Stimulants Use',
       'recreational_drug_use_Which Drugs Used: Sedatives Use',
       'recreational_drug_use_Which Drugs Used: Street Opioids Use',
       'age_sex_at_birth_male'],
      dtype='object')
```

In [18]:

```
features = ['bmi_measure', 'ast_measure', 'alt_measure', 'some_college', 'employmen
            'has_hypertension', 'has_liver_disease', 'has_depression', 'has_anxiety
            'alcoholism_medication', 'age', 'sex_at_birth_male',\

           'race2_Black or African American', 'race2_Hispanic', 'race2_White',\
```

```
            'household_income_Annual Income: 10k 25k', 'household_income_Annual Inco
            'household_income_Annual Income: 35k 50k', 'household_income_Annual Inco
            'household_income_Annual Income: less 10k', 'household_income_Annual Inc

            'physical_health_score', 'mental_health_score', 'social_health_score',

            'marrital_status_Current Marital Status: Married', 'marrital_status_Curr
             'marrital_status_Current Marital Status: Separated', 'marrital_status_C

            'recreational_drug_use_Which Drugs Used: Hallucinogens Use',
            'recreational_drug_use_Which Drugs Used: Inhalants Use',
            'recreational_drug_use_Which Drugs Used: Marijuana Use',
            'recreational_drug_use_Which Drugs Used: Methamphetamine Use',
            'recreational_drug_use_Which Drugs Used: None Of These Drugs',
            'recreational_drug_use_Which Drugs Used: Other Specify',
            'recreational_drug_use_Which Drugs Used: Prescription Opioids Use',
            'recreational_drug_use_Which Drugs Used: Prescription Stimulants Use',
            'recreational_drug_use_Which Drugs Used: Sedatives Use',
            'recreational_drug_use_Which Drugs Used: Street Opioids Use',

            #Interaction terms
            'age_sex_at_birth_male'
            ]


target = 'audit_c'

x = final_cleaned_dummies[features]
#Only numeric variables
# x = final_cleaned_dummies[['bmi_measure', 'ast_measure', 'alt_measure', 'employme
#                                         'has_hypertension', 'has_liver_disease', '
#                                         'alcoholism_medication', 'age']]
y = final_cleaned_dummies[target]
```

In [19]:
```
#split into training and test sets
from sklearn.model_selection import train_test_split
s
x_train, x_test, y_train, y_test = train_test_split(x, y, test_size=0.2, random_sta
```

In [20]:
```
final_cleaned_dummies.isna().sum()
```

Out[20]:
```
sex_at_birth_male                                                  0
audit_c                                                            0
bmi_measure                                                        0
ast_measure                                                        0
alt_measure                                                        0
some_college                                                       0
employment_status                                                  0
physical_health                                                    0
mental_health                                                      0
social_health                                                      0
current_smoker                                                     0
has_hypertension                                                   0
has_liver_disease                                                  0
has_depression                                                     0
has_anxiety                                                        0
alcoholism_medication                                              0
age                                                                0
physical_health_score                                              0
mental_health_score                                                0
social_health_score                                                0
race2_Black or African American                                    0
race2_Hispanic                                                     0
race2_White                                                        0
household_income_Annual Income: 10k 25k                            0
household_income_Annual Income: 150k 200k                          0
household_income_Annual Income: 25k 35k                            0
household_income_Annual Income: 35k 50k                            0
household_income_Annual Income: 50k 75k                            0
household_income_Annual Income: 75k 100k                           0
household_income_Annual Income: less 10k                           0
household_income_Annual Income: more 200k                          0
marrital_status_Current Marital Status: Married                    0
marrital_status_Current Marital Status: Never Married              0
marrital_status_Current Marital Status: Separated                  0
marrital_status_Current Marital Status: Widowed                    0
recreational_drug_use_Which Drugs Used: Hallucinogens Use          0
recreational_drug_use_Which Drugs Used: Inhalants Use              0
recreational_drug_use_Which Drugs Used: Marijuana Use              0
recreational_drug_use_Which Drugs Used: Methamphetamine Use        0
recreational_drug_use_Which Drugs Used: None Of These Drugs        0
recreational_drug_use_Which Drugs Used: Other Specify              0
recreational_drug_use_Which Drugs Used: Prescription Opioids Use   0
recreational_drug_use_Which Drugs Used: Prescription Stimulants Use  0
recreational_drug_use_Which Drugs Used: Sedatives Use              0
recreational_drug_use_Which Drugs Used: Street Opioids Use         0
age_sex_at_birth_male                                              0
dtype: int64
```

Linear Regression Models

In [21]:
```
import numpy as np
#Baselines by just guessing the average each time
```

```
print(
    'RMSE by guessing average each time:',
    round(np.sqrt(((final_cleaned_dummies['audit_c'] - final_cleaned_dummies['audit
)

print(
    'MAE by guessing average each time:',
    round(abs(final_cleaned_dummies['audit_c'] - final_cleaned_dummies['audit_c'].m
)

from sklearn.metrics import r2_score

y_true = final_cleaned_dummies['audit_c']
y_pred_baseline = [y_true.mean()] * len(y_true)

print(
    'R-squared by guessing average each time:',
    round(r2_score(y_true, y_pred_baseline), 4)
)
```

```
RMSE by guessing average each time: 2.2098
MAE by guessing average each time: 1.6912
R-squared by guessing average each time: 0.0
```

In [22]:

```
from sklearn.model_selection import cross_validate
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import make_pipeline
import warnings
import random
import numpy as np

def cross_validation_LR(model):
    warnings.filterwarnings('ignore')
    # Note: random.seed affects Python's random, but sklearn uses np.random
    # or the model's random_state for CV splitting.
    random.seed(42)
    np.random.seed(42)

    pipeline = make_pipeline(
        StandardScaler(),
        model
    )

    # Changed scoring to neg_mean_absolute_error
    results = cross_validate(
        pipeline,
        x_train,
        y_train,
        cv=5,
        scoring='neg_root_mean_squared_error'
    )

    # Scikit-learn returns negative RMSE, so we negate it
    rmse_scores = -results['test_score']

    print(f"Validatiom RMSE: {rmse_scores.mean():.4f}")
```

```
    return rmse_scores.mean() # Returning the mean RMSE as requested
```

In [133…
```
#Sklearn Linear Regression function uses SVD (Singular Value Decompositon) by defau
from sklearn.linear_model import LinearRegression

print('SVD Regression')
lr_model = cross_validation_LR(LinearRegression())
```

```
SVD Regression
Validatiom RMSE: 2.0325
```

In [134…
```
# Version that uses SGD (Stochastic Gradient Descent)
from sklearn.linear_model import SGDRegressor

sgd_model = SGDRegressor()

print('SGD Regression')
lr_model = cross_validation_LR(sgd_model)
```

```
SGD Regression
Validatiom RMSE: 2.0403
```

In [135…
```
#Testing different tol values for SGD Regression
print('SGD Regression with Different Tol')
for tol in [1e-7, 1e-5, 1e-3, 1e-1]:
    print('Tol value:', tol)
    sgd_model = SGDRegressor(tol = tol)
    cross_validation_LR(sgd_model)
    print('\n')
```

```
SGD Regression with Different Tol
Tol value: 1e-07
Validatiom RMSE: 2.0386


Tol value: 1e-05
Validatiom RMSE: 2.0386


Tol value: 0.001
Validatiom RMSE: 2.0403


Tol value: 0.1
Validatiom RMSE: 2.0434
```

In [136…
```
#Testing different eta0 values for SGD Regression
print('SGD Regression with Different Eta0')
for eta0 in [1e-1, 1e-2, 1e-3, 1e-4]:
    print('Eta0 value:', eta0)
    sgd_model = SGDRegressor(eta0 = eta0, tol = 1e-5)
    cross_validation_LR(sgd_model)
    print('\n')
```

```
SGD Regression with Different Eta0
Eta0 value: 0.1
Validatiom RMSE: 2.1633


Eta0 value: 0.01
Validatiom RMSE: 2.0386


Eta0 value: 0.001
Validatiom RMSE: 2.0325


Eta0 value: 0.0001
Validatiom RMSE: 2.0332
```

In [137…

```
# Version that uses SGD with Ridge Regularization
# Alpha is penalty term
print('SGD Regression with Ridge Regularization')
for alpha in [1e-6, 1e-4, 1e-2, 1, 100]:
    print('Alpha value:', alpha)
    sgd_model = SGDRegressor(tol = 1e-5, eta0 = 1e-3, penalty = 'l2', alpha = alpha
    cross_validation_LR(sgd_model)
    print('\n')
```

```
SGD Regression with Ridge Regularization
Alpha value: 1e-06
Validatiom RMSE: 2.0325


Alpha value: 0.0001
Validatiom RMSE: 2.0325


Alpha value: 0.01
Validatiom RMSE: 2.0327


Alpha value: 1
Validatiom RMSE: 2.0684


Alpha value: 100
Validatiom RMSE: 2.2027
```

In [138…

```
# Version that uses SGD with Lasso Regularization
# Alpha is penalty term
print('SGD Regression with Lasso Regularization')
for alpha in [1e-6, 1e-4, 1e-2, 1, 100]:
    print('Alpha value:', alpha)
    sgd_model = SGDRegressor(tol = 1e-5, eta0 = 1e-3, penalty = 'l1', alpha = alpha
```

```
    cross_validation_LR(sgd_model)
    print('\n')
```

```
SGD Regression with Lasso Regularization
Alpha value: 1e-06
Validatiom RMSE: 2.0325


Alpha value: 0.0001
Validatiom RMSE: 2.0326


Alpha value: 0.01
Validatiom RMSE: 2.0348


Alpha value: 1
Validatiom RMSE: 2.2082


Alpha value: 100
Validatiom RMSE: 2.2082
```

In [139…

```
# Version that uses SGD with Elastic Net Regularization
# Alpha is penalty term
print('SGD Regression with Elastic Net Regularization')
for alpha in [1e-6, 1e-4, 1e-2, 1, 100]:
    print('Alpha value:', alpha)
    sgd_model = SGDRegressor(tol = 1e-5, eta0 = 1e-3, penalty = 'elasticnet', l1_ra
    cross_validation_LR(sgd_model)
    print('\n')
```

```
SGD Regression with Elastic Net Regularization
Alpha value: 1e-06
Validatiom RMSE: 2.0325


Alpha value: 0.0001
Validatiom RMSE: 2.0325


Alpha value: 0.01
Validatiom RMSE: 2.0329


Alpha value: 1
Validatiom RMSE: 2.1165


Alpha value: 100
Validatiom RMSE: 2.2082
```

In [140…
```
#Trying Linear Model with best hyperparameters on the testing set
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

pipeline = make_pipeline(
    StandardScaler(),
    LinearRegression()
)
pipeline.fit(x_train, y_train)
y_pred = pipeline.predict(x_test)

# Calculate Regression Metrics
mae = mean_absolute_error(y_test, y_pred)
rmse = np.sqrt(mean_squared_error(y_test, y_pred))
r2 = r2_score(y_test, y_pred)

print("\033[1m\n--- Final Regression Model Performance on Test Set ---\033[0m")
print(f"Testing Root Mean Squared Error (RMSE): {rmse:.4f}")
print(f"Testing Mean Absolute Error (MAE): {mae:.4f}")
print(f"Testing R-squared: {r2:.4f}")
```

```
--- Final Regression Model Performance on Test Set ---
Testing Root Mean Squared Error (RMSE): 2.0543
Testing Mean Absolute Error (MAE): 1.5674
Testing R-squared: 0.1404
```

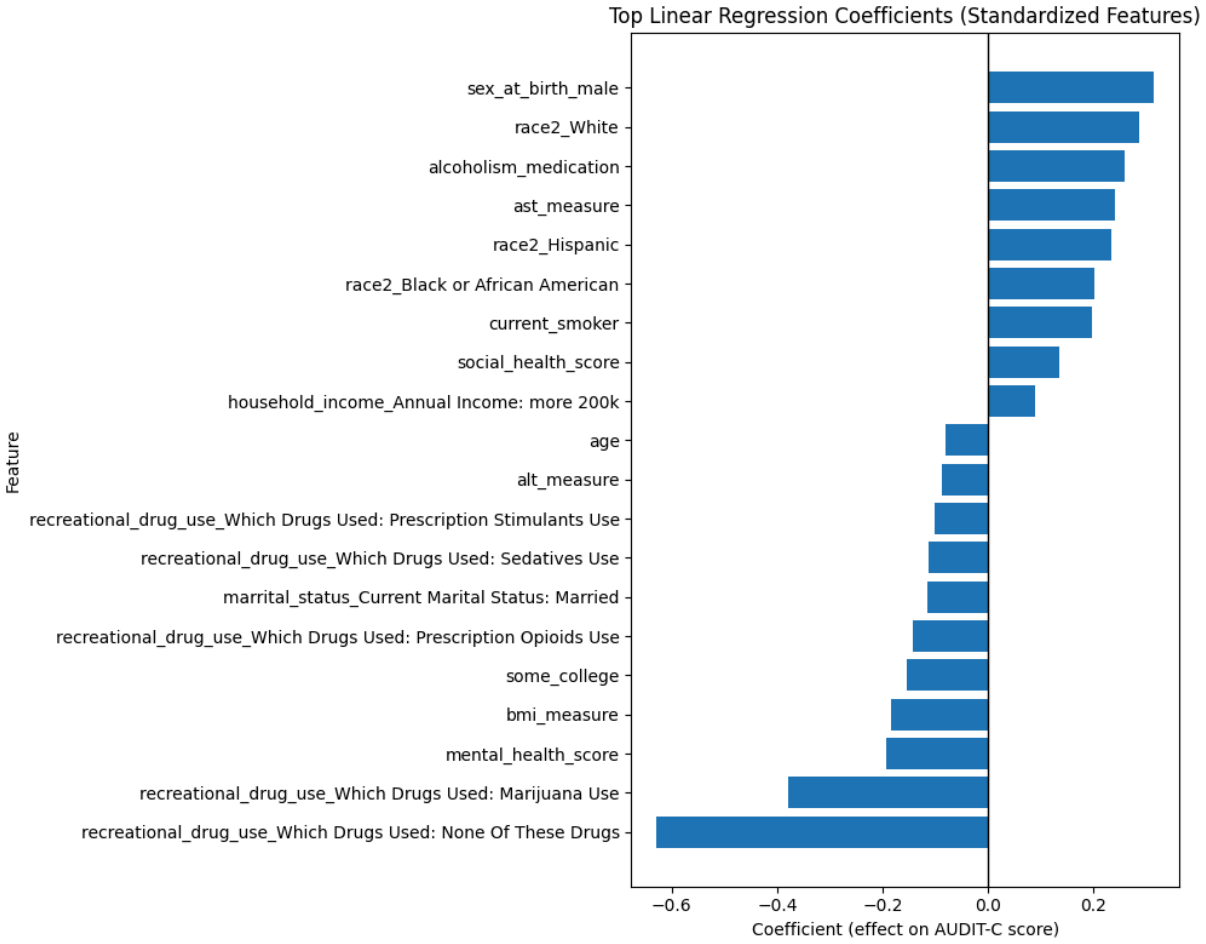
In [141…
```
#Coefficients for Linear Model:
# Because of scaling:

# Coefficients represent change in AUDIT-C per 1 standard deviation increase in the
# NOT per raw unit

# Get the linear regression model from the pipeline
lr = pipeline.named_steps['linearregression']

# Coefficients
coefs = lr.coef_

# Intercept
intercept = lr.intercept_

coef_df = pd.DataFrame({
    'feature': x_train.columns,
    'coefficient': coefs
})

# Sort by importance (absolute value)
coef_df['abs_coef'] = coef_df['coefficient'].abs()
coef_df = coef_df.sort_values(by='abs_coef', ascending=False)

print(coef_df)
```

	feature	coefficient	abs_coef
35	recreational_drug_use_Which Drugs Used: None O...	-0.629468	0.629468
33	recreational_drug_use_Which Drugs Used: Mariju...	-0.378417	0.378417
12	sex_at_birth_male	0.316318	0.316318
15	race2_White	0.287177	0.287177
10	alcoholism_medication	0.260602	0.260602
1	ast_measure	0.241752	0.241752
14	race2_Hispanic	0.235382	0.235382
13	race2_Black or African American	0.201567	0.201567
5	current_smoker	0.197916	0.197916
25	mental_health_score	-0.193146	0.193146
0	bmi_measure	-0.182453	0.182453
3	some_college	-0.154489	0.154489
37	recreational_drug_use_Which Drugs Used: Prescr...	-0.141877	0.141877
26	social_health_score	0.136951	0.136951
27	marrital_status_Current Marital Status: Married	-0.114487	0.114487
39	recreational_drug_use_Which Drugs Used: Sedati...	-0.113150	0.113150
38	recreational_drug_use_Which Drugs Used: Prescr...	-0.101195	0.101195
23	household_income_Annual Income: more 200k	0.089925	0.089925
2	alt_measure	-0.085970	0.085970
11	age	-0.080014	0.080014
4	employment_status	0.072676	0.072676
31	recreational_drug_use_Which Drugs Used: Halluc...	-0.069122	0.069122
24	physical_health_score	0.066538	0.066538
22	household_income_Annual Income: less 10k	0.058293	0.058293
41	age_sex_at_birth_male	0.057903	0.057903
34	recreational_drug_use_Which Drugs Used: Metham...	-0.048271	0.048271
19	household_income_Annual Income: 35k 50k	-0.043409	0.043409
32	recreational_drug_use_Which Drugs Used: Inhala...	-0.043126	0.043126
20	household_income_Annual Income: 50k 75k	-0.041016	0.041016
18	household_income_Annual Income: 25k 35k	-0.039774	0.039774
16	household_income_Annual Income: 10k 25k	-0.039056	0.039056
6	has_hypertension	0.036496	0.036496
40	recreational_drug_use_Which Drugs Used: Street...	-0.035334	0.035334
21	household_income_Annual Income: 75k 100k	-0.023318	0.023318
7	has_liver_disease	0.018688	0.018688
36	recreational_drug_use_Which Drugs Used: Other ...	-0.015944	0.015944
28	marrital_status_Current Marital Status: Never ...	0.013251	0.013251
9	has_anxiety	-0.012296	0.012296
17	household_income_Annual Income: 150k 200k	0.009010	0.009010
29	marrital_status_Current Marital Status: Separated	0.001558	0.001558
30	marrital_status_Current Marital Status: Widowed	0.001293	0.001293
8	has_depression	0.000962	0.000962

In [142…

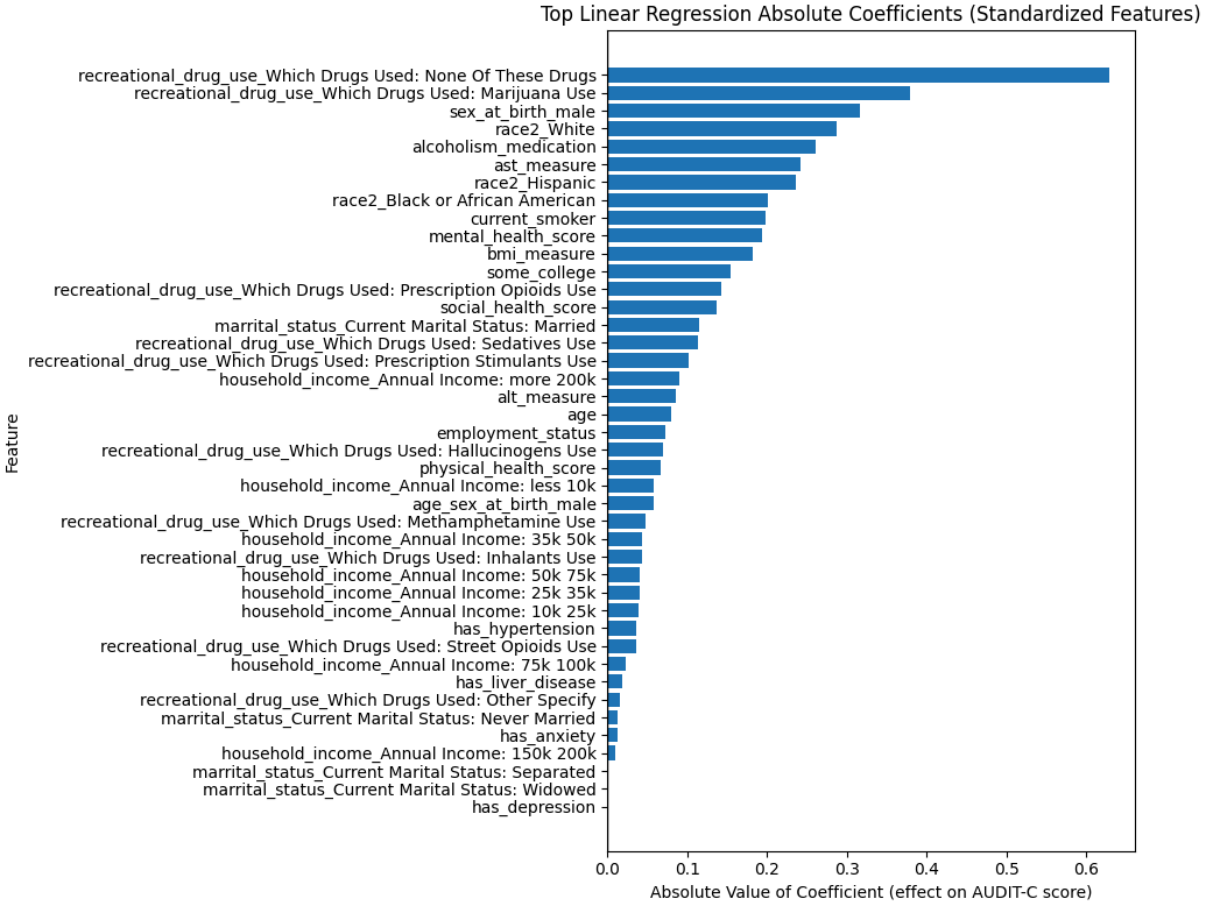
```
import matplotlib.pyplot as plt

# Take top N most important coefficients
top_n = 20
top_coef = coef_df.head(top_n).sort_values(by='coefficient')

plt.figure(figsize=(10, 8))

plt.barh(top_coef['feature'], top_coef['coefficient'])

plt.axvline(0, color='black', linewidth=1)
```

```
plt.title("Top Linear Regression Coefficients (Standardized Features)")
plt.xlabel("Coefficient (effect on AUDIT-C score)")
plt.ylabel("Feature")

plt.tight_layout()
plt.show()
```



In [143…

```
import matplotlib.pyplot as plt

# Take top N most important coefficients
top_n = 20
top_coef = coef_df.sort_values(by='abs_coef')

plt.figure(figsize=(10, 8))

plt.barh(top_coef['feature'], top_coef['abs_coef'])

plt.axvline(0, color='black', linewidth=1)

plt.title("Top Linear Regression Absolute Coefficients (Standardized Features)")
plt.xlabel("Absolute Value of Coefficient (effect on AUDIT-C score)")
plt.ylabel("Feature")

plt.tight_layout()
plt.show()
```



In [144…

```
#Trying Regularized Linear Model with best hyperparameters on the testing set
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

pipeline = make_pipeline(
    StandardScaler(),
    SGDRegressor(tol = 1e-5, eta0 = 1e-3, penalty = 'l1', alpha = 1e-4)
)
pipeline.fit(x_train, y_train)
y_pred = pipeline.predict(x_test)

# Calculate Regression Metrics
mae = mean_absolute_error(y_test, y_pred)
rmse = np.sqrt(mean_squared_error(y_test, y_pred))
r2 = r2_score(y_test, y_pred)

print("\033[1m\n--- Final Regularized Regression Model Performance on Test Set ---\
print(f"Testing Root Mean Squared Error (RMSE): {rmse:.4f}")
print(f"Testing Mean Absolute Error (MAE): {mae:.4f}")
print(f"Testing R-squared: {r2:.4f}")
```

--- Final Regularized Regression Model Performance on Test Set ---
Testing Root Mean Squared Error (RMSE): 2.0543
Testing Mean Absolute Error (MAE): 1.5672
Testing R-squared: 0.1404

In [145…

```
#Coefficients for Reugliarzed Linear Model:
# Because of scaling:
```

```
# Coefficients represent change in AUDIT-C per 1 standard deviation increase in the
# NOT per raw unit

# Get the linear regression model from the pipeline
lr = pipeline.named_steps['sgdregressor']

# Coefficients
coefs = lr.coef_

# Intercept
intercept = lr.intercept_

coef_df = pd.DataFrame({
    'feature': x_train.columns,
    'coefficient': coefs
})

# Sort by importance (absolute value)
coef_df['abs_coef'] = coef_df['coefficient'].abs()
coef_df = coef_df.sort_values(by='abs_coef', ascending=False)

print(coef_df)
```

	feature	coefficient	abs_coef
35	recreational_drug_use_Which Drugs Used: None O...	-0.621533	0.621533
33	recreational_drug_use_Which Drugs Used: Mariju...	-0.372719	0.372719
12	sex_at_birth_male	0.288849	0.288849
15	race2_White	0.273609	0.273609
10	alcoholism_medication	0.259998	0.259998
1	ast_measure	0.238013	0.238013
14	race2_Hispanic	0.223455	0.223455
5	current_smoker	0.198602	0.198602
25	mental_health_score	-0.191807	0.191807
13	race2_Black or African American	0.189898	0.189898
0	bmi_measure	-0.181375	0.181375
3	some_college	-0.151148	0.151148
37	recreational_drug_use_Which Drugs Used: Prescr...	-0.142243	0.142243
26	social_health_score	0.136719	0.136719
27	marrital_status_Current Marital Status: Married	-0.114619	0.114619
39	recreational_drug_use_Which Drugs Used: Sedati...	-0.109971	0.109971
38	recreational_drug_use_Which Drugs Used: Prescr...	-0.098170	0.098170
23	household_income_Annual Income: more 200k	0.087580	0.087580
11	age	-0.085684	0.085684
2	alt_measure	-0.084524	0.084524
41	age_sex_at_birth_male	0.082616	0.082616
4	employment_status	0.074276	0.074276
31	recreational_drug_use_Which Drugs Used: Halluc...	-0.070055	0.070055
24	physical_health_score	0.067816	0.067816
22	household_income_Annual Income: less 10k	0.055524	0.055524
34	recreational_drug_use_Which Drugs Used: Metham...	-0.047332	0.047332
19	household_income_Annual Income: 35k 50k	-0.041441	0.041441
32	recreational_drug_use_Which Drugs Used: Inhala...	-0.041348	0.041348
20	household_income_Annual Income: 50k 75k	-0.040819	0.040819
16	household_income_Annual Income: 10k 25k	-0.038003	0.038003
40	recreational_drug_use_Which Drugs Used: Street...	-0.035950	0.035950
18	household_income_Annual Income: 25k 35k	-0.035245	0.035245
6	has_hypertension	0.034306	0.034306
21	household_income_Annual Income: 75k 100k	-0.021120	0.021120
7	has_liver_disease	0.018395	0.018395
36	recreational_drug_use_Which Drugs Used: Other ...	-0.013930	0.013930
28	marrital_status_Current Marital Status: Never ...	0.013314	0.013314
9	has_anxiety	-0.012265	0.012265
17	household_income_Annual Income: 150k 200k	0.008498	0.008498
29	marrital_status_Current Marital Status: Separated	0.000000	0.000000
30	marrital_status_Current Marital Status: Widowed	0.000000	0.000000
8	has_depression	0.000000	0.000000

Polynomial Regression

In [146…

```
from sklearn.model_selection import cross_validate
from sklearn.preprocessing import StandardScaler, PolynomialFeatures
from sklearn.pipeline import make_pipeline
import warnings
import random
import numpy as np

def cross_validation_PLR(model, degree=1):
    warnings.filterwarnings('ignore')
```

```
    random.seed(42)
    np.random.seed(42)

    pipeline = make_pipeline(
        PolynomialFeatures(degree=degree, include_bias=False),
        StandardScaler(),
        model
    )

    # Changed scoring to 'neg_root_mean_squared_error'
    results = cross_validate(
        pipeline,
        x_train,
        y_train,
        cv=5,
        scoring='neg_root_mean_squared_error'
    )

    # Scikit-learn returns negative RMSE, so we negate it
    rmse_scores = -results['test_score']

    print(f"Degree: {degree}")
    print(f"Validation Mean RMSE: {rmse_scores.mean():.4f}")

    return rmse_scores.mean() # Returning the mean RMSE as requested
```

In [147…

```
from sklearn.linear_model import LinearRegression

  # regular linear
plr_model1 = cross_validation_PLR(LinearRegression(), degree=1)
  # quadratic
plr_model2 = cross_validation_PLR(LinearRegression(), degree=2)
```

```
Degree: 1
Validation Mean RMSE: 2.0325
Degree: 2
Validation Mean RMSE: 2.1377
```

In [148…

```
  # regular linear
plr_model1 = cross_validation_PLR(SGDRegressor(tol = 1e-5, eta0 = 1e-3, penalty = '
  # quadratic
plr_model2 = cross_validation_PLR(SGDRegressor(tol = 1e-5, eta0 = 1e-3, penalty = '
```

```
Degree: 1
Validation Mean RMSE: 2.0326
Degree: 2
Validation Mean RMSE: 2.3442
```

In [149…

```
#Trying Polynomial Linear Model (degree > 1) with best hyperparameters on the testi
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

pipeline = make_pipeline(
    PolynomialFeatures(degree=2, include_bias=False),
    StandardScaler(),
    LinearRegression()
)
pipeline.fit(x_train, y_train)
```

```
y_pred = pipeline.predict(x_test)

# Calculate Regression Metrics
mae = mean_absolute_error(y_test, y_pred)
rmse = np.sqrt(mean_squared_error(y_test, y_pred))
r2 = r2_score(y_test, y_pred)

print("\033[1m\n--- Final Polynomial Regression Model Performance on Test Set ---\0
print(f"Testing Root Mean Squared Error (RMSE): {rmse:.4f}")
print(f"Testing Mean Absolute Error (MAE): {mae:.4f}")
print(f"Testing R-squared: {r2:.4f}")
```

```
--- Final Polynomial Regression Model Performance on Test Set ---
Testing Root Mean Squared Error (RMSE): 2.1015
Testing Mean Absolute Error (MAE): 1.5766
Testing R-squared: 0.1005
```

In [150…

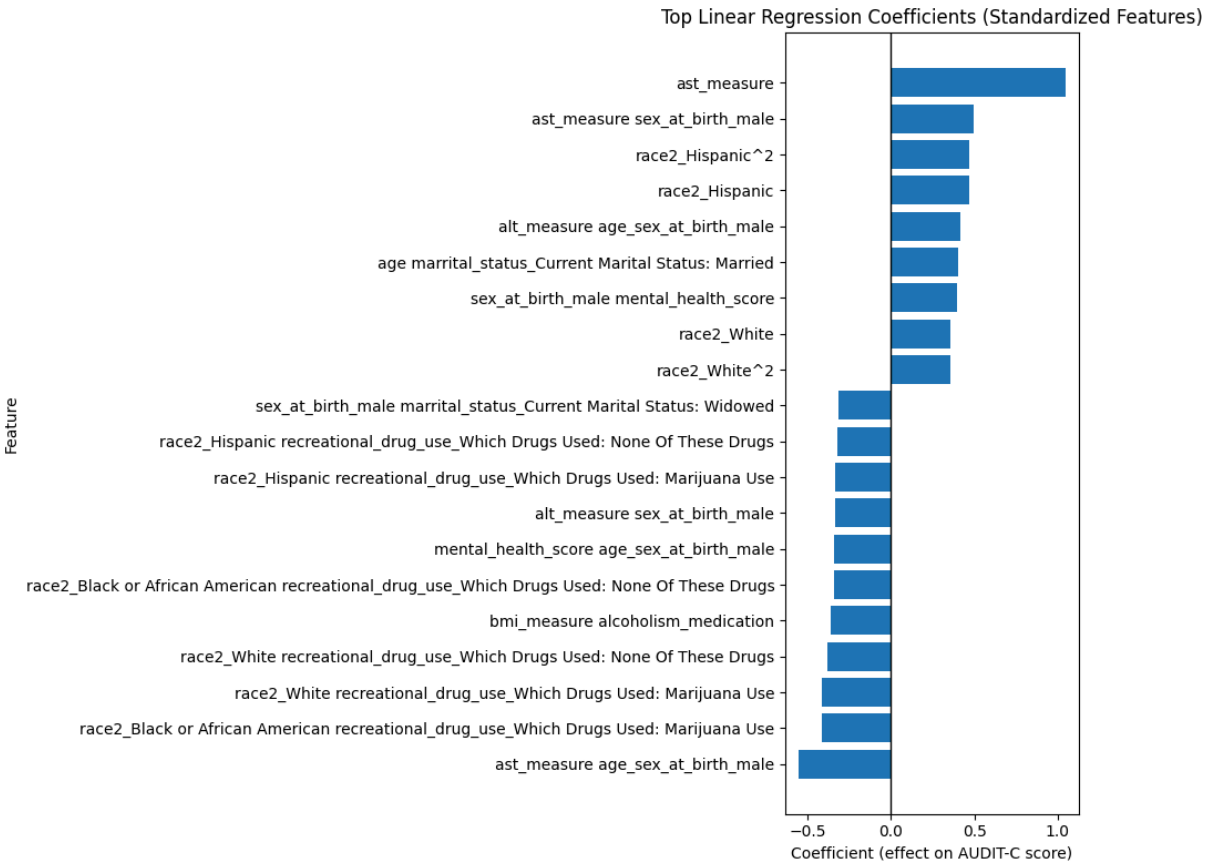
```
#Coefficients for Polynomial Regression

# Get steps from pipeline
poly = pipeline.named_steps['polynomialfeatures']
lr = pipeline.named_steps['linearregression']

# Get expanded feature names (includes interactions & squares)
feature_names = poly.get_feature_names_out(x_train.columns)

# Coefficients and intercept
coefs = lr.coef_
intercept = lr.intercept_

# Build dataframe
coef_df = pd.DataFrame({
    'feature': feature_names,
    'coefficient': coefs
})

# Sort by importance (absolute value)
coef_df['abs_coef'] = coef_df['coefficient'].abs()
coef_df = coef_df.sort_values(by='abs_coef', ascending=False)

print(coef_df.head(50))
```

	feature	coefficient	abs_coef
1	ast_measure	1.047235	1.047235
124	ast_measure age_sex_at_birth_male	-0.554549	0.554549
95	ast_measure sex_at_birth_male	0.495903	0.495903
539	race2_Hispanic^2	0.467051	0.467051
14	race2_Hispanic	0.467051	0.467051
164	alt_measure age_sex_at_birth_male	0.415891	0.415891
530	race2_Black or African American recreational_d...	-0.415201	0.415201
585	race2_White recreational_drug_use_Which Drugs ...	-0.412341	0.412341
465	age marrital_status_Current Marital Status: Ma...	0.399384	0.399384
493	sex_at_birth_male mental_health_score	0.394169	0.394169
587	race2_White recreational_drug_use_Which Drugs ...	-0.380448	0.380448
52	bmi_measure alcoholism_medication	-0.358910	0.358910
15	race2_White	0.357319	0.357319
567	race2_White^2	0.357319	0.357319
532	race2_Black or African American recreational_d...	-0.341354	0.341354
808	mental_health_score age_sex_at_birth_male	-0.339292	0.339292
135	alt_measure sex_at_birth_male	-0.333921	0.333921
558	race2_Hispanic recreational_drug_use_Which Dru...	-0.333411	0.333411
560	race2_Hispanic recreational_drug_use_Which Dru...	-0.324453	0.324453
498	sex_at_birth_male marrital_status_Current Mari...	-0.316604	0.316604
94	ast_measure age	0.312315	0.312315
473	age recreational_drug_use_Which Drugs Used: No...	0.297717	0.297717
108	ast_measure mental_health_score	-0.294319	0.294319
84	ast_measure^2	-0.286392	0.286392
878	marrital_status_Current Marital Status: Widowe...	0.286283	0.286283
64	bmi_measure household_income_Annual Income: le...	-0.286193	0.286193
452	age race2_Hispanic	-0.285169	0.285169
577	race2_White mental_health_score	-0.269868	0.269868
550	race2_Hispanic mental_health_score	-0.265644	0.265644
619	household_income_Annual Income: 10k 25k age_se...	0.260512	0.260512
204	employment_status^2	0.248406	0.248406
4	employment_status	0.248406	0.248406
592	race2_White recreational_drug_use_Which Drugs ...	-0.247794	0.247794
484	sex_at_birth_male household_income_Annual Inco...	-0.243515	0.243515
35	recreational_drug_use_Which Drugs Used: None O...	-0.243485	0.243485
917	recreational_drug_use_Which Drugs Used: None O...	-0.243485	0.243485
522	race2_Black or African American mental_health_...	-0.240414	0.240414
278	current_smoker age_sex_at_birth_male	-0.240190	0.240190
449	age^2	-0.239927	0.239927
249	current_smoker sex_at_birth_male	0.236399	0.236399
58	bmi_measure household_income_Annual Income: 10...	-0.225016	0.225016
73	bmi_measure recreational_drug_use_Which Drugs ...	0.223660	0.223660
357	has_depression race2_White	-0.222548	0.222548
418	alcoholism_medication age	0.210754	0.210754
417	alcoholism_medication^2	0.210568	0.210568
10	alcoholism_medication	0.210568	0.210568
72	bmi_measure marrital_status_Current Marital St...	0.210130	0.210130
510	race2_Black or African American^2	0.208848	0.208848
13	race2_Black or African American	0.208848	0.208848
53	bmi_measure age	-0.202842	0.202842

In [151…

```
import matplotlib.pyplot as plt

# Take top N most important coefficients
top_n = 20
```

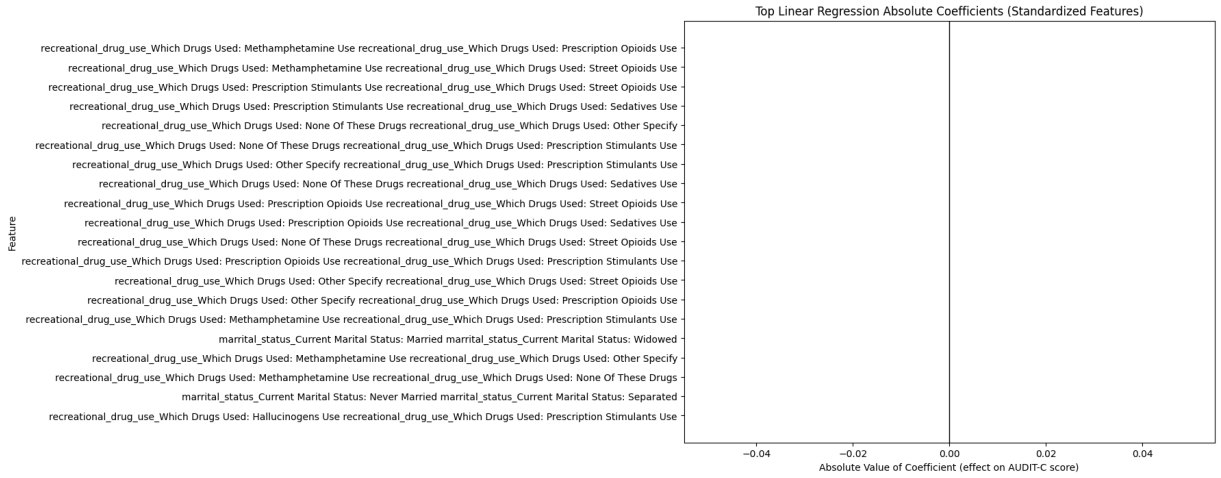
```
top_coef = coef_df.head(top_n).sort_values(by='coefficient')

plt.figure(figsize=(10, 8))

plt.barh(top_coef['feature'], top_coef['coefficient'])

plt.axvline(0, color='black', linewidth=1)

plt.title("Top Linear Regression Coefficients (Standardized Features)")
plt.xlabel("Coefficient (effect on AUDIT-C score)")
plt.ylabel("Feature")

plt.tight_layout()
plt.show()
```



In [152…

```
import matplotlib.pyplot as plt

# Take top N most important coefficients
top_n = 20
top_coef = coef_df.sort_values(by='abs_coef').head(top_n)

plt.figure(figsize=(10, 8))

plt.barh(top_coef['feature'], top_coef['abs_coef'])

plt.axvline(0, color='black', linewidth=1)

plt.title("Top Linear Regression Absolute Coefficients (Standardized Features)")
plt.xlabel("Absolute Value of Coefficient (effect on AUDIT-C score)")
```

```
plt.ylabel("Feature")

plt.tight_layout()
plt.show()
```



Random Forest Models

In [153…

```
from sklearn.ensemble import RandomForestRegressor

#Default Model: n_estimators = 100, max_depth = None, min_samples_split = 2
rf_model = RandomForestRegressor(random_state=42)
rf_results = cross_validation_LR(rf_model)
```

```
Validatiom RMSE: 2.0606
```

In [154…

```
#Testing different max depths for Random Forest
for max_depth in [2, 3, 4, 6, 8, 10, 12]:
    print('Max Depth', max_depth)
    rf_model = RandomForestRegressor(n_estimators=100, random_state=42, max_depth =
    rf_results = cross_validation_LR(rf_model)
    print('\n')
```

```
Max Depth 2
Validatiom RMSE: 2.1063


Max Depth 3
Validatiom RMSE: 2.0783


Max Depth 4
Validatiom RMSE: 2.0619


Max Depth 6
Validatiom RMSE: 2.0416


Max Depth 8
Validatiom RMSE: 2.0328


Max Depth 10
Validatiom RMSE: 2.0310


Max Depth 12
Validatiom RMSE: 2.0332
```

In [155…

```python
max_depth = 10

#Testing different min_samples_split
for min_samples_split in [2, 5, 10, 20, 50, 100, 200]:
    print('Min Samples to Split', min_samples_split)
    rf_model = RandomForestRegressor(n_estimators=100, random_state=42, max_depth =
                                     min_samples_split = min_samples_split)
    rf_results = cross_validation_LR(rf_model)
    print('\n')
```

```
Min Samples to Split 2
Validatiom RMSE: 2.0310


Min Samples to Split 5
Validatiom RMSE: 2.0305


Min Samples to Split 10
Validatiom RMSE: 2.0303


Min Samples to Split 20
Validatiom RMSE: 2.0305


Min Samples to Split 50
Validatiom RMSE: 2.0316


Min Samples to Split 100
Validatiom RMSE: 2.0343


Min Samples to Split 200
Validatiom RMSE: 2.0372
```

In [181…

```
max_depth = 10
min_samples_split = 10

#Testing different n_estimators
for n_estimators in [10, 50, 100, 200, 300]:
    print('Number of Estimators (Trees)', n_estimators)
    rf_model = RandomForestRegressor(n_estimators=n_estimators, random_state=42, ma
                                     min_samples_split = min_samples_split)
    rf_results = cross_validation_LR(rf_model)
    print('\n')
```

```
Number of Estimators (Trees) 10
Validatiom RMSE: 2.0501


Number of Estimators (Trees) 50
Validatiom RMSE: 2.0333


Number of Estimators (Trees) 100
Validatiom RMSE: 2.0303


Number of Estimators (Trees) 200
Validatiom RMSE: 2.0290


Number of Estimators (Trees) 300
Validatiom RMSE: 2.0291
```

In [182…

```python
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

pipeline = make_pipeline(
    StandardScaler(),
    RandomForestRegressor(n_estimators=200, random_state=42, max_depth = 10, min_sa
)
pipeline.fit(x_train, y_train)
y_pred = pipeline.predict(x_test)

# Calculate Regression Metrics
mae = mean_absolute_error(y_test, y_pred)
rmse = np.sqrt(mean_squared_error(y_test, y_pred))
r2 = r2_score(y_test, y_pred)

print("\033[1m\n--- Final Random Forest Model Performance on Test Set ---\033[0m")
print(f"Root Mean Squared Error (RMSE): {rmse:.4f}")
print(f"Mean Absolute Error (MAE): {mae:.4f}")
print(f"R-squared: {r2:.4f}")
```

```
--- Final Random Forest Model Performance on Test Set ---
Root Mean Squared Error (RMSE): 2.0629
Mean Absolute Error (MAE): 1.5724
R-squared: 0.1332
```

In [183…

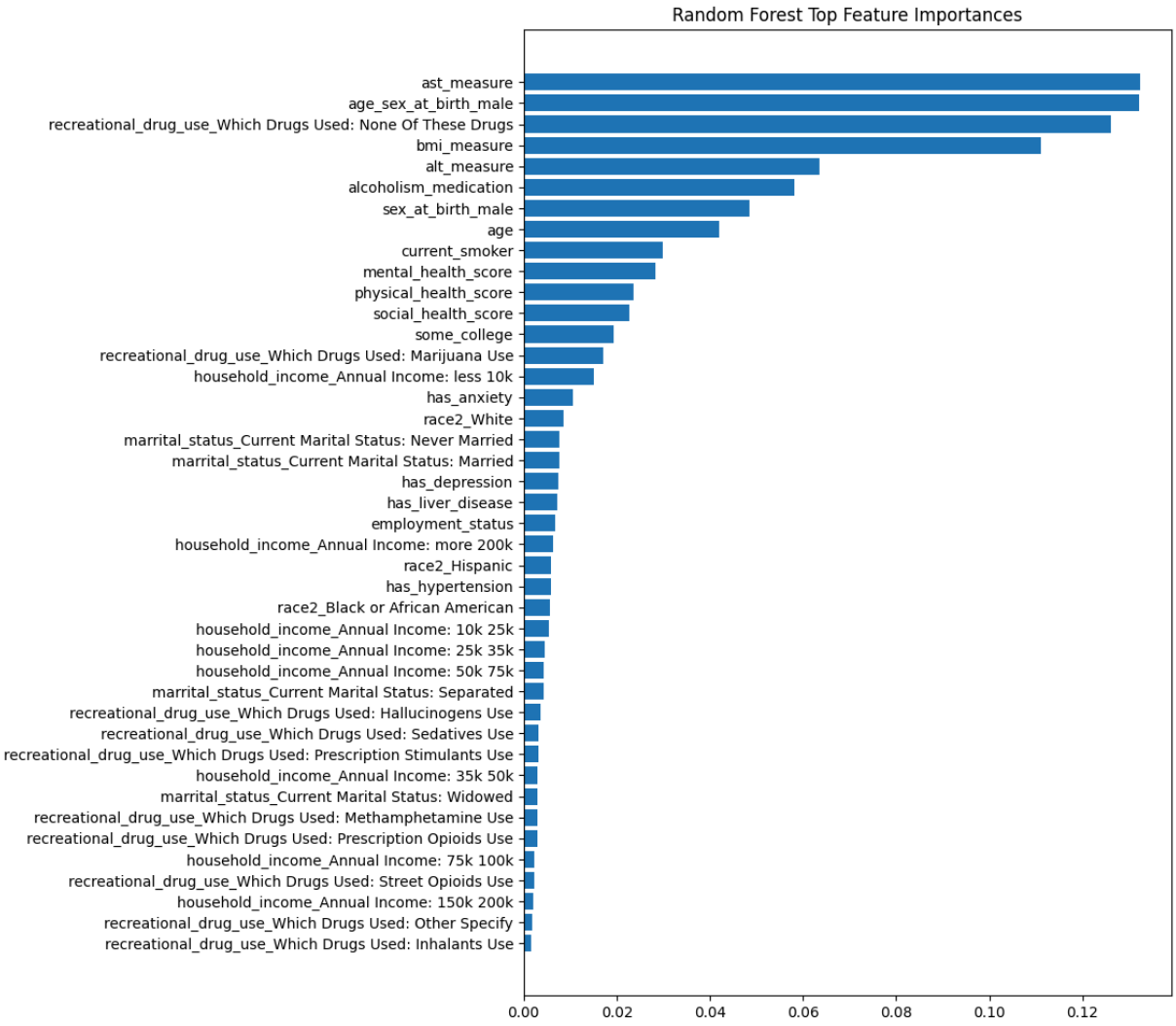
```python
rf = pipeline.named_steps['randomforestregressor']

importances = rf.feature_importances_

importance_df = pd.DataFrame({
    'feature': x_train.columns,
    'importance': importances
})

# Sort from most important to least
```

```
importance_df = importance_df.sort_values(by='importance', ascending=False)

print(importance_df.head(15))
```

```
                                              feature  importance
1                                        ast_measure    0.132467
41                             age_sex_at_birth_male    0.132171
35  recreational_drug_use_Which Drugs Used: None O...    0.126065
0                                        bmi_measure    0.111103
2                                        alt_measure    0.063512
10                             alcoholism_medication    0.058053
12                                 sex_at_birth_male    0.048484
11                                               age    0.042039
5                                    current_smoker    0.029792
25                             mental_health_score    0.028289
24                            physical_health_score    0.023689
26                              social_health_score    0.022741
3                                      some_college    0.019395
33  recreational_drug_use_Which Drugs Used: Mariju...    0.017136
22           household_income_Annual Income: less 10k    0.015049
```

In [184…

```
import matplotlib.pyplot as plt

top_features = importance_df

plt.figure(figsize=(8, 12))  # width, height in inches
plt.barh(top_features['feature'], top_features['importance'])
plt.gca().invert_yaxis()
plt.title("Random Forest Top Feature Importances")
plt.show()
```



XGBoost Models

In [160…

```
from xgboost import XGBRegressor
xgb_model = XGBRegressor(random_state=42, n_jobs=-1)
xgb_results = cross_validation_LR(xgb_model)
```

```
Validatiom RMSE: 2.0848
```

In [161…

```
#Testing different max depths for XGBoost
for max_depth in [2, 3, 4, 6, 8, 10, 12, 15]:
    print('Max Depth', max_depth)
    xgb_model = XGBRegressor(n_estimators=100, random_state=42, n_jobs=-1, max_dept
    xgb_results = cross_validation_LR(xgb_model)
    print('\n')
```

```
Max Depth 2
Validatiom RMSE: 2.0104


Max Depth 3
Validatiom RMSE: 2.0117


Max Depth 4
Validatiom RMSE: 2.0258


Max Depth 6
Validatiom RMSE: 2.0848


Max Depth 8
Validatiom RMSE: 2.1518


Max Depth 10
Validatiom RMSE: 2.1875


Max Depth 12
Validatiom RMSE: 2.1947


Max Depth 15
Validatiom RMSE: 2.1989
```

In [162…

```python
max_depth = 2

#Testing different Learning Rates for XGBoost
for learning_rate in [0.01, 0.05, 0.1, 0.2, 0.3, 0.4, 0.5]:
    print('Learning Rate', learning_rate)
    xgb_model = XGBRegressor(n_estimators=100, random_state=42, n_jobs=-1, max_dept
                             learning_rate = learning_rate)
    xgb_results = cross_validation_LR(xgb_model)
    print('\n')
```

```
Learning Rate 0.01
Validatiom RMSE: 2.1124


Learning Rate 0.05
Validatiom RMSE: 2.0387


Learning Rate 0.1
Validatiom RMSE: 2.0224


Learning Rate 0.2
Validatiom RMSE: 2.0135


Learning Rate 0.3
Validatiom RMSE: 2.0104


Learning Rate 0.4
Validatiom RMSE: 2.0111


Learning Rate 0.5
Validatiom RMSE: 2.0123
```

In [163…

```
max_depth = 2
learning_rate = 0.3

#Testing different Min Child Weights for XGBoost
for min_child_weight in [0, 1, 5, 10, 25, 50]:
    print('Min Child Weight', min_child_weight)
    xgb_model = XGBRegressor(n_estimators=100, random_state=42, n_jobs=-1, max_dept
                             learning_rate = learning_rate, min_child_weight = min_c
    xgb_results = cross_validation_LR(xgb_model)
    print('\n')
```

```
Min Child Weight 0
Validatiom RMSE: 2.0104


Min Child Weight 1
Validatiom RMSE: 2.0104


Min Child Weight 5
Validatiom RMSE: 2.0105


Min Child Weight 10
Validatiom RMSE: 2.0117


Min Child Weight 25
Validatiom RMSE: 2.0098


Min Child Weight 50
Validatiom RMSE: 2.0106
```

In [185…

```
max_depth = 2
learning_rate = 0.3
min_child_weight = 25

#Testing different Reg Alpha for XGBoost
for reg_alpha in [0, 0.01, 0.05, 0.1, 0.25, 0.5, 1]:
    print('Alpha Regularization (Lasso/L1)', reg_alpha)
    xgb_model = XGBRegressor(n_estimators=100, random_state=42, n_jobs=-1, max_dept
                             learning_rate = learning_rate, min_child_weight = min_c
    xgb_results = cross_validation_LR(xgb_model)
    print('\n')
```

```
Alpha Regularization (Lasso/L1) 0
Validatiom RMSE: 2.0098


Alpha Regularization (Lasso/L1) 0.01
Validatiom RMSE: 2.0098


Alpha Regularization (Lasso/L1) 0.05
Validatiom RMSE: 2.0099


Alpha Regularization (Lasso/L1) 0.1
Validatiom RMSE: 2.0089


Alpha Regularization (Lasso/L1) 0.25
Validatiom RMSE: 2.0094


Alpha Regularization (Lasso/L1) 0.5
Validatiom RMSE: 2.0116


Alpha Regularization (Lasso/L1) 1
Validatiom RMSE: 2.0118
```

In [186…

```
max_depth = 2
learning_rate = 0.3
min_child_weight = 25
reg_alpha = 0.1

#Testing different Reg Beta for XGBoost
for reg_beta in [0, 0.01, 0.05, 0.1, 0.25, 0.5, 1]:
    print('Beta Regularization (Ridge/L2)', reg_beta)
    xgb_model = XGBRegressor(n_estimators=100, random_state=42, n_jobs=-1, max_dept
                             learning_rate = learning_rate, min_child_weight = min_c
                             reg_beta = reg_beta)
    xgb_results = cross_validation_LR(xgb_model)
    print('\n')
```

```
Beta Regularization (Ridge/L2) 0
Validatiom RMSE: 2.0089


Beta Regularization (Ridge/L2) 0.01
Validatiom RMSE: 2.0089


Beta Regularization (Ridge/L2) 0.05
Validatiom RMSE: 2.0089


Beta Regularization (Ridge/L2) 0.1
Validatiom RMSE: 2.0089


Beta Regularization (Ridge/L2) 0.25
Validatiom RMSE: 2.0089


Beta Regularization (Ridge/L2) 0.5
Validatiom RMSE: 2.0089


Beta Regularization (Ridge/L2) 1
Validatiom RMSE: 2.0089
```

In [187…

```
max_depth = 2
learning_rate = 0.3
min_child_weight = 25
reg_alpha = 0.1
reg_beta = 0

#Testing different n_estimators for XGBoost
for n_estimators in [10, 50, 100, 200, 300, 500]:
    print('n_estimators', n_estimators)
    xgb_model = XGBRegressor(n_estimators=n_estimators, random_state=42, n_jobs=-1,
                             learning_rate = learning_rate, min_child_weight = min_c
                             reg_beta = reg_beta)
    xgb_results = cross_validation_LR(xgb_model)
    print('\n')
```

```
n_estimators 10
Validatiom RMSE: 2.0550


n_estimators 50
Validatiom RMSE: 2.0156


n_estimators 100
Validatiom RMSE: 2.0089


n_estimators 200
Validatiom RMSE: 2.0075


n_estimators 300
Validatiom RMSE: 2.0068


n_estimators 500
Validatiom RMSE: 2.0103
```

In [188…

```python
from xgboost import XGBRegressor
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

# XGBoost is very powerful but can overfit easily,
# so we set some standard hyperparameters.
xgb_pipeline = make_pipeline(
    StandardScaler(),
    XGBRegressor(
        n_estimators=300,
        max_depth=2,
        learning_rate=0.3,
        min_child_weight=25,
        reg_alpha=0,
        reg_lambda=0.1,   # also fix typo: reg_beta → reg_lambda
        random_state=42,
        n_jobs=1,       # key change
        tree_method='exact'  # more deterministic (but slower)
    )
)

# 2. Fit the model
xgb_pipeline.fit(x_train, y_train)

# 3. Predict
y_pred_xgb = xgb_pipeline.predict(x_test)

# 4. Evaluate
mae_xgb = mean_absolute_error(y_test, y_pred_xgb)
rmse_xgb = np.sqrt(mean_squared_error(y_test, y_pred_xgb))
r2_xgb = r2_score(y_test, y_pred_xgb)
```

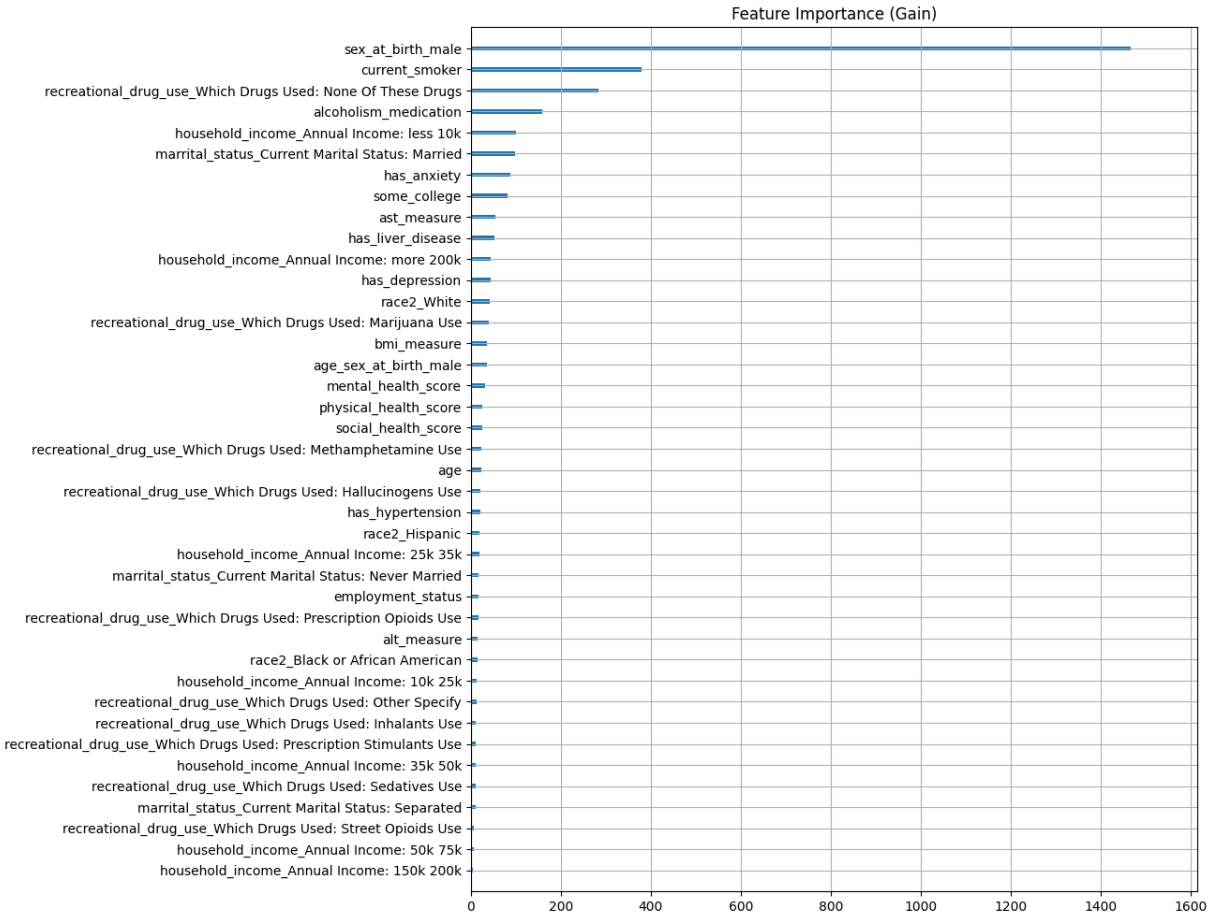
```
print("\033[1m\n--- Final XGBoost Performance on Test Set ---\033[0m")
print(f"Root Mean Squared Error (RMSE): {rmse_xgb:.4f}")
print(f"Mean Absolute Error (MAE): {mae_xgb:.4f}")
print(f"R-squared: {r2_xgb:.4f}")
```

```
--- Final XGBoost Performance on Test Set ---
Root Mean Squared Error (RMSE): 2.0431
Mean Absolute Error (MAE): 1.5541
R-squared: 0.1498
```

Feature Importance

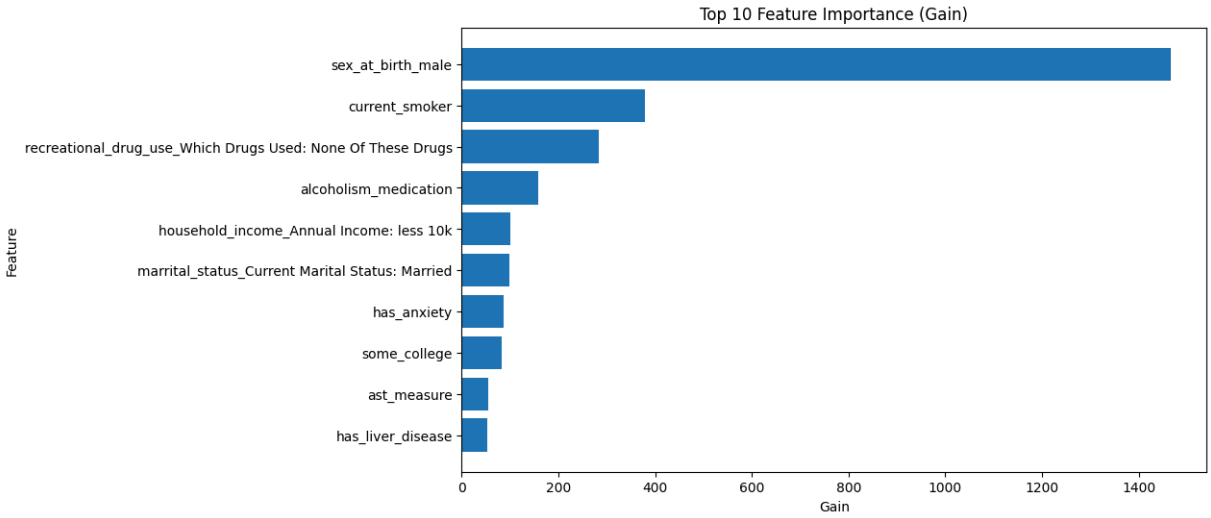
In [189…

```
from xgboost import plot_importance

xgb_model = xgb_pipeline.named_steps['xgbregressor']
xgb_model.get_booster().feature_names = list(x_train.columns)

fig, ax = plt.subplots(figsize=(10, 12))

plot_importance(
    xgb_model,
    importance_type='gain',
    ax = ax,
    show_values=False
)

ax.set_title("Feature Importance (Gain)")
ax.set_xlabel("")   # remove 'F score' / gain label
ax.set_ylabel("")   # optional: remove feature axis label


plt.show()
```



In [190…

```python
import pandas as pd
import matplotlib.pyplot as plt

xgb_model = xgb_pipeline.named_steps['xgbregressor']
xgb_model.get_booster().feature_names = list(x_train.columns)

# Get importance dictionary from booster
importance_dict = xgb_model.get_booster().get_score(importance_type='gain')

# Convert to DataFrame
importance_df = pd.DataFrame({
    'feature': list(importance_dict.keys()),
    'gain': list(importance_dict.values())
})

# Sort and take top 10
top10 = importance_df.sort_values(by='gain', ascending=False).head(10)

# Plot
fig, ax = plt.subplots(figsize=(10, 6))

ax.barh(top10['feature'][::-1], top10['gain'][::-1])
ax.set_title("Top 10 Feature Importance (Gain)")
ax.set_xlabel("Gain")
ax.set_ylabel("Feature")

plt.show()
```



In [191…

```python
import pandas as pd

# Get booster
booster = xgb_model.get_booster()

# Extract gain importance
gain_dict = booster.get_score(importance_type='gain')

# Convert to DataFrame
gain_df = pd.DataFrame(
    list(gain_dict.items()),
    columns=['feature', 'gain']
)

# Sort from most to least important
gain_df = gain_df.sort_values(by='gain', ascending=False)

# Display
print(gain_df)
```

	feature	gain
12	sex_at_birth_male	1467.120850
5	current_smoker	379.010468
33	recreational_drug_use_Which Drugs Used: None O...	283.822174
10	alcoholism_medication	158.224640
21	household_income_Annual Income: less 10k	99.795059
26	marrital_status_Current Marital Status: Married	98.473763
9	has_anxiety	87.232910
3	some_college	82.124146
1	ast_measure	53.905605
7	has_liver_disease	52.653080
22	household_income_Annual Income: more 200k	44.646675
8	has_depression	43.967480
15	race2_White	42.824036
31	recreational_drug_use_Which Drugs Used: Mariju...	40.897816
0	bmi_measure	35.577747
39	age_sex_at_birth_male	35.390163
24	mental_health_score	31.227835
23	physical_health_score	24.790997
25	social_health_score	24.291437
32	recreational_drug_use_Which Drugs Used: Metham...	23.304525
11	age	23.295845
29	recreational_drug_use_Which Drugs Used: Halluc...	22.085096
6	has_hypertension	21.123240
14	race2_Hispanic	19.450113
18	household_income_Annual Income: 25k 35k	18.678846
27	marrital_status_Current Marital Status: Never ...	17.546076
4	employment_status	17.150826
35	recreational_drug_use_Which Drugs Used: Prescr...	16.022900
2	alt_measure	14.866580
13	race2_Black or African American	14.548339
16	household_income_Annual Income: 10k 25k	12.578069
34	recreational_drug_use_Which Drugs Used: Other ...	12.333837
30	recreational_drug_use_Which Drugs Used: Inhala...	11.660208
36	recreational_drug_use_Which Drugs Used: Prescr...	11.179186
19	household_income_Annual Income: 35k 50k	10.382159
37	recreational_drug_use_Which Drugs Used: Sedati...	10.021909
28	marrital_status_Current Marital Status: Separated	9.967813
38	recreational_drug_use_Which Drugs Used: Street...	6.761568
20	household_income_Annual Income: 50k 75k	6.002491
17	household_income_Annual Income: 150k 200k	5.430470

In [192…
```
# !pip install shap
```

In [193…
```
import matplotlib.pyplot as plt

plt.rcParams["figure.figsize"] = (12, 8)
plt.rcParams["figure.dpi"] = 100
```

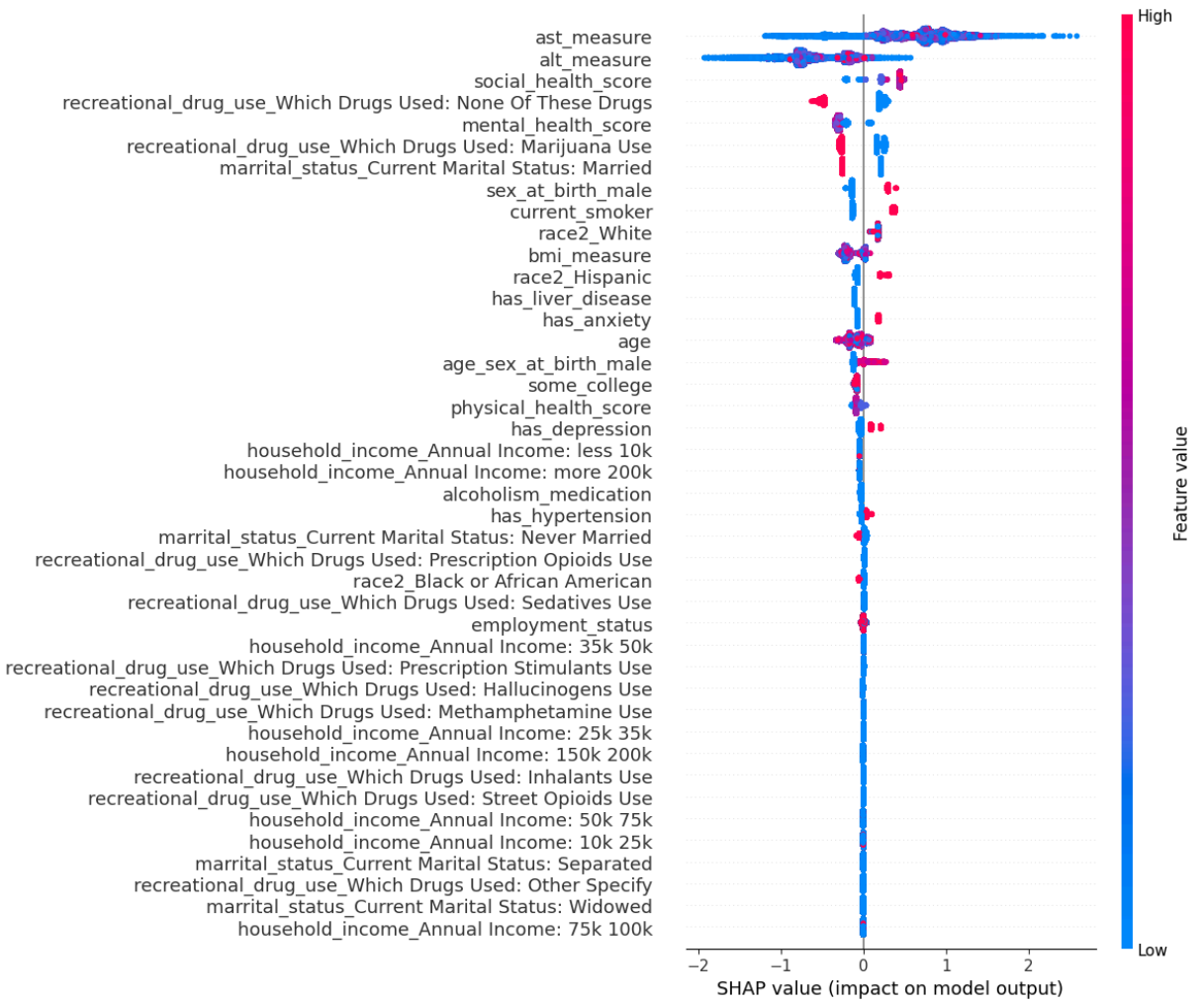
In [194…
```
!pip install shap
```

```
Requirement already satisfied: shap in /home/jupyter/.local/lib/python3.10/site-pack
ages (0.49.1)
Requirement already satisfied: numpy in /opt/conda/lib/python3.10/site-packages (fro
m shap) (1.24.4)
Requirement already satisfied: scipy in /opt/conda/lib/python3.10/site-packages (fro
m shap) (1.11.4)
Requirement already satisfied: scikit-learn in /opt/conda/lib/python3.10/site-packag
es (from shap) (1.6.0)
Requirement already satisfied: pandas in /opt/conda/lib/python3.10/site-packages (fr
om shap) (2.0.3)
Requirement already satisfied: tqdm>=4.27.0 in /opt/conda/lib/python3.10/site-packag
es (from shap) (4.67.1)
Requirement already satisfied: packaging>20.9 in /opt/conda/lib/python3.10/site-pack
ages (from shap) (21.3)
Requirement already satisfied: slicer==0.0.8 in /home/jupyter/.local/lib/python3.10/
site-packages (from shap) (0.0.8)
Requirement already satisfied: numba>=0.54 in /opt/conda/lib/python3.10/site-package
s (from shap) (0.58.1)
Requirement already satisfied: cloudpickle in /opt/conda/lib/python3.10/site-package
s (from shap) (2.2.1)
Requirement already satisfied: typing-extensions in /opt/conda/lib/python3.10/site-p
ackages (from shap) (4.12.2)
Requirement already satisfied: llvmlite<0.42,>=0.41.0dev0 in /opt/conda/lib/python3.
10/site-packages (from numba>=0.54->shap) (0.41.1)
Requirement already satisfied: pyparsing!=3.0.5,>=2.0.2 in /opt/conda/lib/python3.1
0/site-packages (from packaging>20.9->shap) (3.2.0)
Requirement already satisfied: python-dateutil>=2.8.2 in /opt/conda/lib/python3.10/s
ite-packages (from pandas->shap) (2.8.2)
Requirement already satisfied: pytz>=2020.1 in /opt/conda/lib/python3.10/site-packag
es (from pandas->shap) (2023.3)
Requirement already satisfied: tzdata>=2022.1 in /opt/conda/lib/python3.10/site-pack
ages (from pandas->shap) (2024.2)
Requirement already satisfied: six>=1.5 in /opt/conda/lib/python3.10/site-packages
(from python-dateutil>=2.8.2->pandas->shap) (1.17.0)
Requirement already satisfied: joblib>=1.2.0 in /opt/conda/lib/python3.10/site-packa
ges (from scikit-learn->shap) (1.4.2)
Requirement already satisfied: threadpoolctl>=3.1.0 in /opt/conda/lib/python3.10/sit
e-packages (from scikit-learn->shap) (3.5.0)

[notice] A new release of pip is available: 26.0.1 -> 26.1
[notice] To update, run: pip install --upgrade pip
```

In [195…

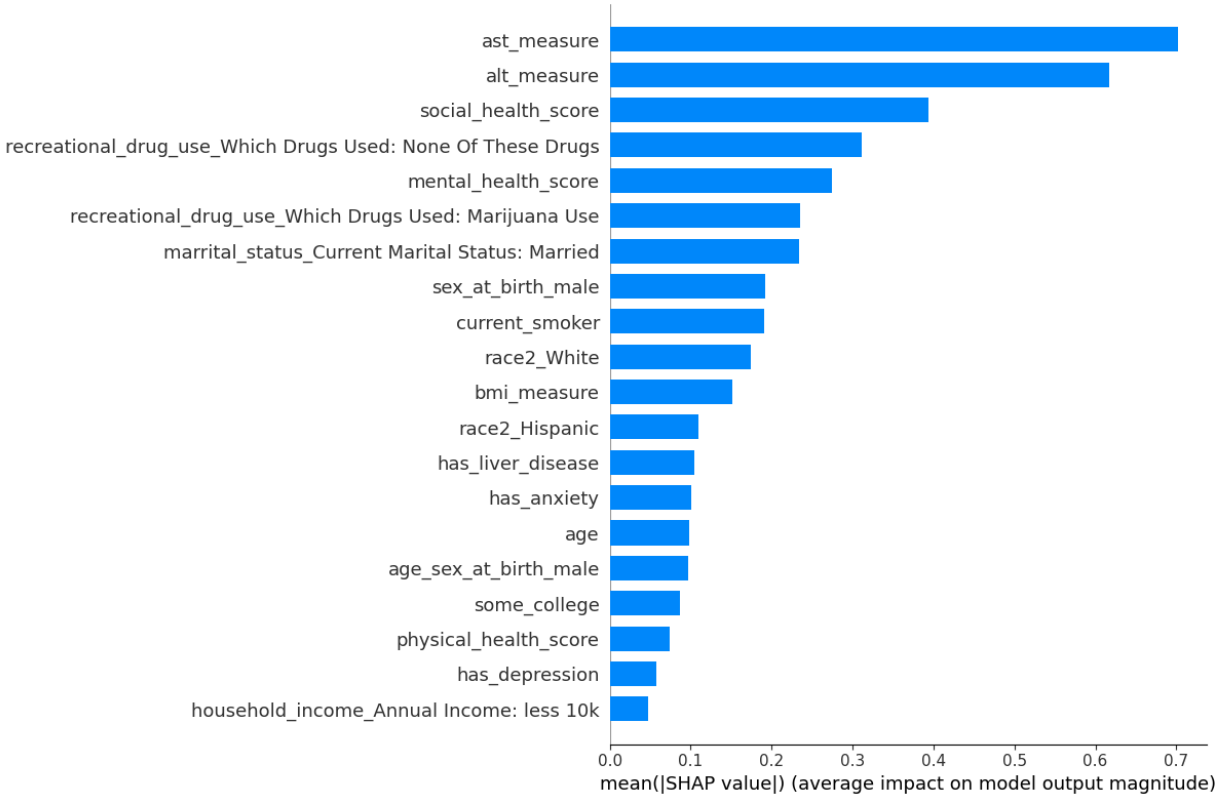
```python
import shap
import matplotlib.pyplot as plt

xgb_model = xgb_pipeline.named_steps['xgbregressor']

explainer = shap.TreeExplainer(xgb_model)
shap_values = explainer.shap_values(x_train)

plt.figure(figsize=(12, 10))

shap.summary_plot(
    shap_values,
    x_train,
    max_display=50,   # <-- show top 50 features
```

```
    plot_size=(12, 10)
)
```



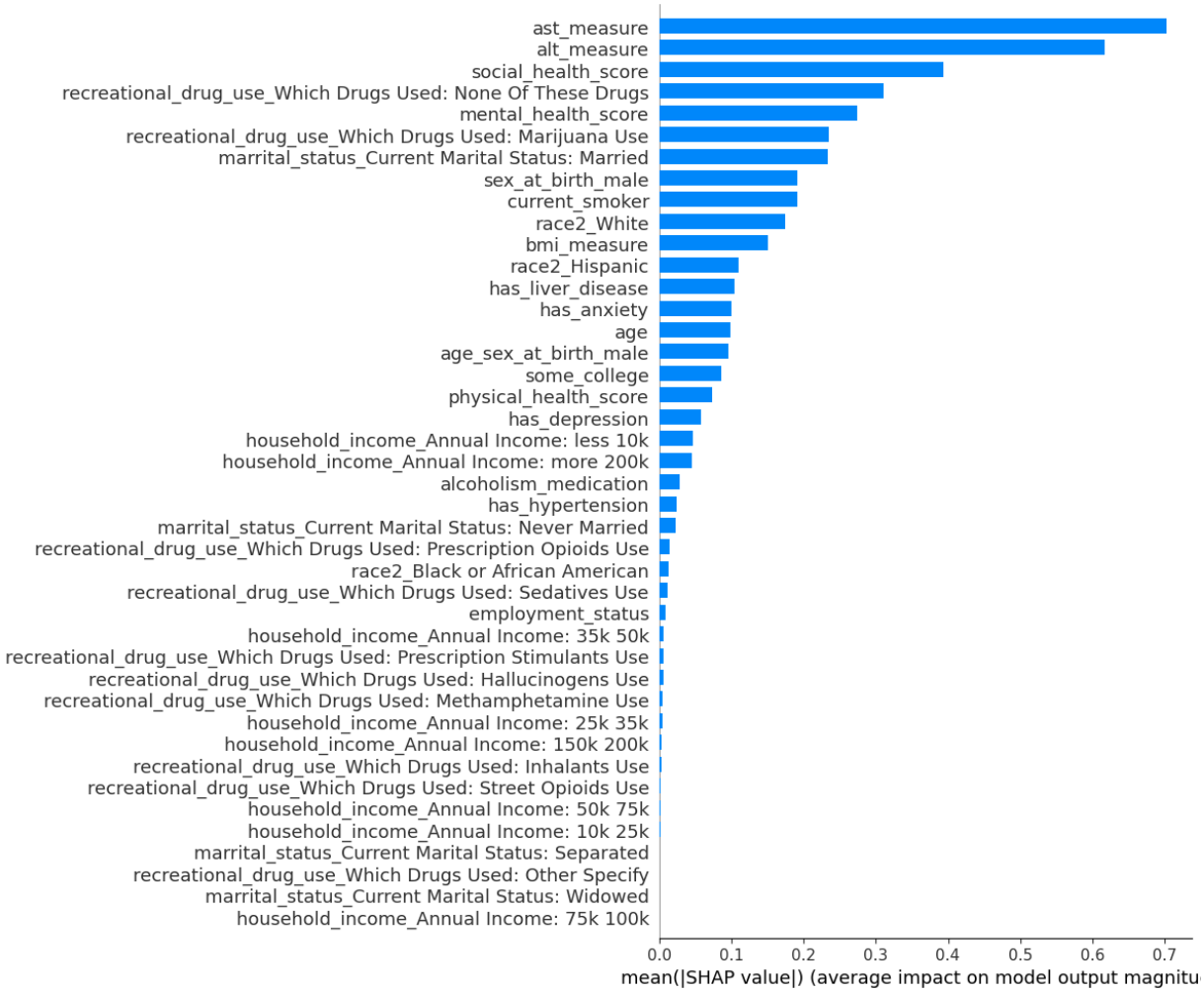
In [196…

```
shap.summary_plot(shap_values, x_train, plot_type="bar", plot_size=(12, 8))
```



In [197…

```
shap.summary_plot(
    shap_values,
    x_train,
    max_display=50,   # <-- show top 50 features,
    plot_type = 'bar',
    plot_size=(12, 10)
)
```



In [ ]:

In [ ]:

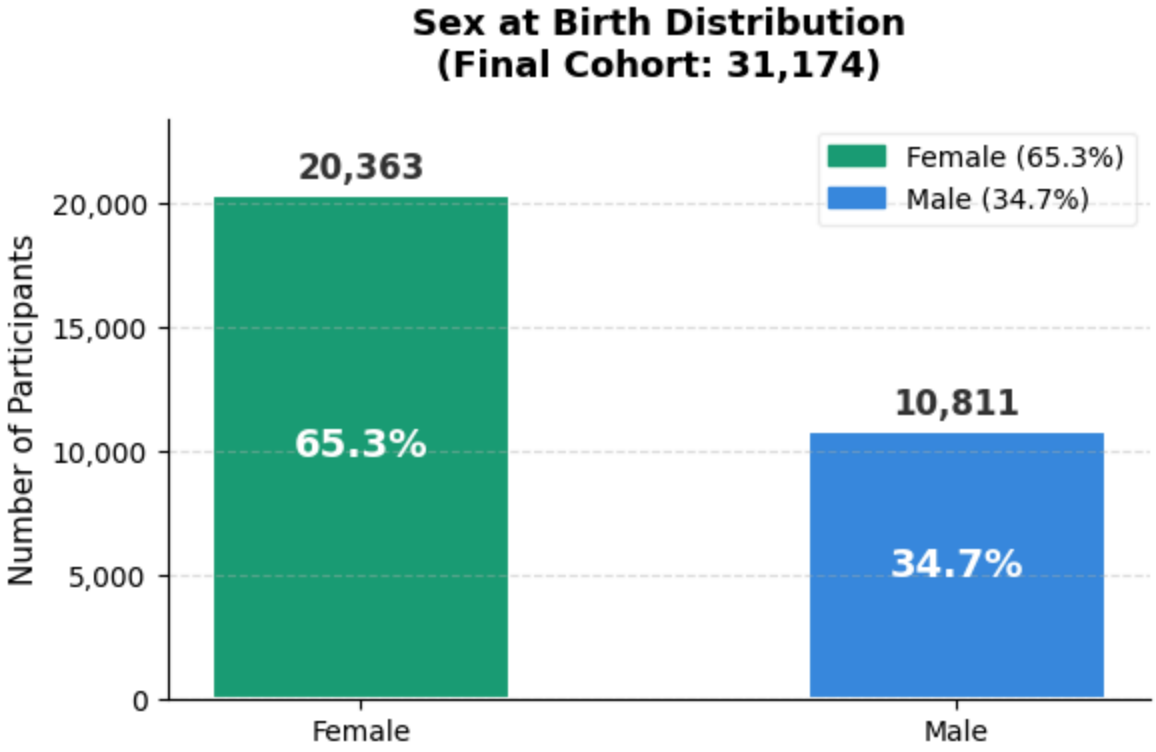
In [177…

```
## Ramcharan Draft Analysis
```

In [48]:

```
import matplotlib.pyplot as plt
import matplotlib.patches as mpatches
import numpy as np

# Data
labels = ['Female', 'Male']
counts = [20363, 10811]
pcts = [65.3, 34.7]
colors = ['#1D9E75', '#378ADD']

fig, ax = plt.subplots(figsize=(6, 4))

bars = ax.bar(labels, counts, color=colors, width=0.5, edgecolor='white', linewidth

# Add count + % labels on top of each bar
for bar, count, pct in zip(bars, counts, pcts):
    ax.text(
        bar.get_x() + bar.get_width() / 2,
```

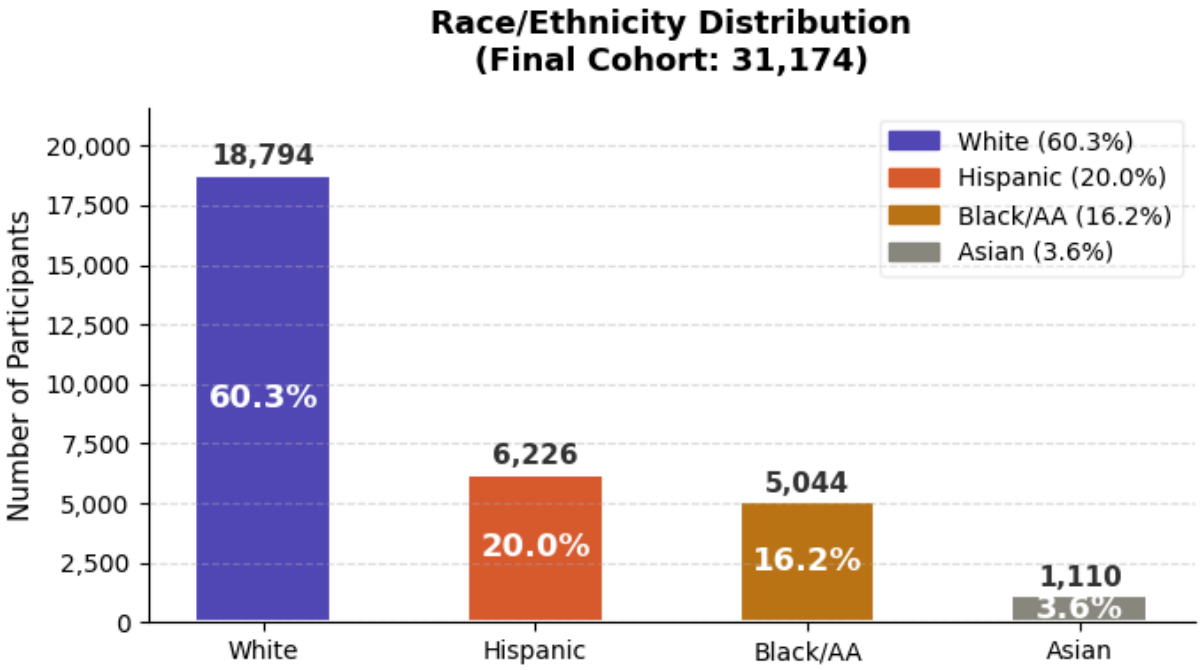
```
        bar.get_height() + 300,
        f'{count:,}',
        ha='center', va='bottom',
        fontsize=12, fontweight='bold', color='#333333'
    )
    ax.text(
        bar.get_x() + bar.get_width() / 2,
        bar.get_height() / 2,
        f'{pct}%',
        ha='center', va='center',
        fontsize=14, fontweight='bold', color='white'
    )

# Styling
ax.set_title('Sex at Birth Distribution\n(Final Cohort: 31,174)', fontsize=13, font
ax.set_ylabel('Number of Participants', fontsize=11)
ax.set_ylim(0, max(counts) * 1.15)
ax.spines['top'].set_visible(False)
ax.spines['right'].set_visible(False)
ax.yaxis.set_major_formatter(plt.FuncFormatter(lambda x, _: f'{int(x):,}'))
ax.grid(axis='y', linestyle='--', alpha=0.4)

# Legend
patches = [mpatches.Patch(color=c, label=f'{l} ({p}%)') for c, l, p in zip(colors,
ax.legend(handles=patches, loc='upper right', fontsize=10, framealpha=0.3)

plt.tight_layout()
plt.savefig('sex_distribution.png', dpi=150, bbox_inches='tight')
plt.show()
```



In [63]:
```
import matplotlib.pyplot as plt
import matplotlib.patches as mpatches
```

```
import os
import pandas as pd

# bucket = os.getenv('M2_WORKSPACE_BUCKET')
# !gsutil cp {bucket}/data/final_cleaned.csv .
# final_cleaned = pd.read_csv('final_cleaned.csv')
# print(f"Loaded! Shape: {final_cleaned.shape}")

# Data - from final_cleaned cohort
race_counts = final_cleaned['race2'].value_counts()
labels = ['White', 'Hispanic', 'Black/AA', 'Asian']
counts = [race_counts.get('White', 0),
          race_counts.get('Hispanic', 0),
          race_counts.get('Black or African American', 0),
          race_counts.get('Asian', 0)]
total = sum(counts)
pcts = [round((c / total) * 100, 1) for c in counts]
colors = ['#534AB7', '#D85A30', '#BA7517', '#888780']

fig, ax = plt.subplots(figsize=(7, 4))

bars = ax.bar(labels, counts, color=colors, width=0.5, edgecolor='white', linewidth

# Count on top, % inside bar
for bar, count, pct in zip(bars, counts, pcts):
    ax.text(
        bar.get_x() + bar.get_width() / 2,
        bar.get_height() + 200,
        f'{count:,}',
        ha='center', va='bottom',
        fontsize=11, fontweight='bold', color='#333333'
    )
    ax.text(
        bar.get_x() + bar.get_width() / 2,
        bar.get_height() / 2,
        f'{pct}%',
        ha='center', va='center',
        fontsize=13, fontweight='bold', color='white'
    )

# Styling
ax.set_title('Race/Ethnicity Distribution\n(Final Cohort: 31,174)', fontsize=13, fo
ax.set_ylabel('Number of Participants', fontsize=11)
ax.set_ylim(0, max(counts) * 1.15)
ax.spines['top'].set_visible(False)
ax.spines['right'].set_visible(False)
ax.yaxis.set_major_formatter(plt.FuncFormatter(lambda x, _: f'{int(x):,}'))
ax.grid(axis='y', linestyle='--', alpha=0.4)

# Legend
patches = [mpatches.Patch(color=c, label=f'{l} ({p}%)')
           for c, l, p in zip(colors, labels, pcts)]
ax.legend(handles=patches, loc='upper right', fontsize=10, framealpha=0.3)

plt.tight_layout()
```

```
# plt.savefig('race_distribution.png', dpi=150, bbox_inches='tight')
plt.show()
```



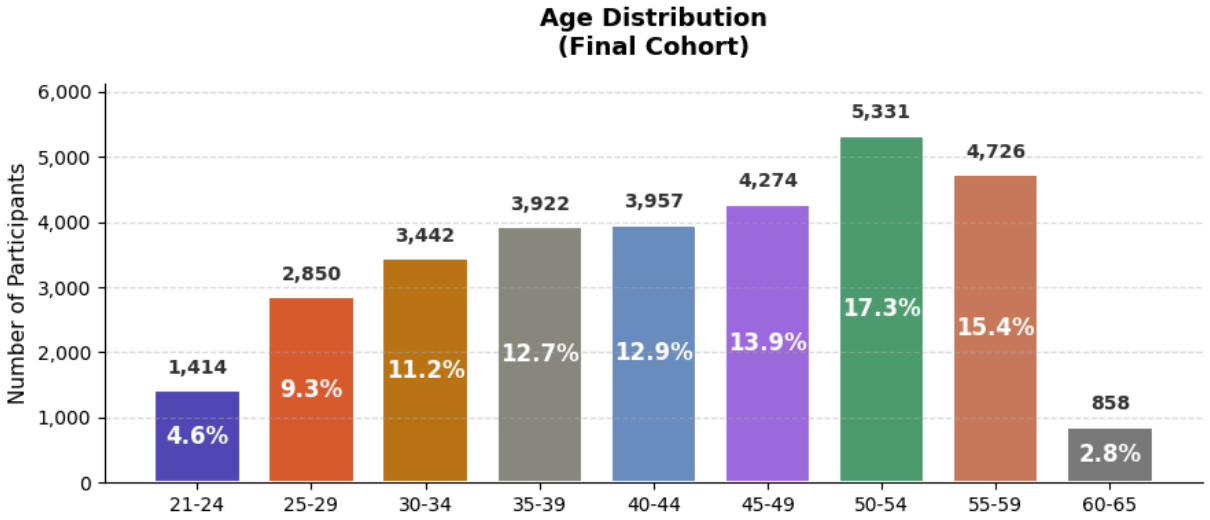
In [35]:
```
final_cleaned.age.describe()
```

Out[35]:
```
count    31174.000000
mean        42.897254
std         11.018650
min         18.000000
25%         34.000000
50%         44.000000
75%         53.000000
max         63.000000
Name: age, dtype: float64
```

In [59]:
```
import matplotlib.pyplot as plt
import pandas as pd

# ----------------------------
# Age binning (your scheme)
# ----------------------------
bins = [21, 25, 30, 35, 40, 45, 50, 55, 60, 66]
labels = ['21-24','25-29','30-34','35-39','40-44','45-49',
          '50-54','55-59','60-65']

final_cleaned = final_cleaned.copy()

final_cleaned['age_group'] = pd.cut(
    final_cleaned['age'],
    bins=bins,
    labels=labels,
    right=False,
    include_lowest=True
)
```

```
# ----------------------------
# Counts + percentages
# ----------------------------
age_counts = final_cleaned['age_group'].value_counts().reindex(labels)
counts = age_counts.values

total = sum(counts)
pcts = [round((c / total) * 100, 1) for c in counts]

colors = ['#534AB7', '#D85A30', '#BA7517', '#888780', '#6C8EBF',
          '#9C6ADE', '#4C9F70', '#C97C5D', '#7A7A7A']

# ----------------------------
# Plot
# ----------------------------
fig, ax = plt.subplots(figsize=(9, 4))

bars = ax.bar(
    labels,
    counts,
    color=colors,
    width=0.75,              # wider bars
    edgecolor='white',
    linewidth=1.5
)

# ----------------------------
# Labels (count + %)
# ----------------------------
for bar, count, pct in zip(bars, counts, pcts):
    ax.text(
        bar.get_x() + bar.get_width() / 2,
        bar.get_height() + 200,
        f'{int(count):,}',
        ha='center', va='bottom',
        fontsize=10, fontweight='bold', color='#333333'
    )
    ax.text(
        bar.get_x() + bar.get_width() / 2,
        bar.get_height() / 2,
        f'{pct}%',
        ha='center', va='center',
        fontsize=12, fontweight='bold', color='white'
    )

# ----------------------------
# Styling
# ----------------------------
ax.set_title('Age Distribution\n(Final Cohort)', fontsize=13, fontweight='bold', pa
ax.set_ylabel('Number of Participants', fontsize=11)
ax.set_ylim(0, max(counts) * 1.15)

ax.spines['top'].set_visible(False)
ax.spines['right'].set_visible(False)
ax.yaxis.set_major_formatter(plt.FuncFormatter(lambda x, _: f'{int(x):,}'))
ax.grid(axis='y', linestyle='--', alpha=0.4)
```

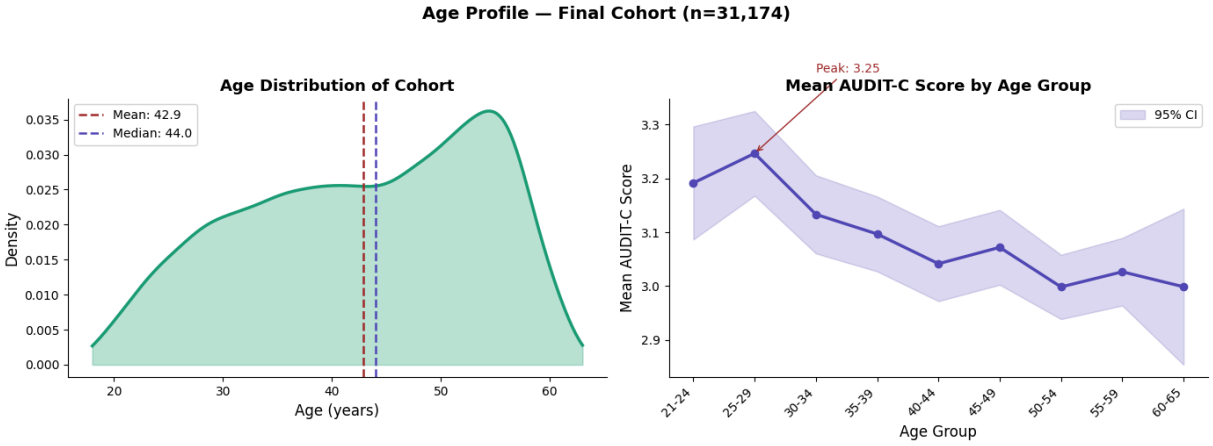
```
# No legend (removed)

plt.tight_layout()
plt.show()
```



In [50]:
```
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
from scipy.stats import gaussian_kde

fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# --- Plot 1: Age distribution (KDE) ---
age_data = final_cleaned['age'].dropna()
kde = gaussian_kde(age_data, bw_method=0.15)
x_range = np.linspace(age_data.min(), age_data.max(), 300)

axes[0].fill_between(x_range, kde(x_range), alpha=0.3, color='#1D9E75')
axes[0].plot(x_range, kde(x_range), color='#1D9E75', linewidth=2.5)

mean_age = age_data.mean()
median_age = age_data.median()

axes[0].axvline(mean_age, color='#A32D2D', linestyle='--', linewidth=1.8,
                label=f'Mean: {mean_age:.1f}')
axes[0].axvline(median_age, color='#534AB7', linestyle='--', linewidth=1.8,
                label=f'Median: {median_age:.1f}')

axes[0].set_xlabel('Age (years)', fontsize=12)
axes[0].set_ylabel('Density', fontsize=12)
axes[0].set_title('Age Distribution of Cohort', fontsize=13, fontweight='bold')
axes[0].legend(fontsize=10)
axes[0].spines['top'].set_visible(False)
axes[0].spines['right'].set_visible(False)

# --- Plot 2: Age vs AUDIT-C ---
bins = [21, 25, 30, 35, 40, 45, 50, 55, 60, 66]
labels = ['21-24','25-29','30-34','35-39','40-44','45-49',
```

```
          '50-54','55-59','60-65']

final_cleaned['age_group'] = pd.cut(
    final_cleaned['age'],
    bins=bins,
    labels=labels,
    right=False
)

grouped = (
    final_cleaned
    .groupby('age_group', observed=True)['audit_c']
    .agg(['mean','std','count'])
    .reset_index()
)

# Compute CI
grouped['se'] = grouped['std'] / np.sqrt(grouped['count'])
grouped['ci'] = 1.96 * grouped['se']

# X positions based on ACTUAL groups (fix)
x = range(len(grouped))

axes[1].plot(x, grouped['mean'],
             color='#534AB7', linewidth=2.5,
             marker='o', markersize=6, zorder=3)

axes[1].fill_between(x,
                     grouped['mean'] - grouped['ci'],
                     grouped['mean'] + grouped['ci'],
                     alpha=0.2, color='#534AB7', label='95% CI')

# ✅ FIX: match ticks + labels to grouped data
axes[1].set_xticks(x)
axes[1].set_xticklabels(grouped['age_group'].astype(str),
                        rotation=45, ha='right', fontsize=10)

axes[1].set_xlabel('Age Group', fontsize=12)
axes[1].set_ylabel('Mean AUDIT-C Score', fontsize=12)
axes[1].set_title('Mean AUDIT-C Score by Age Group',
                  fontsize=13, fontweight='bold')
axes[1].legend(fontsize=10)
axes[1].spines['top'].set_visible(False)
axes[1].spines['right'].set_visible(False)

# Annotate peak
peak_idx = grouped['mean'].idxmax()
axes[1].annotate(f"Peak: {grouped.loc[peak_idx, 'mean']:.2f}",
                 xy=(peak_idx, grouped.loc[peak_idx, 'mean']),
                 xytext=(peak_idx + 1, grouped.loc[peak_idx, 'mean'] + 0.15),
                 arrowprops=dict(arrowstyle='->', color='#A32D2D'),
                 fontsize=10, color='#A32D2D')

plt.suptitle('Age Profile — Final Cohort (n=31,174)',
             fontsize=14, fontweight='bold', y=1.02)
```

```
plt.tight_layout()
plt.savefig('age_audit_distribution.png', dpi=150, bbox_inches='tight')
plt.show()

print(f"Mean age: {mean_age:.1f} | Median age: {median_age:.1f}")
print("\nAUDIT-C by age group:")
print(grouped[['age_group','mean','count']].to_string(index=False))
```



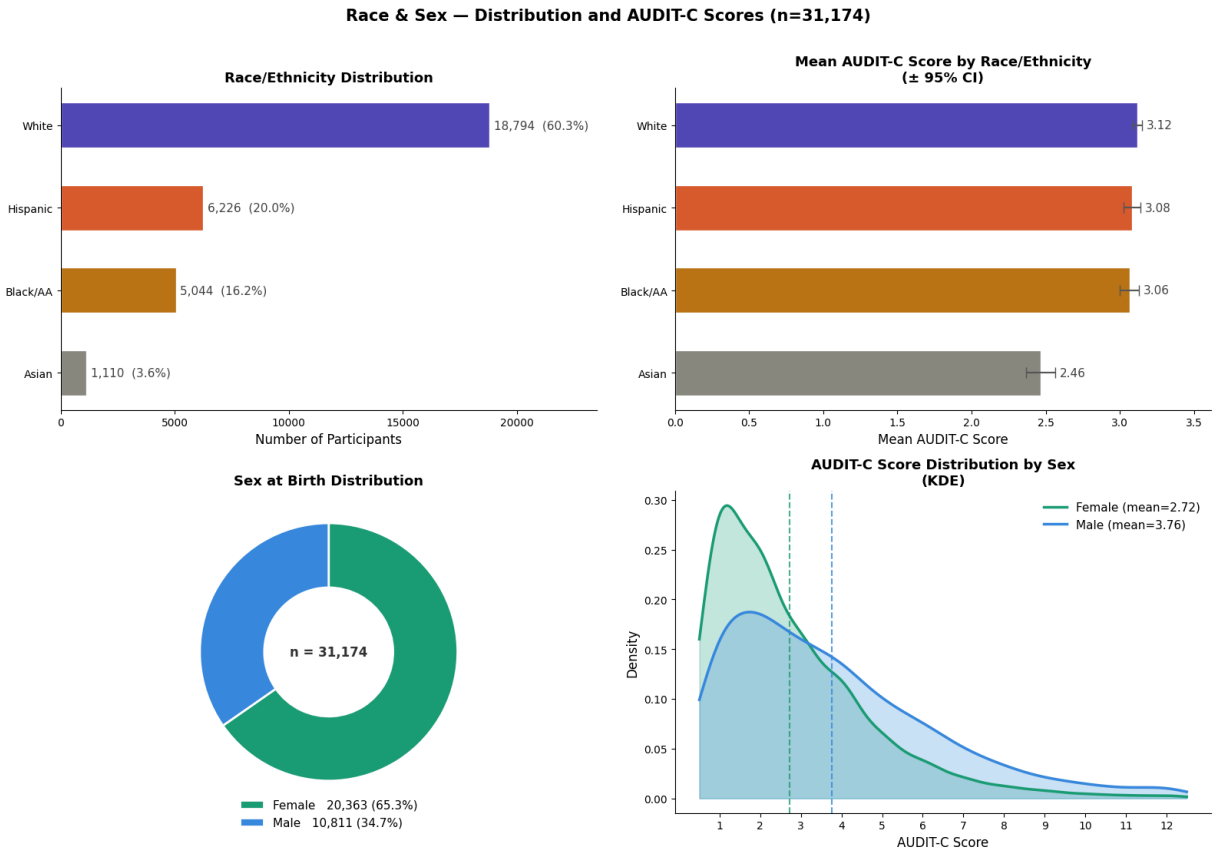

```
Mean age: 42.9 | Median age: 44.0

AUDIT-C by age group:
age_group     mean  count
    21-24 3.191655   1414
    25-29 3.246667   2850
    30-34 3.133353   3442
    35-39 3.096889   3922
    40-44 3.041698   3957
    45-49 3.072064   4274
    50-54 2.998499   5331
    55-59 3.026661   4726
    60-65 2.998834    858
```

In [51]:

```
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd

fig, axes = plt.subplots(2, 2, figsize=(16, 11))

# ── helper to remove top/right spines ──
def clean_ax(ax):
    ax.spines['top'].set_visible(False)
    ax.spines['right'].set_visible(False)

# ═══════════════════════════════════════
# ROW 1 — RACE
# ═══════════════════════════════════════

race_order  = ['White', 'Hispanic', 'Black or African American', 'Asian']
short_labels = ['White', 'Hispanic', 'Black/AA', 'Asian']
race_colors  = ['#534AB7', '#D85A30', '#BA7517', '#888780']

# -- Plot 1a: Race distribution - horizontal bar --
race_counts = final_cleaned['race2'].value_counts()
```

```
counts = [race_counts.get(r, 0) for r in race_order]
pcts   = [c / sum(counts) * 100 for c in counts]

bars = axes[0, 0].barh(short_labels, counts, color=race_colors,
                       edgecolor='white', height=0.55)

for bar, count, pct in zip(bars, counts, pcts):
    axes[0, 0].text(bar.get_width() + 200, bar.get_y() + bar.get_height() / 2,
                    f'{count:,}  ({pct:.1f}%)',
                    va='center', fontsize=11, color='#444')

axes[0, 0].set_xlabel('Number of Participants', fontsize=12)
axes[0, 0].set_title('Race/Ethnicity Distribution', fontsize=13, fontweight='bold')
axes[0, 0].invert_yaxis()
axes[0, 0].set_xlim(0, max(counts) * 1.25)
clean_ax(axes[0, 0])

# -- Plot 1b: Mean AUDIT-C by race with error bars --
race_stats = (final_cleaned.groupby('race2', observed=True)['audit_c']
                           .agg(['mean', 'std', 'count'])
                           .reindex(race_order)
                           .reset_index())
race_stats['ci'] = 1.96 * race_stats['std'] / np.sqrt(race_stats['count'])

axes[0, 1].barh(short_labels, race_stats['mean'],
                xerr=race_stats['ci'], color=race_colors,
                edgecolor='white', height=0.55,
                error_kw=dict(ecolor='#555', capsize=5, linewidth=1.5))

for i, (mean, ci) in enumerate(zip(race_stats['mean'], race_stats['ci'])):
    axes[0, 1].text(mean + ci + 0.03, i, f'{mean:.2f}',
                    va='center', fontsize=11, color='#444')

axes[0, 1].set_xlabel('Mean AUDIT-C Score', fontsize=12)
axes[0, 1].set_title('Mean AUDIT-C Score by Race/Ethnicity\n(± 95% CI)',
                     fontsize=13, fontweight='bold')
axes[0, 1].invert_yaxis()
axes[0, 1].set_xlim(0, race_stats['mean'].max() + race_stats['ci'].max() + 0.4)
clean_ax(axes[0, 1])

# ═══════════════════════════════════════
# ROW 2 — SEX
# ═══════════════════════════════════════

sex_map    = {0: 'Female', 1: 'Male'}
sex_colors = {'Female': '#1D9E75', 'Male': '#378ADD'}
final_cleaned['sex_label'] = final_cleaned['sex_at_birth_male'].map(sex_map)

# -- Plot 2a: Sex distribution - donut chart --
sex_counts = final_cleaned['sex_label'].value_counts()
sex_labels_ordered = ['Female', 'Male']
sex_vals   = [sex_counts.get(s, 0) for s in sex_labels_ordered]
sex_pcts   = [v / sum(sex_vals) * 100 for v in sex_vals]
colors_sex = [sex_colors[s] for s in sex_labels_ordered]

wedges, _ = axes[1, 0].pie(
```

```
    sex_vals, colors=colors_sex,
    startangle=90, counterclock=False,
    wedgeprops=dict(width=0.5, edgecolor='white', linewidth=2)
)

# Center label
axes[1, 0].text(0, 0, f'n = {sum(sex_vals):,}',
                ha='center', va='center', fontsize=12,
                fontweight='bold', color='#333')

# Custom legend with counts + %
legend_labels = [f'{s}   {sex_vals[i]:,} ({sex_pcts[i]:.1f}%)'
                 for i, s in enumerate(sex_labels_ordered)]
axes[1, 0].legend(wedges, legend_labels, loc='lower center',
                  bbox_to_anchor=(0.5, -0.08), fontsize=11, frameon=False)
axes[1, 0].set_title('Sex at Birth Distribution', fontsize=13, fontweight='bold')

# -- Plot 2b: AUDIT-C distribution by sex - KDE overlay --
from scipy.stats import gaussian_kde

for sex, color in sex_colors.items():
    data = final_cleaned[final_cleaned['sex_label'] == sex]['audit_c'].dropna()
    # jitter the discrete scores slightly for smoother KDE
    jittered = data + np.random.normal(0, 0.15, size=len(data))
    kde  = gaussian_kde(jittered, bw_method=0.25)
    x    = np.linspace(0.5, 12.5, 300)
    y    = kde(x)
    axes[1, 1].fill_between(x, y, alpha=0.25, color=color)
    axes[1, 1].plot(x, y, color=color, linewidth=2.5,
                    label=f'{sex} (mean={data.mean():.2f})')
    axes[1, 1].axvline(data.mean(), color=color,
                       linestyle='--', linewidth=1.5, alpha=0.8)

axes[1, 1].set_xlabel('AUDIT-C Score', fontsize=12)
axes[1, 1].set_ylabel('Density', fontsize=12)
axes[1, 1].set_title('AUDIT-C Score Distribution by Sex\n(KDE)',
                     fontsize=13, fontweight='bold')
axes[1, 1].set_xticks(range(1, 13))
axes[1, 1].legend(fontsize=11, frameon=False)
clean_ax(axes[1, 1])

plt.suptitle('Race & Sex — Distribution and AUDIT-C Scores (n=31,174)',
             fontsize=15, fontweight='bold', y=1.01)
plt.tight_layout()
plt.savefig('race_sex_auditc.png', dpi=150, bbox_inches='tight')
plt.show()

# ── Summary stats ──
print("=== Mean AUDIT-C by Race ===")
for _, row in race_stats.iterrows():
    print(f"  {row['race2']:<30} mean={row['mean']:.2f}  n={int(row['count']):,}")

print("\n=== Mean AUDIT-C by Sex ===")
for sex in sex_labels_ordered:
    d = final_cleaned[final_cleaned['sex_label'] == sex]['audit_c']
    print(f"  {sex:<10} mean={d.mean():.2f}  n={len(d):,}")
```




```
=== Mean AUDIT-C by Race ===
  White                          mean=3.12  n=18,794
  Hispanic                       mean=3.08  n=6,226
  Black or African American      mean=3.06  n=5,044
  Asian                          mean=2.46  n=1,110

=== Mean AUDIT-C by Sex ===
  Female     mean=2.72  n=20,363
  Male       mean=3.76  n=10,811
```

In [52]:

```python
import matplotlib.pyplot as plt
import numpy as np
from scipy.stats import gaussian_kde

fig, axes = plt.subplots(2, 3, figsize=(20, 12))

def clean_ax(ax):
    ax.spines['top'].set_visible(False)
    ax.spines['right'].set_visible(False)

# ═══════════════════════════════════════════
# PULL EVERYTHING FROM final_cleaned
# ═══════════════════════════════════════════

total = len(final_cleaned)

# Comorbidities — from final_cleaned columns
hyp_count  = int(final_cleaned['has_hypertension'].sum())
anx_count  = int(final_cleaned['has_anxiety'].sum())
dep_count  = int(final_cleaned['has_depression'].sum())
liv_count  = int(final_cleaned['has_liver_disease'].sum())
```

```
# Medication — from final_cleaned
med_count     = int(final_cleaned['alcoholism_medication'].sum())
no_med_count  = total - med_count

# Smoking — from final_cleaned
smoke_count     = int(final_cleaned['current_smoker'].sum())
no_smoke_count  = total - smoke_count

# At-risk drinking threshold (AUDIT-C >= 3)
final_cleaned['at_risk'] = (final_cleaned['audit_c'] >= 3).astype(int)
at_risk_count     = int(final_cleaned['at_risk'].sum())
not_at_risk_count = total - at_risk_count

print("=== Counts from final_cleaned ===")
print(f"Total:           {total:,}")
print(f"Hypertension:    {hyp_count:,}  ({hyp_count/total*100:.1f}%)")
print(f"Anxiety:         {anx_count:,}  ({anx_count/total*100:.1f}%)")
print(f"Depression:      {dep_count:,}  ({dep_count/total*100:.1f}%)")
print(f"Liver disease:   {liv_count:,}  ({liv_count/total*100:.1f}%)")
print(f"On medication:   {med_count:,}  ({med_count/total*100:.1f}%)")
print(f"Current smoker:  {smoke_count:,}  ({smoke_count/total*100:.1f}%)")
print(f"At-risk drinker: {at_risk_count:,}  ({at_risk_count/total*100:.1f}%)")

# ═══════════════════════════════════════
# ROW 1 — DISTRIBUTIONS
# ═══════════════════════════════════════

# ── Plot 1a: Comorbidities — lollipop ──
comorbidity_labels = ['Hypertension', 'Anxiety', 'Depression', 'Liver Disease']
comorbidity_counts = [hyp_count, anx_count, dep_count, liv_count]
comorbidity_colors = ['#534AB7', '#D85A30', '#BA7517', '#A32D2D']
comorbidity_pcts   = [c / total * 100 for c in comorbidity_counts]

y_pos = range(len(comorbidity_labels))

for i, (count, color) in enumerate(zip(comorbidity_counts, comorbidity_colors)):
    axes[0, 0].hlines(i, 0, count, color=color, alpha=0.35, linewidth=2.5)
    axes[0, 0].scatter(count, i, color=color, s=180, zorder=3)

for i, (count, pct) in enumerate(zip(comorbidity_counts, comorbidity_pcts)):
    axes[0, 0].text(count + max(comorbidity_counts)*0.02, i,
                    f'{count:,}  ({pct:.1f}%)',
                    va='center', fontsize=11, color='#444')

axes[0, 0].set_yticks(y_pos)
axes[0, 0].set_yticklabels(comorbidity_labels, fontsize=12)
axes[0, 0].set_xlabel('Number of Participants', fontsize=12)
axes[0, 0].set_title('Comorbidity Prevalence\nin Final Cohort',
                     fontsize=13, fontweight='bold')
axes[0, 0].set_xlim(0, max(comorbidity_counts) * 1.35)
axes[0, 0].invert_yaxis()
clean_ax(axes[0, 0])

# ── Plot 1b: Medication — donut ──
med_vals   = [med_count, no_med_count]
```

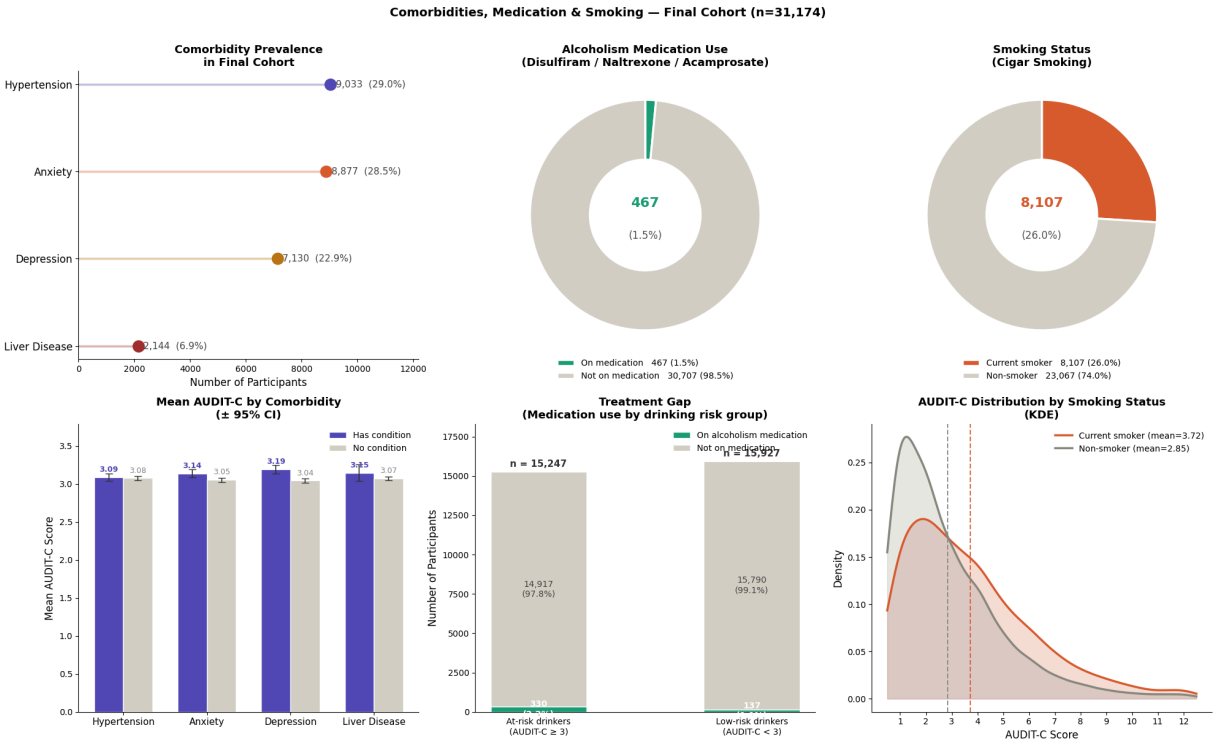
```
med_pcts   = [v / total * 100 for v in med_vals]
med_colors = ['#1D9E75', '#D3D1C7']
med_labels = ['On medication', 'Not on medication']

wedges1, _ = axes[0, 1].pie(
    med_vals, colors=med_colors,
    startangle=90, counterclock=False,
    wedgeprops=dict(width=0.52, edgecolor='white', linewidth=2.5)
)
axes[0, 1].text(0, 0.1, f'{med_count:,}',
                ha='center', va='center', fontsize=16,
                fontweight='bold', color='#1D9E75')
axes[0, 1].text(0, -0.18, f'({med_pcts[0]:.1f}%)',
                ha='center', va='center', fontsize=12, color='#555')

legend_labels1 = [f'{med_labels[i]}   {med_vals[i]:,} ({med_pcts[i]:.1f}%)'
                  for i in range(2)]
axes[0, 1].legend(wedges1, legend_labels1, loc='lower center',
                  bbox_to_anchor=(0.5, -0.1), fontsize=10, frameon=False)
axes[0, 1].set_title('Alcoholism Medication Use\n(Disulfiram / Naltrexone / Acampro
                     fontsize=13, fontweight='bold')

# ── Plot 1c: Smoking — donut ──
smoke_vals   = [smoke_count, no_smoke_count]
smoke_pcts   = [v / total * 100 for v in smoke_vals]
smoke_colors = ['#D85A30', '#D3D1C7']
smoke_labels = ['Current smoker', 'Non-smoker']

wedges2, _ = axes[0, 2].pie(
    smoke_vals, colors=smoke_colors,
    startangle=90, counterclock=False,
    wedgeprops=dict(width=0.52, edgecolor='white', linewidth=2.5)
)
axes[0, 2].text(0, 0.1, f'{smoke_count:,}',
                ha='center', va='center', fontsize=16,
                fontweight='bold', color='#D85A30')
axes[0, 2].text(0, -0.18, f'({smoke_pcts[0]:.1f}%)',
                ha='center', va='center', fontsize=12, color='#555')

legend_labels2 = [f'{smoke_labels[i]}   {smoke_vals[i]:,} ({smoke_pcts[i]:.1f}%)'
                  for i in range(2)]
axes[0, 2].legend(wedges2, legend_labels2, loc='lower center',
                  bbox_to_anchor=(0.5, -0.1), fontsize=10, frameon=False)
axes[0, 2].set_title('Smoking Status\n(Cigar Smoking)',
                     fontsize=13, fontweight='bold')

# ═══════════════════════════════════════
# ROW 2 — RELATIONSHIP WITH AUDIT-C
# ═══════════════════════════════════════

# ── Plot 2a: Mean AUDIT-C by comorbidity presence — grouped bars ──
comorbidity_cols = ['has_hypertension', 'has_anxiety', 'has_depression', 'has_liver

means_with    = []
means_without = []
cis_with      = []
```

```
cis_without   = []

for col in comorbidity_cols:
    grp_with    = final_cleaned[final_cleaned[col] == 1]['audit_c']
    grp_without = final_cleaned[final_cleaned[col] == 0]['audit_c']
    means_with.append(grp_with.mean())
    means_without.append(grp_without.mean())
    cis_with.append(1.96 * grp_with.std() / np.sqrt(len(grp_with)))
    cis_without.append(1.96 * grp_without.std() / np.sqrt(len(grp_without)))

x     = np.arange(len(comorbidity_labels))
width = 0.35

bars1 = axes[1, 0].bar(x - width/2, means_with, width,
                       label='Has condition', color='#534AB7',
                       yerr=cis_with, capsize=4,
                       error_kw=dict(ecolor='#333', linewidth=1.2),
                       edgecolor='white')
bars2 = axes[1, 0].bar(x + width/2, means_without, width,
                       label='No condition', color='#D3D1C7',
                       yerr=cis_without, capsize=4,
                       error_kw=dict(ecolor='#333', linewidth=1.2),
                       edgecolor='white')

for bar in bars1:
    axes[1, 0].text(bar.get_x() + bar.get_width()/2,
                    bar.get_height() + 0.07,
                    f'{bar.get_height():.2f}',
                    ha='center', fontsize=9,
                    color='#534AB7', fontweight='bold')
for bar in bars2:
    axes[1, 0].text(bar.get_x() + bar.get_width()/2,
                    bar.get_height() + 0.07,
                    f'{bar.get_height():.2f}',
                    ha='center', fontsize=9, color='#888')

axes[1, 0].set_xticks(x)
axes[1, 0].set_xticklabels(comorbidity_labels, fontsize=11)
axes[1, 0].set_ylabel('Mean AUDIT-C Score', fontsize=12)
axes[1, 0].set_title('Mean AUDIT-C by Comorbidity\n(± 95% CI)',
                     fontsize=13, fontweight='bold')
axes[1, 0].legend(fontsize=10, frameon=False)
axes[1, 0].set_ylim(0, max(means_with) + 0.6)
clean_ax(axes[1, 0])

# ── Plot 2b: Treatment gap — stacked bar ──
at_risk_on_med     = int(final_cleaned[
    (final_cleaned['at_risk'] == 1) &
    (final_cleaned['alcoholism_medication'] == 1)].shape[0])
at_risk_not_on_med = at_risk_count - at_risk_on_med

low_risk_on_med    = int(final_cleaned[
    (final_cleaned['at_risk'] == 0) &
    (final_cleaned['alcoholism_medication'] == 1)].shape[0])
low_risk_not_on_med = not_at_risk_count - low_risk_on_med
```

```
categories   = ['At-risk drinkers\n(AUDIT-C ≥ 3)', 'Low-risk drinkers\n(AUDIT-C < 3
on_med_vals  = [at_risk_on_med, low_risk_on_med]
off_med_vals = [at_risk_not_on_med, low_risk_not_on_med]
totals       = [at_risk_count, not_at_risk_count]

b1 = axes[1, 1].bar(categories, on_med_vals,
                    color='#1D9E75', label='On alcoholism medication',
                    edgecolor='white', width=0.45)
b2 = axes[1, 1].bar(categories, off_med_vals, bottom=on_med_vals,
                    color='#D3D1C7', label='Not on medication',
                    edgecolor='white', width=0.45)

for bar, val, tot in zip(b1, on_med_vals, totals):
    pct = val / tot * 100
    if pct > 0.5:
        axes[1, 1].text(bar.get_x() + bar.get_width()/2,
                        val / 2,
                        f'{val:,}\n({pct:.1f}%)',
                        ha='center', va='center',
                        fontsize=10, fontweight='bold', color='white')

for bar, bot, val, tot in zip(b2, on_med_vals, off_med_vals, totals):
    pct = val / tot * 100
    axes[1, 1].text(bar.get_x() + bar.get_width()/2,
                    bot + val / 2,
                    f'{val:,}\n({pct:.1f}%)',
                    ha='center', va='center',
                    fontsize=10, color='#444')

for i, tot in enumerate(totals):
    axes[1, 1].text(i, tot + max(totals)*0.02,
                    f'n = {tot:,}',
                    ha='center', fontsize=11,
                    fontweight='bold', color='#333')

axes[1, 1].set_ylabel('Number of Participants', fontsize=12)
axes[1, 1].set_title('Treatment Gap\n(Medication use by drinking risk group)',
                     fontsize=13, fontweight='bold')
axes[1, 1].legend(fontsize=10, frameon=False, loc='upper right')
axes[1, 1].set_ylim(0, max(totals) * 1.15)
clean_ax(axes[1, 1])

# ── Plot 2c: AUDIT-C KDE — smoker vs non-smoker ──
smoker_group    = final_cleaned[final_cleaned['current_smoker'] == 1]['audit_c'].dr
nonsmoker_group = final_cleaned[final_cleaned['current_smoker'] == 0]['audit_c'].dr

for data, label, color in [
    (smoker_group,    f'Current smoker (mean={smoker_group.mean():.2f})',       '#D
    (nonsmoker_group, f'Non-smoker (mean={nonsmoker_group.mean():.2f})',        '#8
]:
    jittered = data + np.random.normal(0, 0.15, size=len(data))
    kde      = gaussian_kde(jittered, bw_method=0.25)
    x_vals   = np.linspace(0.5, 12.5, 300)
    y_vals   = kde(x_vals)
    axes[1, 2].fill_between(x_vals, y_vals, alpha=0.2, color=color)
    axes[1, 2].plot(x_vals, y_vals, color=color, linewidth=2.5, label=label)
```

```
    axes[1, 2].axvline(data.mean(), color=color,
                       linestyle='--', linewidth=1.5, alpha=0.85)

axes[1, 2].set_xlabel('AUDIT-C Score', fontsize=12)
axes[1, 2].set_ylabel('Density', fontsize=12)
axes[1, 2].set_title('AUDIT-C Distribution by Smoking Status\n(KDE)',
                     fontsize=13, fontweight='bold')
axes[1, 2].set_xticks(range(1, 13))
axes[1, 2].legend(fontsize=10, frameon=False)
clean_ax(axes[1, 2])

plt.suptitle('Comorbidities, Medication & Smoking — Final Cohort (n=31,174)',
             fontsize=14, fontweight='bold', y=1.01)
plt.tight_layout()
plt.savefig('comorbidity_medication_smoking.png', dpi=150, bbox_inches='tight')
plt.show()

# ── Final summary printout ──
print("\n=== Mean AUDIT-C by Comorbidity ===")
for col, name in zip(comorbidity_cols, comorbidity_labels):
    w  = final_cleaned[final_cleaned[col] == 1]['audit_c'].mean()
    wo = final_cleaned[final_cleaned[col] == 0]['audit_c'].mean()
    print(f"  {name:<20} with={w:.2f}  without={wo:.2f}")

print(f"\n=== Treatment Gap ===")
print(f"  At-risk drinkers (AUDIT-C ≥ 3):  {at_risk_count:,} ({at_risk_count/total*
print(f"  Of those — on medication:         {at_risk_on_med:,} ({at_risk_on_med/at_
print(f"  Of those — NOT on medication:     {at_risk_not_on_med:,} ({at_risk_not_on

print(f"\n=== AUDIT-C by Smoking ===")
print(f"  Current smoker:  mean={smoker_group.mean():.2f}  n={len(smoker_group):,}"
print(f"  Non-smoker:      mean={nonsmoker_group.mean():.2f}  n={len(nonsmoker_grou
```

```
=== Counts from final_cleaned ===
Total:           31,174
Hypertension:    9,033  (29.0%)
Anxiety:         8,877  (28.5%)
Depression:      7,130  (22.9%)
Liver disease:   2,144  (6.9%)
On medication:   467  (1.5%)
Current smoker:  8,107  (26.0%)
At-risk drinker: 15,247  (48.9%)
```

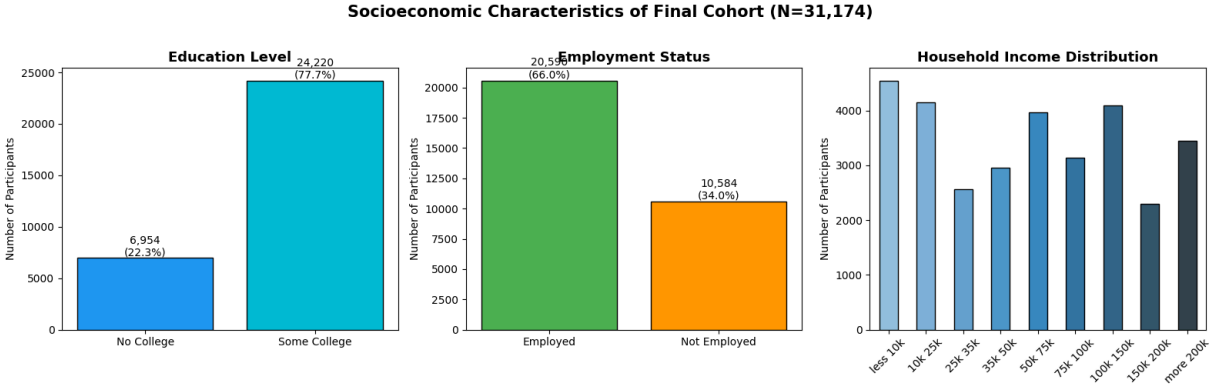



```
=== Mean AUDIT-C by Comorbidity ===
  Hypertension          with=3.09  without=3.08
  Anxiety               with=3.14  without=3.05
  Depression            with=3.19  without=3.04
  Liver Disease         with=3.15  without=3.07

=== Treatment Gap ===
  At-risk drinkers (AUDIT-C ≥ 3):  15,247 (48.9%)
  Of those — on medication:          330 (2.2%)
  Of those — NOT on medication:     14,917 (97.8%)

=== AUDIT-C by Smoking ===
  Current smoker:  mean=3.72  n=8,107
  Non-smoker:      mean=2.85  n=23,067
```

In [53]:

```python
import matplotlib.pyplot as plt
import seaborn as sns
import pandas as pd

fig, axes = plt.subplots(1, 3, figsize=(16, 5))

# Education
edu_labels = ['No College', 'Some College']
edu_counts = [
    (final_cleaned['some_college'] == 0).sum(),
    (final_cleaned['some_college'] == 1).sum()
]
axes[0].bar(edu_labels, edu_counts, color=['#2196F3', '#00BCD4'], edgecolor='black'
axes[0].set_title('Education Level', fontsize=13, fontweight='bold')
axes[0].set_ylabel('Number of Participants')
for i, v in enumerate(edu_counts):
    axes[0].text(i, v + 200, f'{v:,}\n({v/len(final_cleaned)*100:.1f}%)',
                 ha='center', fontsize=10)
```

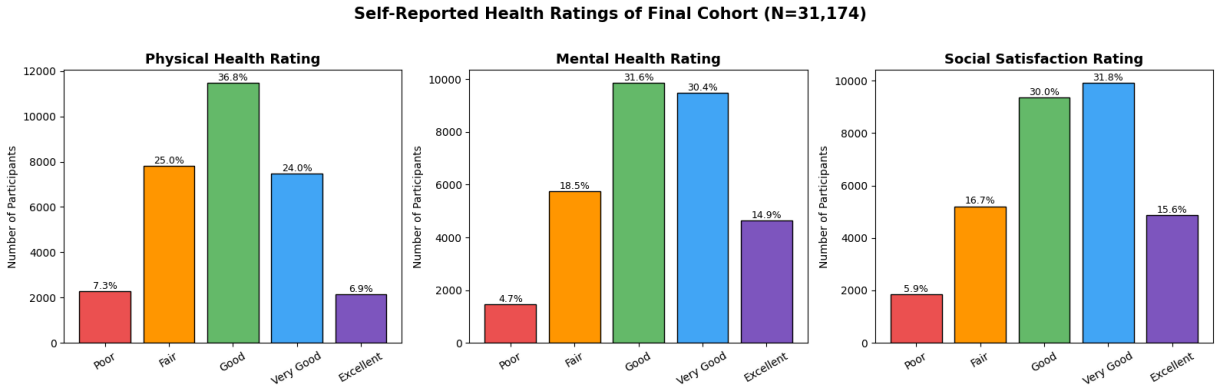
```
# Employment
emp_labels = ['Employed', 'Not Employed']
emp_counts = [
    (final_cleaned['employment_status'] == 1).sum(),
    (final_cleaned['employment_status'] == 0).sum()
]
axes[1].bar(emp_labels, emp_counts, color=['#4CAF50', '#FF9800'], edgecolor='black'
axes[1].set_title('Employment Status', fontsize=13, fontweight='bold')
axes[1].set_ylabel('Number of Participants')
for i, v in enumerate(emp_counts):
    axes[1].text(i, v + 200, f'{v:,}\n({v/len(final_cleaned)*100:.1f}%)',
                 ha='center', fontsize=10)

# Household Income
income_counts = final_cleaned['household_income'].value_counts()
income_labels = [
    'less 10k', '10k 25k', '25k 35k', '35k 50k',
    '50k 75k', '75k 100k', '100k 150k', '150k 200k', 'more 200k'
]
income_order = [f'Annual Income: {x}' for x in income_labels]
income_filtered = income_counts.reindex(income_order, fill_value=0)
income_filtered.index = income_labels

income_filtered.plot(kind='bar', ax=axes[2],
                     color=sns.color_palette('Blues_d', len(income_filtered)),
                     edgecolor='black')
axes[2].set_title('Household Income Distribution', fontsize=13, fontweight='bold')
axes[2].set_ylabel('Number of Participants')
axes[2].set_xlabel('')
axes[2].tick_params(axis='x', rotation=45)

plt.suptitle('Socioeconomic Characteristics of Final Cohort (N=31,174)',
             fontsize=15, fontweight='bold', y=1.02)
plt.tight_layout()
plt.show()
```



In [54]:
```
fig, axes = plt.subplots(1, 3, figsize=(16, 5))

# Color palette
colors = ['#EF5350', '#FF9800', '#66BB6A', '#42A5F5', '#7E57C2']

# Physical Health
phys_order = [
```

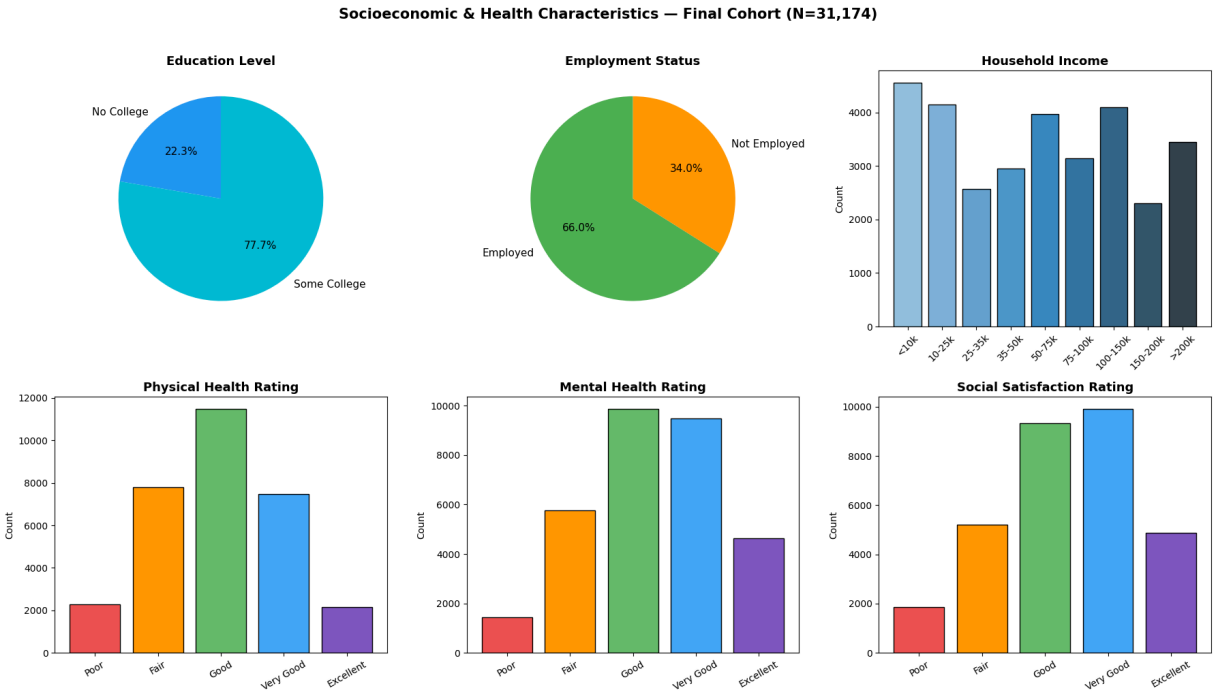
```
    'General Physical Health: Poor',
    'General Physical Health: Fair',
    'General Physical Health: Good',
    'General Physical Health: Very Good',
    'General Physical Health: Excellent'
]
phys_counts = final_cleaned['physical_health'].value_counts().reindex(phys_order, f
phys_labels = ['Poor', 'Fair', 'Good', 'Very Good', 'Excellent']

axes[0].bar(phys_labels, phys_counts.values, color=colors, edgecolor='black')
axes[0].set_title('Physical Health Rating', fontsize=13, fontweight='bold')
axes[0].set_ylabel('Number of Participants')
axes[0].tick_params(axis='x', rotation=30)
for i, v in enumerate(phys_counts.values):
    axes[0].text(i, v + 100, f'{v/len(final_cleaned)*100:.1f}%',
                 ha='center', fontsize=9)

# Mental Health
mental_order = [
    'General Mental Health: Poor',
    'General Mental Health: Fair',
    'General Mental Health: Good',
    'General Mental Health: Very Good',
    'General Mental Health: Excellent'
]
mental_counts = final_cleaned['mental_health'].value_counts().reindex(mental_order,
mental_labels = ['Poor', 'Fair', 'Good', 'Very Good', 'Excellent']

axes[1].bar(mental_labels, mental_counts.values, color=colors, edgecolor='black')
axes[1].set_title('Mental Health Rating', fontsize=13, fontweight='bold')
axes[1].set_ylabel('Number of Participants')
axes[1].tick_params(axis='x', rotation=30)
for i, v in enumerate(mental_counts.values):
    axes[1].text(i, v + 100, f'{v/len(final_cleaned)*100:.1f}%',
                 ha='center', fontsize=9)

# Social Health
social_order = [
    'Social Satisfaction: Poor',
    'Social Satisfaction: Fair',
    'Social Satisfaction: Good',
    'Social Satisfaction: Very Good',
    'Social Satisfaction: Excellent'
]
social_counts = final_cleaned['social_health'].value_counts().reindex(social_order,
social_labels = ['Poor', 'Fair', 'Good', 'Very Good', 'Excellent']

axes[2].bar(social_labels, social_counts.values, color=colors, edgecolor='black')
axes[2].set_title('Social Satisfaction Rating', fontsize=13, fontweight='bold')
axes[2].set_ylabel('Number of Participants')
axes[2].tick_params(axis='x', rotation=30)
for i, v in enumerate(social_counts.values):
    axes[2].text(i, v + 100, f'{v/len(final_cleaned)*100:.1f}%',
                 ha='center', fontsize=9)

plt.suptitle('Self-Reported Health Ratings of Final Cohort (N=31,174)',
```

```
             fontsize=15, fontweight='bold', y=1.02)
plt.tight_layout()
plt.show()
```



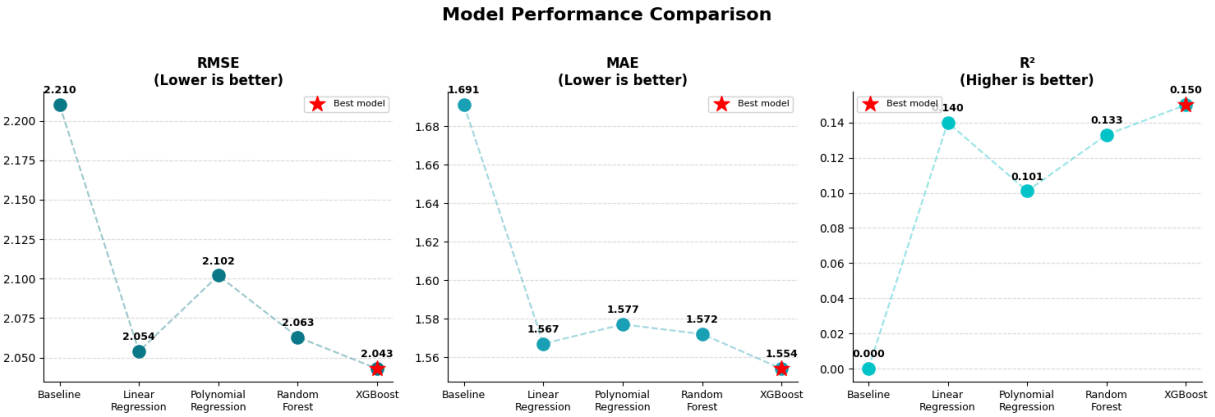
In [55]:
```
fig, axes = plt.subplots(2, 3, figsize=(18, 10))

# Row 1: Socioeconomic
# Education (pie chart)
edu_counts = [
    (final_cleaned['some_college'] == 0).sum(),
    (final_cleaned['some_college'] == 1).sum()
]
axes[0,0].pie(edu_counts, labels=['No College', 'Some College'],
              colors=['#2196F3', '#00BCD4'], autopct='%1.1f%%',
              startangle=90, textprops={'fontsize': 11})
axes[0,0].set_title('Education Level', fontsize=13, fontweight='bold')

# Employment (pie chart)
emp_counts = [
    (final_cleaned['employment_status'] == 1).sum(),
    (final_cleaned['employment_status'] == 0).sum()
]
axes[0,1].pie(emp_counts, labels=['Employed', 'Not Employed'],
              colors=['#4CAF50', '#FF9800'], autopct='%1.1f%%',
              startangle=90, textprops={'fontsize': 11})
axes[0,1].set_title('Employment Status', fontsize=13, fontweight='bold')

# Income (bar)
income_counts = final_cleaned['household_income'].value_counts()
income_labels = ['<10k', '10-25k', '25-35k', '35-50k',
                 '50-75k', '75-100k', '100-150k', '150-200k', '>200k']
income_order = [
    'Annual Income: less 10k', 'Annual Income: 10k 25k',
    'Annual Income: 25k 35k', 'Annual Income: 35k 50k',
    'Annual Income: 50k 75k', 'Annual Income: 75k 100k',
    'Annual Income: 100k 150k', 'Annual Income: 150k 200k',
    'Annual Income: more 200k'
]
income_vals = [income_counts.get(x, 0) for x in income_order]
axes[0,2].bar(income_labels, income_vals,
              color=sns.color_palette('Blues_d', 9), edgecolor='black')
axes[0,2].set_title('Household Income', fontsize=13, fontweight='bold')
axes[0,2].set_ylabel('Count')
axes[0,2].tick_params(axis='x', rotation=45)
```

```
# Row 2: Health Ratings
health_labels = ['Poor', 'Fair', 'Good', 'Very Good', 'Excellent']
colors = ['#EF5350', '#FF9800', '#66BB6A', '#42A5F5', '#7E57C2']

# Physical
phys_order = [
    'General Physical Health: Poor', 'General Physical Health: Fair',
    'General Physical Health: Good', 'General Physical Health: Very Good',
    'General Physical Health: Excellent'
]
phys_vals = [final_cleaned['physical_health'].value_counts().get(x, 0) for x in phy
axes[1,0].bar(health_labels, phys_vals, color=colors, edgecolor='black')
axes[1,0].set_title('Physical Health Rating', fontsize=13, fontweight='bold')
axes[1,0].set_ylabel('Count')
axes[1,0].tick_params(axis='x', rotation=30)

# Mental
mental_order = [
    'General Mental Health: Poor', 'General Mental Health: Fair',
    'General Mental Health: Good', 'General Mental Health: Very Good',
    'General Mental Health: Excellent'
]
mental_vals = [final_cleaned['mental_health'].value_counts().get(x, 0) for x in men
axes[1,1].bar(health_labels, mental_vals, color=colors, edgecolor='black')
axes[1,1].set_title('Mental Health Rating', fontsize=13, fontweight='bold')
axes[1,1].set_ylabel('Count')
axes[1,1].tick_params(axis='x', rotation=30)

# Social
social_order = [
    'Social Satisfaction: Poor', 'Social Satisfaction: Fair',
    'Social Satisfaction: Good', 'Social Satisfaction: Very Good',
    'Social Satisfaction: Excellent'
]
social_vals = [final_cleaned['social_health'].value_counts().get(x, 0) for x in soc
axes[1,2].bar(health_labels, social_vals, color=colors, edgecolor='black')
axes[1,2].set_title('Social Satisfaction Rating', fontsize=13, fontweight='bold')
axes[1,2].set_ylabel('Count')
axes[1,2].tick_params(axis='x', rotation=30)

plt.suptitle('Socioeconomic & Health Characteristics — Final Cohort (N=31,174)',
             fontsize=15, fontweight='bold', y=1.02)
plt.tight_layout()
plt.show()
```



In [47]:
```
import matplotlib.pyplot as plt
import numpy as np

# Model performance data
models = ['Baseline', 'Linear\nRegression', 'Polynomial\nRegression', 'Random\nFore

rmse   = [2.210, 2.054, 2.102, 2.063, 2.043]
mae    = [1.691, 1.567, 1.577, 1.572, 1.554]
r2     = [0.000, 0.140, 0.101, 0.133, 0.150]

x = np.arange(len(models))
fig, axes = plt.subplots(1, 3, figsize=(15, 5))
fig.suptitle('Model Performance Comparison', fontsize=16, fontweight='bold', y=1.02

metrics = [
    (rmse, 'RMSE', 'Lower is better', '#0D7A8A', True),
    (mae,  'MAE',  'Lower is better', '#1AA3B5', True),
    (r2,   'R²',   'Higher is better','#00C4CC', False),
]

for ax, (values, title, subtitle, color, lower_better) in zip(axes, metrics):
    # Plot lines connecting points
    ax.plot(x, values, color=color, linewidth=1.5, linestyle='--', alpha=0.4)

    # Plot points
    ax.scatter(x, values, color=color, s=120, zorder=5)

    # Highlight best model
    best_idx = np.argmin(values) if lower_better else np.argmax(values)
    ax.scatter(x[best_idx], values[best_idx], color='red', s=200, zorder=6, marker=

    # Add value labels
    for i, v in enumerate(values):
        ax.annotate(f'{v:.3f}', (x[i], v),
```

```
                    textcoords="offset points",
                    xytext=(0, 10),
                    ha='center', fontsize=9, fontweight='bold')

    ax.set_title(f'{title}\n({subtitle})', fontsize=12, fontweight='bold')
    ax.set_xticks(x)
    ax.set_xticklabels(models, fontsize=9)
    ax.grid(axis='y', linestyle='--', alpha=0.4)
    ax.spines['top'].set_visible(False)
    ax.spines['right'].set_visible(False)
    ax.legend(fontsize=8)

plt.tight_layout()
plt.savefig('model_performance.png', dpi=150, bbox_inches='tight')
plt.show()
```



In [ ]: